predictive-power-moving-averages

October 29, 2026

0.1 Post Updates

Update 10/29/2026: The analysis has been updated to use month-end data only, incorporates a "base rate" of return for comparison purposes, and updates the conclusions accordingly.

0.2 Introduction

The theory of time series momentum (AKA, trend following) heavily relies on the idea that if the price at time t is higher than the price at time t-x, then the price at time t+y is likely to be higher than the price at time t.

In this post, we will investigate the idea of moving averages, and whether if the price at time t is higher or lower than the moving average up until time t (for several different lookback periods) has any predictive power for forward returns (for several different time horizons).

0.3 Python Imports

[1]:
```
# Standard Library
import os
import sys
import warnings

from pathlib import Path

# Data Handling
import pandas as pd

# Suppress warnings
warnings.filterwarnings("ignore")
```

[2]:
```
# Add the source subdirectory to the system path to allow import config from␣
 ↪settings.py
current_directory = Path(os.getcwd())
website_base_directory = current_directory.parent.parent.parent
src_directory = website_base_directory / "src"
sys.path.append(str(src_directory)) if str(src_directory) not in sys.path else␣
 ↪None

# Import settings.py
```

```
from settings import config

# Add configured directories from config to path
SOURCE_DIR = config("SOURCE_DIR")
sys.path.append(str(Path(SOURCE_DIR))) if str(Path(SOURCE_DIR)) not in sys.path␣
 ↪else None

# Add other configured directories
BASE_DIR = config("BASE_DIR")
CONTENT_DIR = config("CONTENT_DIR")
POSTS_DIR = config("POSTS_DIR")
PAGES_DIR = config("PAGES_DIR")
PUBLIC_DIR = config("PUBLIC_DIR")
SOURCE_DIR = config("SOURCE_DIR")
DATA_DIR = config("DATA_DIR")
DATA_MANUAL_DIR = config("DATA_MANUAL_DIR")
```

0.4 Python Functions

Here are the functions needed for this project:

- load_data: Load data from a CSV, Excel, or Pickle file into a pandas DataFrame.
- pandas_set_decimal_places: Set the number of decimal places displayed for floating-point numbers in pandas.
- plot_heatmap: Plot a heatmap from a DataFrame.
- plot_histogram: Plot the histogram of a data set from a DataFrame.
- plot_scatter: Plot the data from a DataFrame for a specified date range and columns.
- plot_time_series: Plot the timeseries data from a DataFrame for a specified date range and columns.
- run_regression: Run a linear regression using statsmodels OLS and return the results.
- summary_stats: Generate summary statistics for a series of returns.
- yf_pull_data: Download daily price data from Yahoo Finance and export it.

[3]:
```
from load_data import load_data
from pandas_set_decimal_places import pandas_set_decimal_places
from plot_heatmap import plot_heatmap
from plot_histogram import plot_histogram
from plot_scatter import plot_scatter
from plot_time_series import plot_time_series
from run_regression import run_regression
from summary_stats import summary_stats
from yf_pull_data import yf_pull_data
```

0.5 Data Overview

For this exercise, we will investigate the predictive power of moving averages for the following ETFs:

- IVV - iShares Core S&P 500 ETF
- EFA - iShares MSCI EAFE ETF

- EEM - iShares MSCI Emerging Markets ETF
- GSG - iShares S&P GSCI Commodity-Indexed Trust
- IAU - iShares Gold Trust
- IEF - iShares 7-10 Year Treasury Bond ETF
- TLT - iShares 20+ Year Treasury Bond ETF

We'll use the adjusted close prices for each of these ETFs.

0.5.1 Acquire Data

First, let's get the data for these ETFs. If we already have the desired data, we can load it from a local pickle file. Otherwise, we can download it from Yahoo Finance using the `yf_pull_data` function.

[4]:
```
pandas_set_decimal_places(2)

# Create a list of the ETF tickers
fund_list = ["IVV", "EFA", "EEM", "GSG", "IAU", "IEF", "TLT"]
```

[5]:
```
# Pull data for each ETF and cache it locally
for fund in fund_list:
    yf_pull_data(
        base_directory=DATA_DIR,
        ticker=fund,
        adjusted=False,
        source="Yahoo_Finance",
        asset_class="Exchange_Traded_Funds",
        excel_export=True,
        pickle_export=True,
        output_confirmation=False,
    )
```

0.5.2 Create Data Dictionary

Then, create a dictionary to hold the data for each ETF.

[6]:
```
# Create an empty dictionary to hold the data for each ETF
fund_data = {}

# Load the data for each ETF, rename the columns to include the ticker, store␣
 ↪it in the fund_data dictionary
for fund in fund_list:
    data = load_data(
        base_directory=DATA_DIR,
        ticker=fund,
        source="Yahoo_Finance",
        asset_class="Exchange_Traded_Funds",
        timeframe="Daily",
        file_format="pickle",
```

```
    )

    data = data.rename(columns={
        "Adj Close": f"{fund}_Adj_Close",
        "Close": f"{fund}_Close",
        "High": f"{fund}_High",
        "Low": f"{fund}_Low",
        "Open": f"{fund}_Open",
        "Volume": f"{fund}_Volume"
    })

    fund_data[fund] = data
```

0.5.3 Plot Data

Next, we will check the date ranges, and plot the adjusted close prices, cumulative returns, and drawdowns.

[7]:
```
for fund, data in fund_data.items():
    print(f"Fund: {fund}")
    print(f"Date Range: {data.index.min()} - {data.index.max()}")
```

```
Fund: IVV
Date Range: 2000-05-19 00:00:00 - 2026-10-28 00:00:00
Fund: EFA
Date Range: 2001-08-27 00:00:00 - 2026-10-28 00:00:00
Fund: EEM
Date Range: 2003-04-14 00:00:00 - 2026-10-28 00:00:00
Fund: GSG
Date Range: 2006-07-21 00:00:00 - 2026-10-28 00:00:00
Fund: IAU
Date Range: 2005-01-28 00:00:00 - 2026-10-28 00:00:00
Fund: IEF
Date Range: 2002-07-30 00:00:00 - 2026-10-28 00:00:00
Fund: TLT
Date Range: 2002-07-30 00:00:00 - 2026-10-28 00:00:00
```

Our shortest range is GSG, with just over 20 years of data.

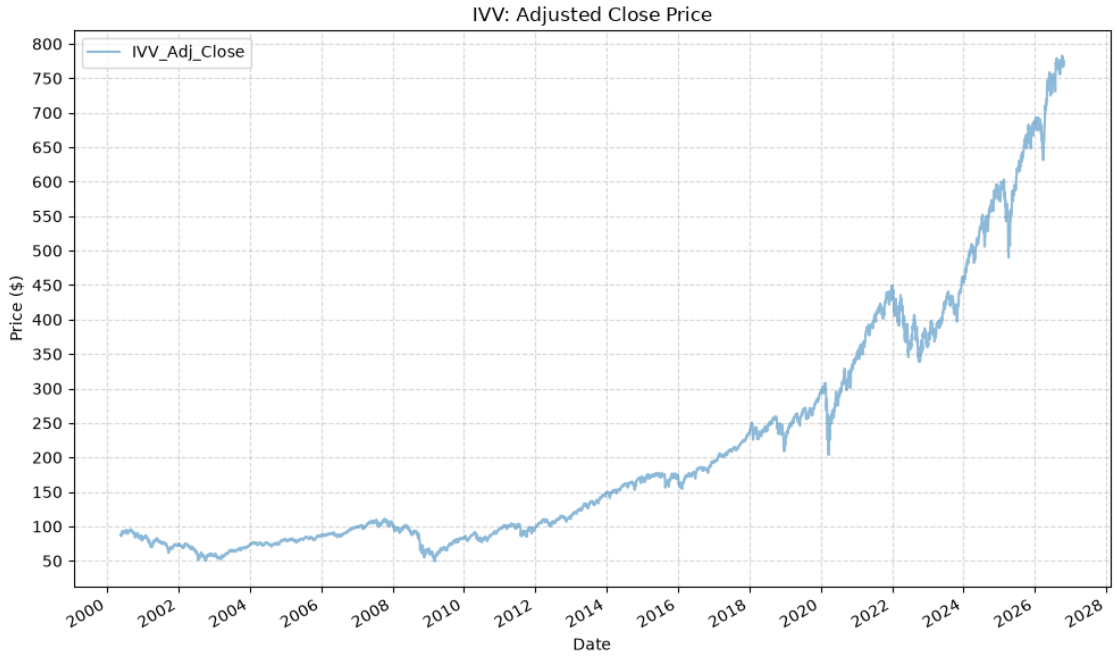
[8]:
```
for fund, data in fund_data.items():
    display(data)

    plot_time_series(
        df=data,
        plot_start_date=None,
        plot_end_date=None,
        plot_columns=[f"{fund}_Adj_Close"],
        title=f"{fund}: Adjusted Close Price",
```

```
        x_label="Date",
        x_format="Year",
        x_tick_spacing=2,
        x_tick_start=None,
        x_tick_rotation=30,
        y_label="Price ($)",
        y_format="Decimal",
        y_format_decimal_places=0,
        y_tick_spacing="Auto",
        y_tick_rotation=0,
    )
```

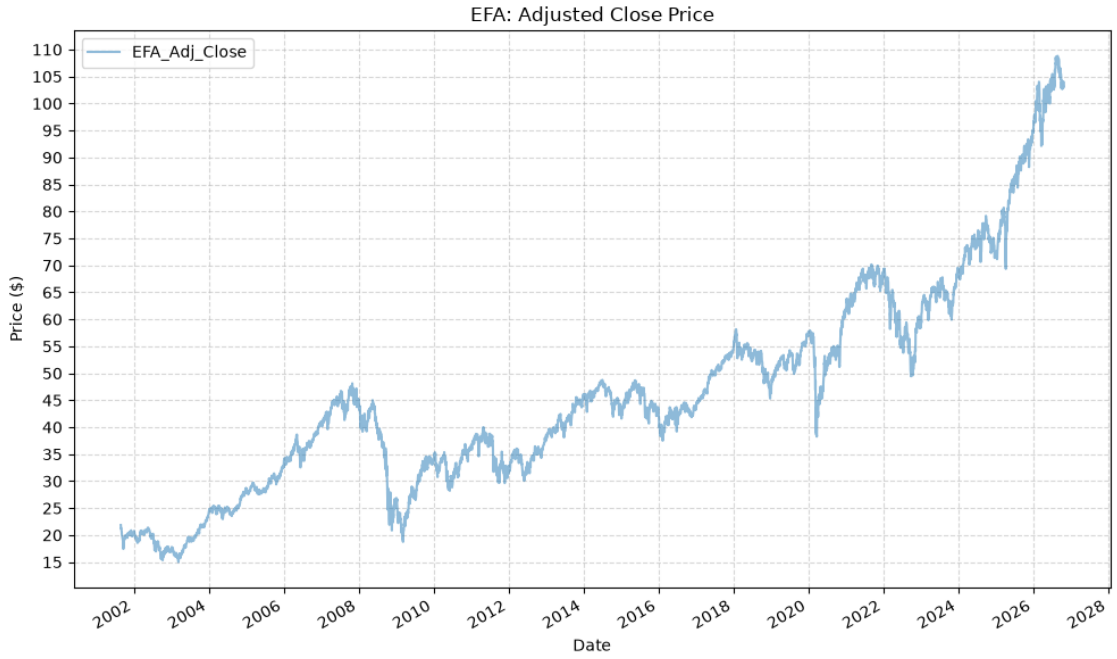
Date	IVV_Adj_Close	IVV_Close	IVV_High	IVV_Low	IVV_Open	IVV_Volume
2000-05-19	88.08	140.69	142.66	140.25	142.66	775500
2000-05-22	87.53	139.81	140.59	136.81	140.59	1850600
2000-05-23	86.20	137.69	140.22	137.69	140.22	373900
2000-05-24	87.49	139.75	140.06	136.66	137.75	400300
2000-05-25	86.69	138.47	140.94	137.88	140.03	69600
…	…	…	…	…	…	…
2026-10-22	767.41	767.41	771.09	764.57	769.71	4262000
2026-10-23	763.25	763.25	766.86	762.04	763.26	4190600
2026-10-26	768.69	768.69	771.12	765.00	765.64	2583600
2026-10-27	773.25	773.25	774.59	768.65	770.84	8862200
2026-10-28	775.65	775.65	776.75	772.69	774.86	2757200

[6650 rows x 6 columns]



Date	EFA_Adj_Close	EFA_Close	EFA_High	EFA_Low	EFA_Open	EFA_Volume
2001-08-27	21.86	42.83	42.92	42.76	42.92	44700
2001-08-28	21.61	42.34	42.58	42.26	42.53	319800
2001-08-29	21.51	42.15	42.47	42.10	42.45	128400
2001-08-30	21.20	41.55	41.67	41.43	41.67	36900
2001-08-31	21.25	41.65	41.72	41.47	41.53	1656900
…	…	…	…	…	…	…
2026-10-22	103.15	103.15	103.47	102.79	103.26	11508000
2026-10-23	102.63	102.63	102.82	102.09	102.25	9997900
2026-10-26	102.91	102.91	103.43	102.89	103.12	9119600
2026-10-27	103.59	103.59	103.71	102.97	103.12	9909100
2026-10-28	103.68	103.68	103.82	103.09	103.34	12880500

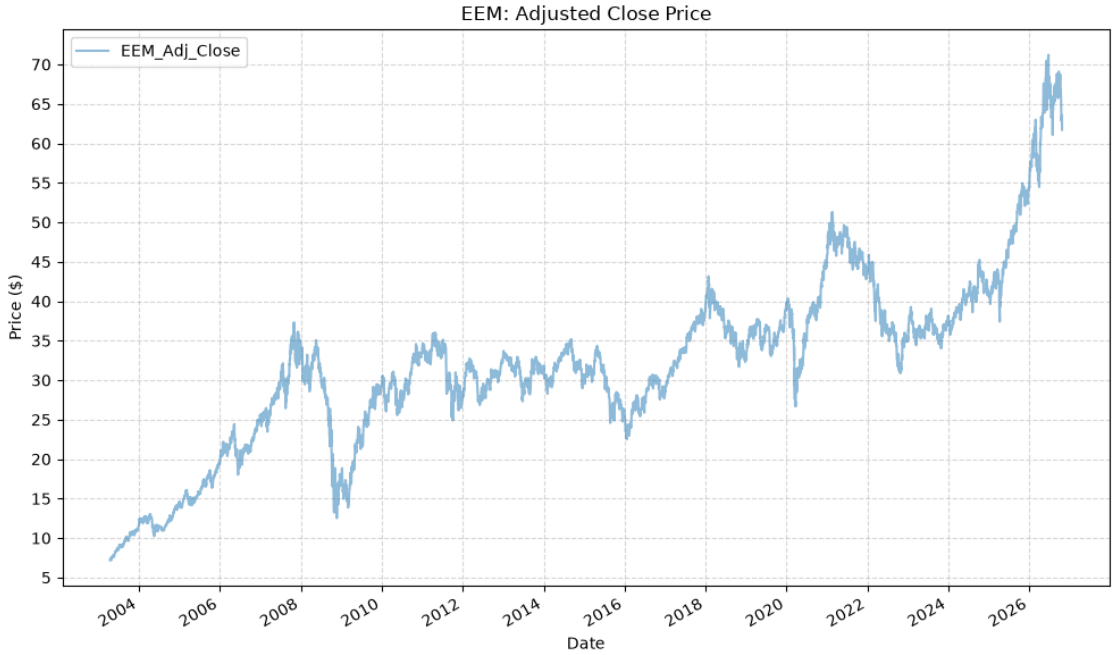
[6330 rows x 6 columns]



Date	EEM_Adj_Close	EEM_Close	EEM_High	EEM_Low	EEM_Open	EEM_Volume
2003-04-14	7.20	11.22	11.22	11.14	11.14	93600
2003-04-15	7.28	11.36	11.39	11.29	11.29	421200
2003-04-16	7.37	11.49	11.53	11.48	11.48	9000
2003-04-17	7.42	11.57	11.57	11.53	11.55	17100
2003-04-21	7.42	11.56	11.58	11.56	11.58	72900
…	…	…	…	…	…	…
2026-10-22	61.72	61.72	62.36	61.50	62.05	28385500

2026-10-23	62.01	62.01	62.08	61.60	61.66	22024100
2026-10-26	61.82	61.82	62.08	61.25	61.29	17851100
2026-10-27	62.52	62.52	62.72	62.14	62.28	23457400
2026-10-28	63.52	63.52	63.65	63.00	63.21	23641100

[5924 rows x 6 columns]



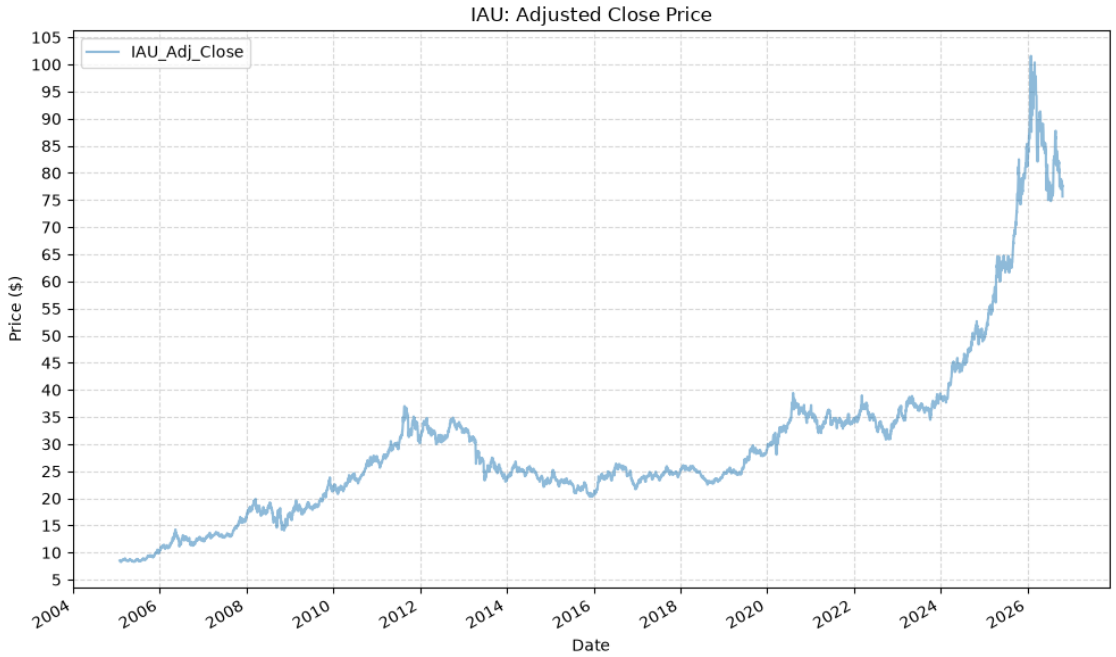
Date	GSG_Adj_Close	GSG_Close	GSG_High	GSG_Low	GSG_Open	GSG_Volume
2006-07-21	49.25	49.25	49.64	49.22	49.35	37400
2006-07-24	49.70	49.70	49.70	48.98	49.25	220900
2006-07-25	49.25	49.25	50.15	49.20	50.15	41600
2006-07-26	49.62	49.62	49.80	49.35	49.35	17800
2006-07-27	50.15	50.15	50.35	49.93	50.15	41800
…	…	…	…	…	…	…
2026-10-22	36.99	36.99	37.18	36.70	37.11	2320000
2026-10-23	37.70	37.70	37.75	37.33	37.35	2325500
2026-10-26	37.31	37.31	37.65	37.07	37.30	2217800
2026-10-27	36.99	36.99	37.28	36.84	37.18	782500
2026-10-28	37.15	37.15	37.46	37.15	37.22	1451500

[5100 rows x 6 columns]



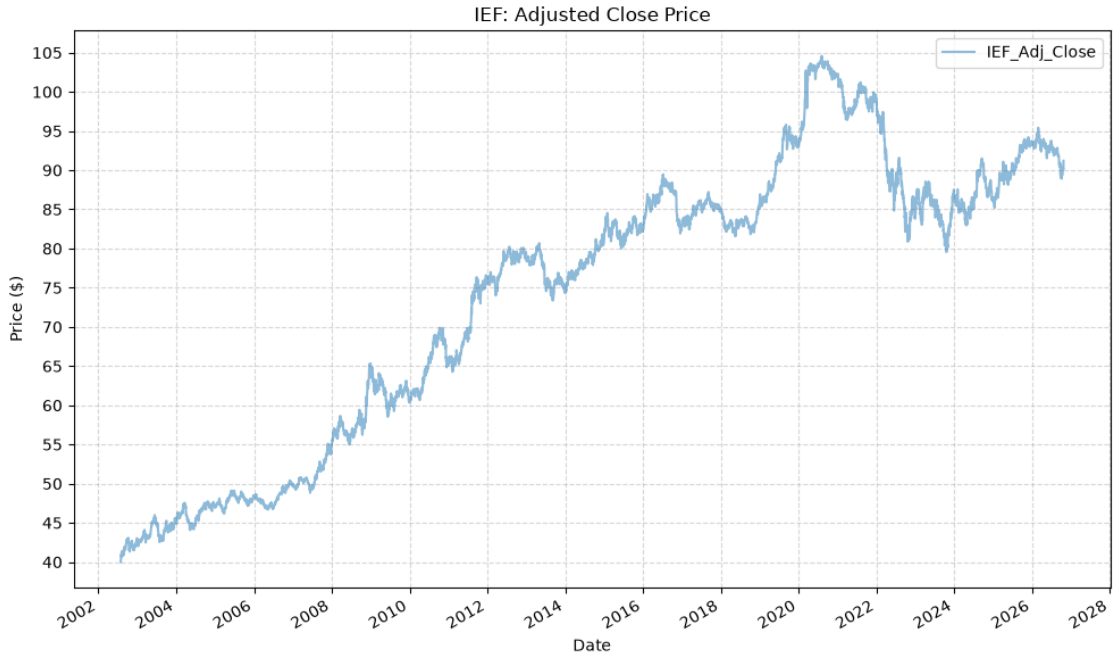
Date	IAU_Adj_Close	IAU_Close	IAU_High	IAU_Low	IAU_Open	IAU_Volume
2005-01-28	8.54	8.54	8.55	8.49	8.55	2888500
2005-01-31	8.45	8.45	8.46	8.40	8.45	759500
2005-02-01	8.42	8.42	8.43	8.39	8.42	347500
2005-02-02	8.45	8.45	8.45	8.41	8.45	1496500
2005-02-03	8.34	8.34	8.35	8.30	8.32	534000
…	…	…	…	…	…	…
2026-10-22	75.66	75.66	76.67	75.54	76.59	6251200
2026-10-23	75.15	75.15	75.71	74.70	75.64	4172600
2026-10-26	75.42	75.42	75.71	74.95	75.31	5149900
2026-10-27	76.33	76.33	76.39	75.55	75.58	2816700
2026-10-28	76.57	76.57	76.58	75.64	76.15	6140900

[5472 rows x 6 columns]



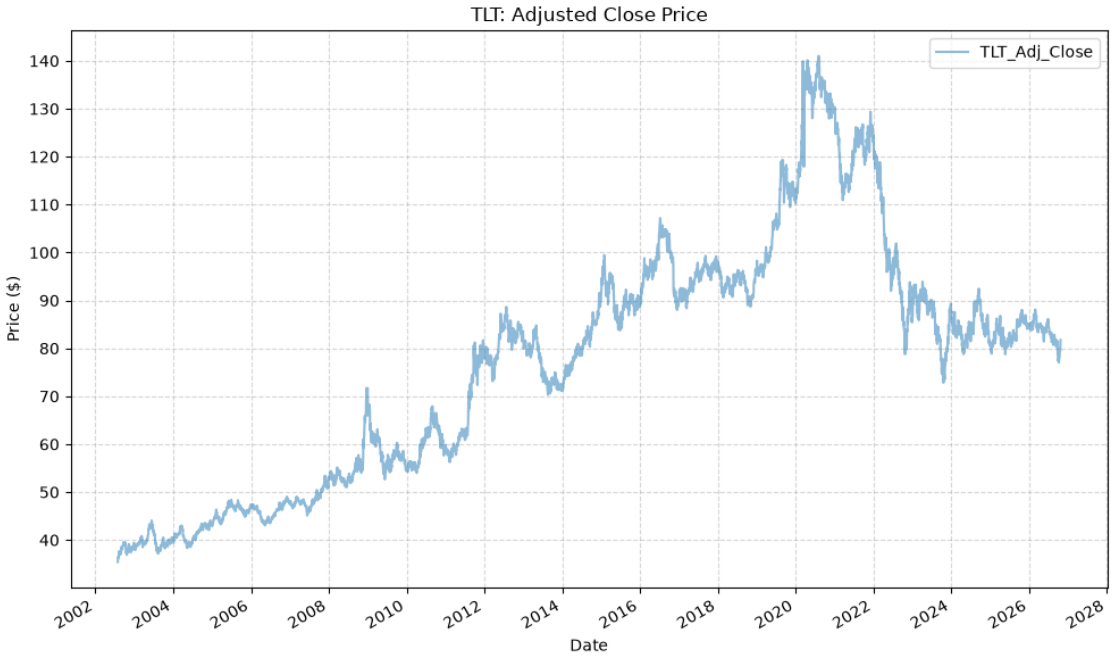
Date	IEF_Adj_Close	IEF_Close	IEF_High	IEF_Low	IEF_Open	IEF_Volume
2002-07-30	40.00	81.77	82.12	81.70	81.94	41300
2002-07-31	40.37	82.52	82.58	82.05	82.05	32600
2002-08-01	40.53	82.86	82.90	82.52	82.54	71400
2002-08-02	40.85	83.50	83.70	82.90	83.02	120300
2002-08-05	41.05	83.92	83.92	83.53	83.68	159300
…	…	…	…	…	…	…
2026-10-22	91.20	91.20	91.40	90.65	90.75	16518600
2026-10-23	91.05	91.05	91.25	91.01	91.23	32715900
2026-10-26	91.12	91.12	91.27	91.04	91.16	11267200
2026-10-27	91.10	91.10	91.16	90.99	91.06	19236700
2026-10-28	90.85	90.85	91.09	90.74	91.06	10812000

[6102 rows x 6 columns]



Date	TLT_Adj_Close	TLT_Close	TLT_High	TLT_Low	TLT_Open	TLT_Volume
2002-07-30	35.41	81.52	81.90	81.52	81.75	6100
2002-07-31	35.85	82.53	82.80	81.90	81.95	29400
2002-08-01	36.06	83.00	83.02	82.54	82.54	25000
2002-08-02	36.43	83.85	84.10	82.88	83.16	52800
2002-08-05	36.59	84.22	84.44	83.85	84.04	61100
…	…	…	…	…	…	…
2026-10-22	81.74	81.74	81.92	80.87	81.00	65635100
2026-10-23	81.34	81.34	81.80	81.27	81.79	35149600
2026-10-26	81.68	81.68	81.90	81.54	81.66	24676500
2026-10-27	81.63	81.63	81.73	81.47	81.65	19544700
2026-10-28	80.99	80.99	81.76	80.73	81.73	35127200

[6102 rows x 6 columns]



[9]:
```
# Create empty DF
data_merged = pd.DataFrame()

# Merge all data, calc cumulative returns, drawdowns
for fund, data in fund_data.items():
    data_merged = data_merged.merge(data[[f"{fund}_Adj_Close"]],␣
 ↪left_index=True, right_index=True, how='outer') if not data_merged.empty␣
 ↪else data[[f"{fund}_Adj_Close"]]
    data_merged[f"{fund}_Return"] = data_merged[f"{fund}_Adj_Close"].
 ↪pct_change()
    data_merged[f"{fund}_Cumulative_Return"] = (1 +␣
 ↪data_merged[f"{fund}_Return"]).cumprod() - 1
    data_merged[f"{fund}_Cumulative_Return_Plus_One"] = 1 +␣
 ↪data_merged[f"{fund}_Cumulative_Return"]
    data_merged[f"{fund}_Rolling_Max"] =␣
 ↪data_merged[f"{fund}_Cumulative_Return_Plus_One"].cummax()
    data_merged[f"{fund}_Drawdown"] =␣
 ↪data_merged[f"{fund}_Cumulative_Return_Plus_One"] /␣
 ↪data_merged[f"{fund}_Rolling_Max"] - 1
    data_merged.drop(columns=[f"{fund}_Cumulative_Return_Plus_One",␣
 ↪f"{fund}_Rolling_Max"], inplace=True)

display(data_merged)
```

```
            IVV_Adj_Close  IVV_Return  IVV_Cumulative_Return  IVV_Drawdown  \
```

```
Date
2000-05-19          88.08         NaN                    NaN           NaN
2000-05-22          87.53       -0.01                  -0.01          0.00
2000-05-23          86.20       -0.02                  -0.02         -0.02
2000-05-24          87.49        0.01                  -0.01         -0.00
2000-05-25          86.69       -0.01                  -0.02         -0.01
…                     …           …                      …             …
2026-10-22         767.41       -0.01                   7.71         -0.02
2026-10-23         763.25       -0.01                   7.67         -0.02
2026-10-26         768.69        0.01                   7.73         -0.02
2026-10-27         773.25        0.01                   7.78         -0.01
2026-10-28         775.65        0.00                   7.81         -0.01

            EFA_Adj_Close  EFA_Return  EFA_Cumulative_Return  EFA_Drawdown  \
Date
2000-05-19            NaN         NaN                    NaN           NaN
2000-05-22            NaN         NaN                    NaN           NaN
2000-05-23            NaN         NaN                    NaN           NaN
2000-05-24            NaN         NaN                    NaN           NaN
2000-05-25            NaN         NaN                    NaN           NaN
…                       …           …                      …             …
2026-10-22         103.15       -0.01                   3.72         -0.05
2026-10-23         102.63       -0.01                   3.70         -0.06
2026-10-26         102.91        0.00                   3.71         -0.05
2026-10-27         103.59        0.01                   3.74         -0.05
2026-10-28         103.68        0.00                   3.74         -0.05

            EEM_Adj_Close  EEM_Return  …  IAU_Cumulative_Return  \
Date                                   …
2000-05-19            NaN         NaN  …                    NaN
2000-05-22            NaN         NaN  …                    NaN
2000-05-23            NaN         NaN  …                    NaN
2000-05-24            NaN         NaN  …                    NaN
2000-05-25            NaN         NaN  …                    NaN
…                       …           …  …                      …
2026-10-22          61.72       -0.03  …                   7.86
2026-10-23          62.01        0.00  …                   7.80
2026-10-26          61.82       -0.00  …                   7.83
2026-10-27          62.52        0.01  …                   7.94
2026-10-28          63.52        0.02  …                   7.97

            IAU_Drawdown  IEF_Adj_Close  IEF_Return  IEF_Cumulative_Return  \
Date
2000-05-19           NaN            NaN         NaN                    NaN
2000-05-22           NaN            NaN         NaN                    NaN
2000-05-23           NaN            NaN         NaN                    NaN
2000-05-24           NaN            NaN         NaN                    NaN
2000-05-25           NaN            NaN         NaN                    NaN
```

```
…                  …              …           …                        …
2026-10-22         -0.26          91.20        0.01                    1.28
2026-10-23         -0.26          91.05       -0.00                    1.28
2026-10-26         -0.26          91.12        0.00                    1.28
2026-10-27         -0.25          91.10       -0.00                    1.28
2026-10-28         -0.25          90.85       -0.00                    1.27

            IEF_Drawdown  TLT_Adj_Close  TLT_Return  TLT_Cumulative_Return  \
Date
2000-05-19           NaN            NaN         NaN                     NaN
2000-05-22           NaN            NaN         NaN                     NaN
2000-05-23           NaN            NaN         NaN                     NaN
2000-05-24           NaN            NaN         NaN                     NaN
2000-05-25           NaN            NaN         NaN                     NaN
…                    …              …           …                       …
2026-10-22         -0.13          81.74        0.02                    1.31
2026-10-23         -0.13          81.34       -0.00                    1.30
2026-10-26         -0.13          81.68        0.00                    1.31
2026-10-27         -0.13          81.63       -0.00                    1.31
2026-10-28         -0.13          80.99       -0.01                    1.29

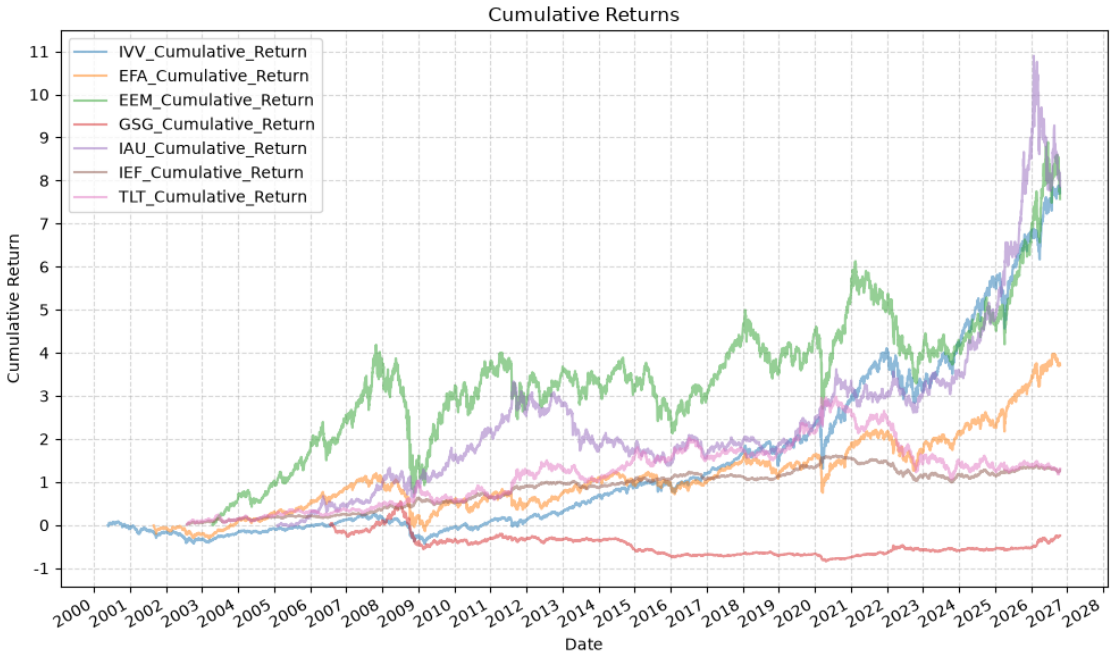
            TLT_Drawdown
Date
2000-05-19           NaN
2000-05-22           NaN
2000-05-23           NaN
2000-05-24           NaN
2000-05-25           NaN
…                    …
2026-10-22         -0.42
2026-10-23         -0.42
2026-10-26         -0.42
2026-10-27         -0.42
2026-10-28         -0.43

[6650 rows x 28 columns]
```

And now the plots for the cumulative returns and drawdowns:

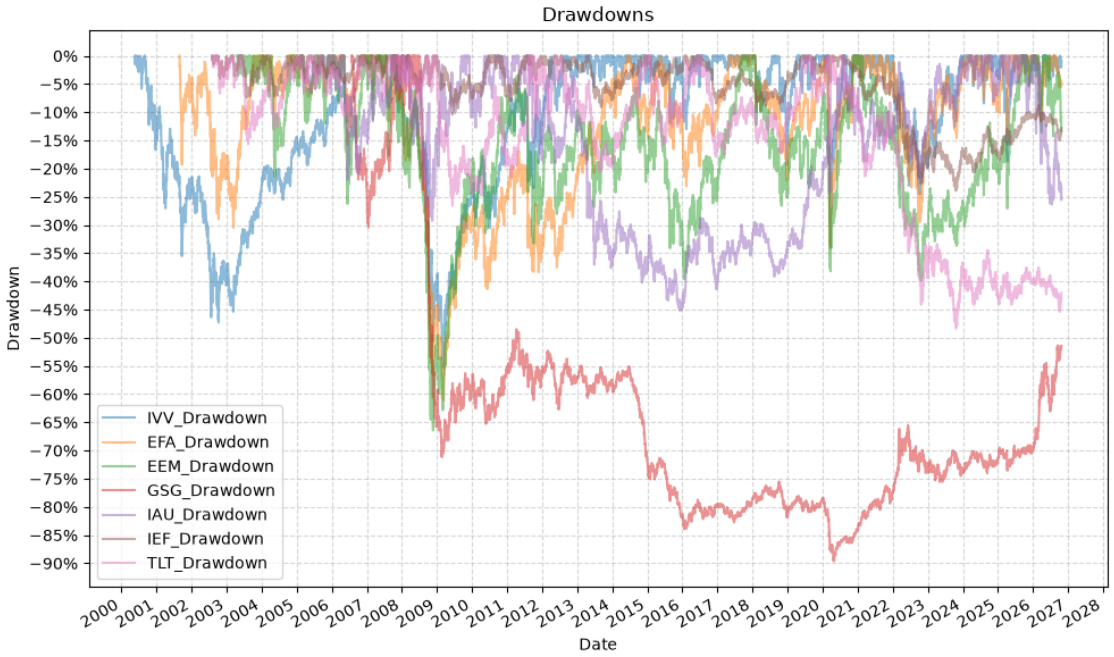
[10]:
```
plot_time_series(
    df=data_merged,
    plot_start_date=None,
    plot_end_date=None,
    plot_columns=[col for col in data_merged.columns if "Cumulative_Return" in␣
 ↪col],
    title="Cumulative Returns",
    x_label="Date",
    x_format="Year",
```

```
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=30,
    y_label="Cumulative Return",
    y_format="Decimal",
    y_format_decimal_places=0,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
)
```



[11]:
```
plot_time_series(
    df=data_merged,
    plot_start_date=None,
    plot_end_date=None,
    plot_columns=[col for col in data_merged.columns if "Drawdown" in col],
    title="Drawdowns",
    x_label="Date",
    x_format="Year",
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=30,
    y_label="Drawdown",
    y_format="Percentage",
    y_format_decimal_places=0,
    y_tick_spacing="Auto",
```

```
    y_tick_rotation=0,
)
```



This time we'll drop the empty rows and normalize the starting values to provide a more accurate comparison, but this will reduce our data set to the inception of GSG in 2006:

[12]:
```
# Create empty DF
data_merged_aligned = pd.DataFrame()

# Merge all data, calc cumulative returns, drawdowns
for fund, data in fund_data.items():
    data_merged_aligned = data_merged_aligned.
 ↪merge(data[[f"{fund}_Adj_Close"]], left_index=True, right_index=True,␣
 ↪how='outer') if not data_merged_aligned.empty else␣
 ↪data[[f"{fund}_Adj_Close"]]
    data_merged_aligned = data_merged_aligned.dropna()

for fund in fund_data.keys():
    data_merged_aligned[f"{fund}_Return"] =␣
 ↪data_merged_aligned[f"{fund}_Adj_Close"].pct_change()
    data_merged_aligned[f"{fund}_Cumulative_Return"] = (1 +␣
 ↪data_merged_aligned[f"{fund}_Return"]).cumprod() - 1
    data_merged_aligned[f"{fund}_Cumulative_Return_Plus_One"] = 1 +␣
 ↪data_merged_aligned[f"{fund}_Cumulative_Return"]
```

```
    data_merged_aligned[f"{fund}_Rolling_Max"] =␣
 ↪data_merged_aligned[f"{fund}_Cumulative_Return_Plus_One"].cummax()
    data_merged_aligned[f"{fund}_Drawdown"] =␣
 ↪data_merged_aligned[f"{fund}_Cumulative_Return_Plus_One"] /␣
 ↪data_merged_aligned[f"{fund}_Rolling_Max"] - 1
    data_merged_aligned.drop(columns=[f"{fund}_Cumulative_Return_Plus_One",␣
 ↪f"{fund}_Rolling_Max"], inplace=True)

display(data_merged_aligned)
```

```
            IVV_Adj_Close  EFA_Adj_Close  EEM_Adj_Close  GSG_Adj_Close  \
Date
2006-07-21          85.73          34.42          19.76          49.25
2006-07-24          87.19          35.04          20.65          49.70
2006-07-25          87.53          35.03          20.70          49.25
2006-07-26          87.69          35.28          20.62          49.62
2006-07-27          87.58          35.50          20.90          50.15
…                     …              …              …              …
2026-10-22         767.41         103.15          61.72          36.99
2026-10-23         763.25         102.63          62.01          37.70
2026-10-26         768.69         102.91          61.82          37.31
2026-10-27         773.25         103.59          62.52          36.99
2026-10-28         775.65         103.68          63.52          37.15

            IAU_Adj_Close  IEF_Adj_Close  TLT_Adj_Close  IVV_Return  \
Date
2006-07-21          12.37          47.66          44.81         NaN
2006-07-24          12.22          47.66          44.79        0.02
2006-07-25          12.34          47.62          44.65        0.00
2006-07-26          12.40          47.73          44.82        0.00
2006-07-27          12.59          47.73          44.75       -0.00
…                     …              …              …           …
2026-10-22          75.66          91.20          81.74       -0.01
2026-10-23          75.15          91.05          81.34       -0.01
2026-10-26          75.42          91.12          81.68        0.01
2026-10-27          76.33          91.10          81.63        0.01
2026-10-28          76.57          90.85          80.99        0.00

            IVV_Cumulative_Return  IVV_Drawdown  …  GSG_Drawdown  \
Date                                             …
2006-07-21                    NaN           NaN  …           NaN
2006-07-24                   0.02          0.00  …          0.00
2006-07-25                   0.02          0.00  …         -0.01
2006-07-26                   0.02          0.00  …         -0.00
2006-07-27                   0.02         -0.00  …          0.00
…                             …             …  …             …
2026-10-22                   7.95         -0.02  …         -0.52
```

```
2026-10-23                  7.90         -0.02  …         -0.51
2026-10-26                  7.97         -0.02  …         -0.51
2026-10-27                  8.02         -0.01  …         -0.52
2026-10-28                  8.05         -0.01  …         -0.51

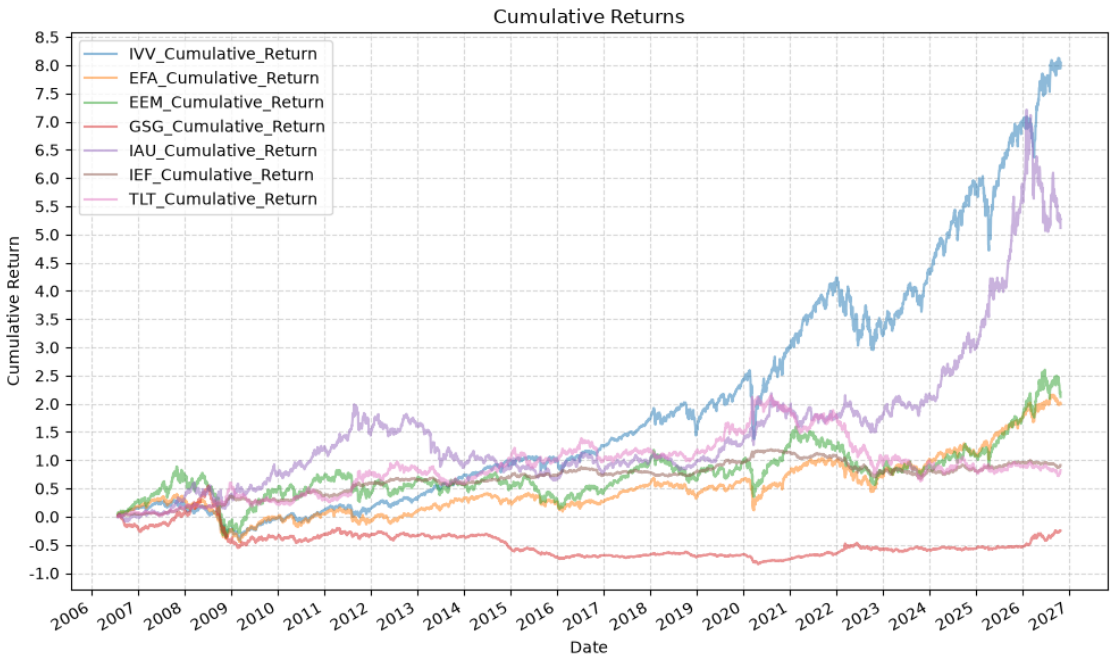
            IAU_Return  IAU_Cumulative_Return  IAU_Drawdown  IEF_Return  \
Date
2006-07-21         NaN                   NaN           NaN         NaN
2006-07-24       -0.01                 -0.01          0.00        0.00
2006-07-25        0.01                 -0.00          0.00       -0.00
2006-07-26        0.01                  0.00          0.00        0.00
2006-07-27        0.02                  0.02          0.00       -0.00
…                 …                     …             …          …
2026-10-22       -0.03                  5.12         -0.26        0.01
2026-10-23       -0.01                  5.08         -0.26       -0.00
2026-10-26        0.00                  5.10         -0.26        0.00
2026-10-27        0.01                  5.17         -0.25       -0.00
2026-10-28        0.00                  5.19         -0.25       -0.00

            IEF_Cumulative_Return  IEF_Drawdown  TLT_Return  \
Date
2006-07-21                   NaN           NaN         NaN
2006-07-24                  0.00          0.00       -0.00
2006-07-25                 -0.00         -0.00       -0.00
2006-07-26                  0.00          0.00        0.00
2006-07-27                  0.00         -0.00       -0.00
…                           …             …           …
2026-10-22                  0.91         -0.13        0.02
2026-10-23                  0.91         -0.13       -0.00
2026-10-26                  0.91         -0.13        0.00
2026-10-27                  0.91         -0.13       -0.00
2026-10-28                  0.91         -0.13       -0.01

            TLT_Cumulative_Return  TLT_Drawdown
Date
2006-07-21                   NaN           NaN
2006-07-24                 -0.00          0.00
2006-07-25                 -0.00         -0.00
2006-07-26                  0.00          0.00
2006-07-27                 -0.00         -0.00
…                           …             …
2026-10-22                  0.82         -0.42
2026-10-23                  0.82         -0.42
2026-10-26                  0.82         -0.42
2026-10-27                  0.82         -0.42
2026-10-28                  0.81         -0.43

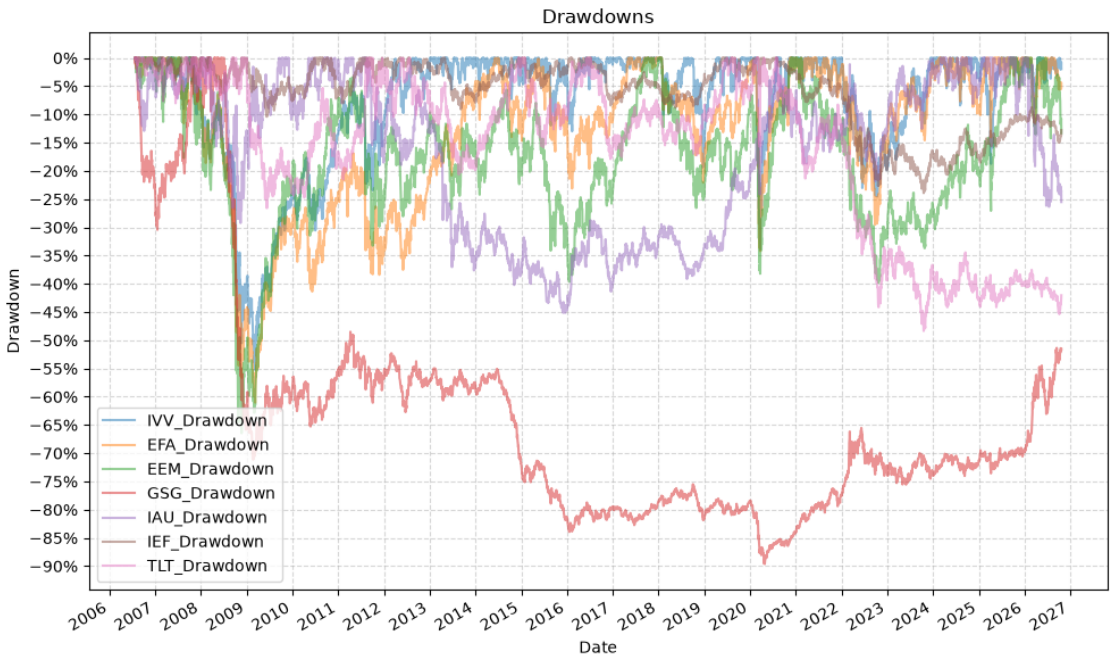
[5100 rows x 28 columns]
```

[13]:
```
plot_time_series(
    df=data_merged_aligned,
    plot_start_date=None,
    plot_end_date=None,
    plot_columns=[col for col in data_merged_aligned.columns if␣
 ↪"Cumulative_Return" in col],
    title="Cumulative Returns",
    x_label="Date",
    x_format="Year",
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=30,
    y_label="Cumulative Return",
    y_format="Decimal",
    y_format_decimal_places=1,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
)
```



From the above plot, we can see that US stocks have outperformed all other asset class, but this is since mid-2006, and misses the dotcom bubble of the late 1990s and early 2000s. Right behind US stocks, we have gold, which has made a historic comback since COVID, and at one point had higher cumulative perfomance than US stocks.

[14]:
```
plot_time_series(
    df=data_merged_aligned,
    plot_start_date=None,
    plot_end_date=None,
    plot_columns=[col for col in data_merged_aligned.columns if "Drawdown" in␣
 ↪col],
    title="Drawdowns",
    x_label="Date",
    x_format="Year",
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=30,
    y_label="Drawdown",
    y_format="Percentage",
    y_format_decimal_places=0,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
)
```



The drawdowns plot gives us insight into the risk and of the different asset classes. Not volatility necessiarly, but the risk of permanent or semi-permanent loss of capital. Assets that experience multi-year or decade drawdowns are really an opportunity cost, where capital is tied up and cannot be deployed elsewhere. We can see that stocks, bonds, and gold have all gone through ~50% drawdowns, and recovered to make new highs. But commodities, specifically GSG, have experienced a nearly 90% drawdown. There are certainly points in time when investing in commodities has paid off (particularly recently post-COVID when inflation spiked), but the long-term risk is apparent.

0.5.4 Summary Statistics

Let's look at the summary statistics. Keep in mind that we are not aligning the dates here, so the number of observations is different for each ETF.

[15]:
```
sum_stats = pd.DataFrame()

for fund in fund_data.keys():
    data_stats = summary_stats(
        fund_list=[f"{fund}"],
        df=data_merged[[f"{fund}_Return"]].dropna(),
        period="Daily",
        use_calendar_days=False,
        excel_export=False,
        pickle_export=False,
        output_confirmation=False,
    )

    sum_stats = pd.concat([sum_stats, data_stats])

display(sum_stats)
```

	Number of Observations	Data Start Date	Data End Date	\
IVV_Return	6649	2000-05-22	2026-10-28	
EFA_Return	6329	2001-08-28	2026-10-28	
EEM_Return	5923	2003-04-15	2026-10-28	
GSG_Return	5099	2006-07-24	2026-10-28	
IAU_Return	5471	2005-01-31	2026-10-28	
IEF_Return	6101	2002-07-31	2026-10-28	
TLT_Return	6101	2002-07-31	2026-10-28	

	Annual Mean Return (Arithmetic)	Annualized Volatility	\
IVV_Return	0.10	0.19	
EFA_Return	0.08	0.21	
EEM_Return	0.13	0.27	
GSG_Return	0.01	0.23	
IAU_Return	0.12	0.18	
IEF_Return	0.04	0.07	
TLT_Return	0.04	0.14	

	Annualized Sharpe Ratio	CAGR (Geometric)	Daily Max Return	\
IVV_Return	0.53	0.09	0.11	
EFA_Return	0.40	0.06	0.16	
EEM_Return	0.48	0.10	0.23	
GSG_Return	0.06	-0.01	0.08	
IAU_Return	0.64	0.11	0.12	
IEF_Return	0.53	0.03	0.03	
TLT_Return	0.31	0.03	0.08	

```
           Daily Max Return (Date)  Daily Min Return Daily Min Return (Date)  \
IVV_Return              2008-10-28             -0.12              2020-03-16
EFA_Return              2008-10-13             -0.11              2008-09-29
EEM_Return              2008-10-13             -0.16              2008-10-15
GSG_Return              2008-11-04             -0.12              2020-03-09
IAU_Return              2008-09-17             -0.10              2026-01-30
IEF_Return              2009-03-18             -0.03              2020-03-17
TLT_Return              2020-03-20             -0.07              2020-03-17

            Max Drawdown       Peak     Trough Recovery Date  \
IVV_Return         -0.55 2007-10-09 2009-03-09    2012-08-16
EFA_Return         -0.61 2007-10-31 2009-03-09    2014-06-06
EEM_Return         -0.66 2007-10-31 2008-11-20    2017-09-18
GSG_Return         -0.90 2008-07-02 2020-04-28           NaT
IAU_Return         -0.45 2011-08-22 2015-12-02    2020-07-27
IEF_Return         -0.24 2020-08-04 2023-10-19           NaT
TLT_Return         -0.48 2020-08-04 2023-10-19           NaT

            Calendar Days to Recovery  MAR Ratio
IVV_Return                    1256.00       0.16
EFA_Return                    1915.00       0.10
EEM_Return                    3224.00       0.15
GSG_Return                        NaN      -0.02
IAU_Return                    1699.00       0.24
IEF_Return                        NaN       0.14
TLT_Return                        NaN       0.07
```

Note that emerging markets have outperformed all other equity assets when factoring in the drawdown that US markets experienced in the early 2000s.

0.6 Calculate Moving Averages

Now, let's calculate the various different moving averages for each ETF. We will resample the data to use the month end values only, and calculate the 3 - 12 month moving averages.

[16]:
```python
# Resample to month end values
for fund, data in fund_data.items():
    fund_data[fund] = data.resample('ME').last()
```

[17]:
```python
# Define moving average windows in months
ma_windows = {
    '3m': 3,     # 3 months (~63 trading days)
    '4m': 4,     # 4 months (~84 trading days)
    '5m': 5,     # 5 months (~105 trading days)
    '6m': 6,     # 6 months (~126 trading days)
    '7m': 7,     # 7 months (~147 trading days)
    '8m': 8,     # 8 months (~168 trading days)
    '9m': 9,     # 9 months (~189 trading days)
```

```
    '10m': 10,   # 10 months (~210 trading days)
    '11m': 11,   # 11 months (~231 trading days)
    '12m': 12    # 12 months (~252 trading days)
}
```

[18]:
```
for fund, data in fund_data.items():
    for window, months in ma_windows.items():
        data[f"{fund}_MA_{window}"] = data[f"{fund}_Adj_Close"].
 ↪rolling(window=months).mean()
```

0.7 Calculate Forward Return Windows

Next, we will calculate the forward return for each ETF for the same time periods as the moving averages.

[19]:
```
# Define forward return windows in months
fr_windows = {
    '3m': 3,     # 3 months (~63 trading days)
    '4m': 4,     # 4 months (~84 trading days)
    '5m': 5,     # 5 months (~105 trading days)
    '6m': 6,     # 6 months (~126 trading days)
    '7m': 7,     # 7 months (~147 trading days)
    '8m': 8,     # 8 months (~168 trading days)
    '9m': 9,     # 9 months (~189 trading days)
    '10m': 10,   # 10 months (~210 trading days)
    '11m': 11,   # 11 months (~231 trading days)
    '12m': 12    # 12 months (~252 trading days)
}
```

[20]:
```
for fund, data in fund_data.items():
    for window, months in fr_windows.items():
        data[f"{fund}_Forward_Return_{window}"] = data[f"{fund}_Adj_Close"].
 ↪shift(-months) / data[f"{fund}_Adj_Close"] - 1
```

0.8 Calculate Moving Average Predictions

Now, we will calculate the moving average predictions, as follows:

- If the month-end price is above the moving average, the prediction is 1 (i.e. the price will continue to be above in the future).
- If the month-end price is below the moving average, the prediction is -1 (i.e. the price will continue to be below in the future).
- Calculate the accuracy of the predictions by comparing them to the actual forward returns (i.e. if the forward return is positive, the actual outcome is 1, and if the forward return is negative, the actual outcome is -1).
- Calculate the difference between the price and moving average (percentage difference), z-score (standardize) both that difference and the forward returns, and run a standardized OLS regression of forward return on the price-MA difference. Standardizing both sides puts every fund, moving average, and forward return window on the same scale, so the regression

slope (a standardized beta, equal to the Pearson correlation) is directly comparable across them.

- Because the forward returns overlap, meaning an N-month forward return sampled monthly shares $N-1$ months of its window with the next observation, the results are heavily autocorrelated, and ordinary OLS standard errors would overstate the significance of the results. We therefore report Newey-West (HAC) standard errors with the lag set to the forward-return horizon. This leaves the slope unchanged and only corrects the standard errors and p-value. Note that at the longer horizons (e.g. 12 months) there are relatively few independent observations, so those p-values should still be read with caution.

[21]:
```
ma_prediction_results = pd.DataFrame(columns=[
    "Fund",
    "MA_Window",
    "FR_Window",
    "Overall_Accuracy",
    "Positive_Accuracy",
    "Negative_Accuracy",
    "Std_Beta",
    "Std_Beta_PValue",
    "R_Squared",
])

pair_data_dict = {}
model_dict = {}

for fund, data in fund_data.items():
    for ma_label, ma_window in ma_windows.items():
        # Prediction and the price-vs-MA gap depend only on the MA window, so␣
 ↪compute them
        # once per MA on the full frame (these columns persist for the plots␣
 ↪further down).
        data[f"{fund}_MA_Prediction_{ma_label}"] = 0
        data.loc[data[f"{fund}_Adj_Close"] > data[f"{fund}_MA_{ma_label}"],␣
 ↪f"{fund}_MA_Prediction_{ma_label}"] = 1
        data.loc[data[f"{fund}_Adj_Close"] < data[f"{fund}_MA_{ma_label}"],␣
 ↪f"{fund}_MA_Prediction_{ma_label}"] = -1

        # Calculate the percentage difference between price and moving average
        data[f"{fund}_Price_MA_Diff_Percent_{ma_label}"] =␣
 ↪(data[f"{fund}_Adj_Close"] - data[f"{fund}_MA_{ma_label}"]) /␣
 ↪data[f"{fund}_MA_{ma_label}"]

        for fr_label, fr_window in fr_windows.items():
            data[f"{fund}_Actual_{fr_label}"] = 0
            data.loc[data[f"{fund}_Forward_Return_{fr_label}"] > 0,␣
 ↪f"{fund}_Actual_{fr_label}"] = 1
```

```
            data.loc[data[f"{fund}_Forward_Return_{fr_label}"] < 0,␣
↪f"{fund}_Actual_{fr_label}"] = -1

            pred_col = f"{fund}_MA_Prediction_{ma_label}"
            actual_col = f"{fund}_Actual_{fr_label}"
            diff_col = f"{fund}_Price_MA_Diff_Percent_{ma_label}"
            fr_col = f"{fund}_Forward_Return_{fr_label}"

            # Restrict to rows where THIS MA and THIS forward return are both␣
↪defined, so each window pair is evaluated on its own full sample rather than␣
↪the intersection of all 12 windows.
            pair_data = data.dropna(subset=[f"{fund}_MA_{ma_label}", fr_col])

            overall_accuracy = (pair_data[pred_col] == pair_data[actual_col]).
↪mean()
            pos_accuracy = ((pair_data[pred_col] == 1) & (pair_data[actual_col]␣
↪== 1)).sum() / (pair_data[pred_col] == 1).sum()
            neg_accuracy = ((pair_data[pred_col] == -1) &␣
↪(pair_data[actual_col] == -1)).sum() / (pair_data[pred_col] == -1).sum()

            # Base rate is the share of positive forward returns in this sample.
↪ Because these assets have positive drift, this is usually well above 0.50,␣
↪so it is the best benchmark for the accuracy numbers above. A model that␣
↪always guessed "up" would already score `base_rate` on positive predictions␣
↪and `max(base_rate, 1 - base_rate)` overall, so the MA only adds value to␣
↪the extent accuracy exceeds those. We record the lift (accuracy minus base␣
↪ratebenchmark).
            base_rate = (pair_data[fr_col] > 0).mean()
            overall_lift = overall_accuracy - max(base_rate, 1 - base_rate)
            positive_lift = pos_accuracy - base_rate
            negative_lift = neg_accuracy - (1 - base_rate)

            # Calculate the mean forward return (and +/- 2 std bands) for the␣
↪cases where the MA predicted a positive return and where it predicted a␣
↪negative return
            positive_returns = pair_data.loc[pair_data[pred_col] == 1, fr_col]
            positive_mean_return = positive_returns.mean()
            positive_mean_plus_two_std = positive_mean_return +␣
↪positive_returns.std() * 2
            positive_mean_minus_two_std = positive_mean_return -␣
↪positive_returns.std() * 2
            negative_returns = pair_data.loc[pair_data[pred_col] == -1, fr_col]
            negative_mean_return = negative_returns.mean()
            negative_mean_plus_two_std = negative_mean_return +␣
↪negative_returns.std() * 2
```

```
            negative_mean_minus_two_std = negative_mean_return -␣
↪negative_returns.std() * 2

            # Z-score (standardize) the predictor (% diff between price and␣
↪moving average) and the forward return so the regression slope is a␣
↪standardized beta, comparable across funds, MAs, and forward windows. With␣
↪both sides standardized the slope equals the Pearson correlation.
            diff_z_col = f"{diff_col}_Z"
            fr_z_col = f"{fr_col}_Z"
            pair_data = pair_data.assign(**{
                diff_z_col: (pair_data[diff_col] - pair_data[diff_col].mean()) /
↪ pair_data[diff_col].std(),
                fr_z_col: (pair_data[fr_col] - pair_data[fr_col].mean()) /␣
↪pair_data[fr_col].std(),
            })

            # Add pair_data DataFrame to dictionary
            pair_data_dict[(fund, ma_label, fr_label)] = pair_data

            # Run a standardized OLS regression of forward return on the␣
↪price-MA difference. The forward windows overlap (an N-month forward return␣
↪sampled monthly shares N-1 months with its neighbor), which inflates the␣
↪apparent significance. Use Newey-West (HAC) standard errors with the lag set␣
↪to the forward-return horizon so the p-value accounts for that␣
↪autocorrelation. The slope/beta is unchanged; only the standard errors (and␣
↪p-value) are corrected.
            model = run_regression(
                df=pair_data,
                x_plot_column=diff_z_col,
                y_plot_column=fr_z_col,
                regression_model="OLS-statsmodels",
                regression_constant=True,
                cov_type="HAC",
                cov_kwds={"maxlags": fr_window},
            )

            # Add model to dictionary
            model_dict[(fund, ma_label, fr_label)] = model

            std_beta = model.params.iloc[1]          # standardized slope␣
↪(Pearson correlation)
            std_beta_pvalue = model.pvalues.iloc[1]  # p-value for the slope
            r_squared = model.rsquared               # R-squared of the␣
↪regression

            results = pd.DataFrame([{
```

```
                    "Fund": fund,
                    "MA_Window": ma_label,
                    "FR_Window": fr_label,
                    "Overall_Accuracy": overall_accuracy,
                    "Positive_Accuracy": pos_accuracy,
                    "Negative_Accuracy": neg_accuracy,
                    "Base_Rate": base_rate,
                    "Overall_Lift": overall_lift,
                    "Positive_Lift": positive_lift,
                    "Negative_Lift": negative_lift,
                    "Positive_Mean_Return": positive_mean_return,
                    "Positive_Mean_Plus_Two_Std": positive_mean_plus_two_std,
                    "Positive_Mean_Minus_Two_Std": positive_mean_minus_two_std,
                    "Negative_Mean_Return": negative_mean_return,
                    "Negative_Mean_Plus_Two_Std": negative_mean_plus_two_std,
                    "Negative_Mean_Minus_Two_Std": negative_mean_minus_two_std,
                    "Std_Beta": std_beta,
                    "Std_Beta_PValue": std_beta_pvalue,
                    "R_Squared": r_squared,
                }])

                ma_prediction_results = pd.concat([ma_prediction_results, results],␣
 ↪ignore_index=True)
```

Executing the code above gives us the following table of results.

[22]:
```
# The results frame was created by concatenating onto an empty (all-object)␣
 ↪DataFrame, which leaves the metric columns as dtype object. Coerce every␣
 ↪column except the string labels tonumeric so downstream plots (heatmaps in␣
 ↪particular) get real floats.
label_cols = ["Fund", "MA_Window", "FR_Window"]
numeric_cols = ma_prediction_results.columns.difference(label_cols)
ma_prediction_results[numeric_cols] = ma_prediction_results[numeric_cols].
 ↪apply(pd.to_numeric)

display(ma_prediction_results)
```

	Fund	MA_Window	FR_Window	Overall_Accuracy	Positive_Accuracy	\
0	IVV	3m	3m	0.57	0.70	
1	IVV	3m	4m	0.60	0.74	
2	IVV	3m	5m	0.63	0.77	
3	IVV	3m	6m	0.63	0.77	
4	IVV	3m	7m	0.67	0.83	
..	…	…	…	…	…	
695	TLT	12m	8m	0.45	0.53	
696	TLT	12m	9m	0.45	0.56	
697	TLT	12m	10m	0.44	0.55	
698	TLT	12m	11m	0.45	0.56	

```
699  TLT       12m       12m              0.48               0.60

     Negative_Accuracy  Std_Beta  Std_Beta_PValue  R_Squared  Base_Rate  \
0                 0.30      0.03             0.73       0.00       0.70
1                 0.30      0.05             0.60       0.00       0.73
2                 0.36      0.04             0.67       0.00       0.73
3                 0.34      0.03             0.75       0.00       0.74
4                 0.35      0.06             0.40       0.00       0.77
..                 …         …                …          …          …
695               0.33     -0.09             0.46       0.01       0.59
696               0.30     -0.08             0.51       0.01       0.61
697               0.29     -0.07             0.54       0.01       0.62
698               0.29     -0.06             0.65       0.00       0.62
699               0.32     -0.04             0.75       0.00       0.63

     Overall_Lift  Positive_Lift  Negative_Lift  Positive_Mean_Return  \
0           -0.13           0.00           0.00                  0.02
1           -0.13           0.01           0.03                  0.03
2           -0.09           0.04           0.09                  0.04
3           -0.11           0.04           0.08                  0.05
4           -0.10           0.06           0.12                  0.06
..            …              …              …                     …
695         -0.14          -0.06          -0.08                  0.02
696         -0.16          -0.06          -0.08                  0.02
697         -0.18          -0.07          -0.09                  0.02
698         -0.17          -0.06          -0.08                  0.03
699         -0.15          -0.03          -0.05                  0.03

     Positive_Mean_Plus_Two_Std  Positive_Mean_Minus_Two_Std  \
0                          0.15                        -0.11
1                          0.18                        -0.12
2                          0.21                        -0.13
3                          0.24                        -0.15
4                          0.26                        -0.14
..                          …                           …
695                        0.23                        -0.19
696                        0.24                        -0.20
697                        0.27                        -0.22
698                        0.28                        -0.22
699                        0.30                        -0.23

     Negative_Mean_Return  Negative_Mean_Plus_Two_Std  \
0                    0.02                        0.21
1                    0.03                        0.25
2                    0.04                        0.29
3                    0.05                        0.32
4                    0.05                        0.35
..                    …                           …
```

```
695                   0.04                          0.26
696                   0.04                          0.28
697                   0.05                          0.28
698                   0.05                          0.30
699                   0.05                          0.31

     Negative_Mean_Minus_Two_Std
0                          -0.16
1                          -0.19
2                          -0.21
3                          -0.22
4                          -0.25
..                           …
695                        -0.19
696                        -0.19
697                        -0.19
698                        -0.20
699                        -0.20

[700 rows x 19 columns]
```

0.9 Predictive Power Across Moving Average And Forward-Return Windows

The results table has one row per (fund, MA window, FR window), resulting in 7 x 10 x 10 = 700 combinations. Rather than plotting every combination individually, we summarize each fund as a heatmap over the full MA-window (rows) by FR-window (columns) grid. We present two separate heatmaps:

- **Standardized beta:** The slope of the standardized regression of FR on the price-MA gap, which (with both sides standardized) equals the Pearson correlation. This is the cleanest single measure of predictive strength, because a positive value means a wider price-above-MA gap is associated with higher subsequent returns.
- **Accuracy lift:** How much the prediction beats its naive benchmark. Because these assets drift upward, FRs are positive well over half the time, so raw accuracy alone is misleading. A model that always guessed "up" would already be right at the *base rate*. We therefore plot the lift, the accuracy minus the base rate, so the neutral point is zero (red = the MA beats always-guess-up, blue = it does worse) rather than an arbitrary coin flip.

[23]:
```
# Order the axes by window length rather than alphabetically ("10m" would sort␣
 ↪before "3m").
ma_order = list(ma_windows.keys())
fr_order = list(fr_windows.keys())


def prediction_heatmap(metric, cmap="coolwarm", vmin=-1.0, vmax=1.0, center=0.
 ↪0, fmt=".2f"):
    """Plot a MA-window by FR-window heatmap of `metric` for each fund."""
    for fund in fund_list:
```

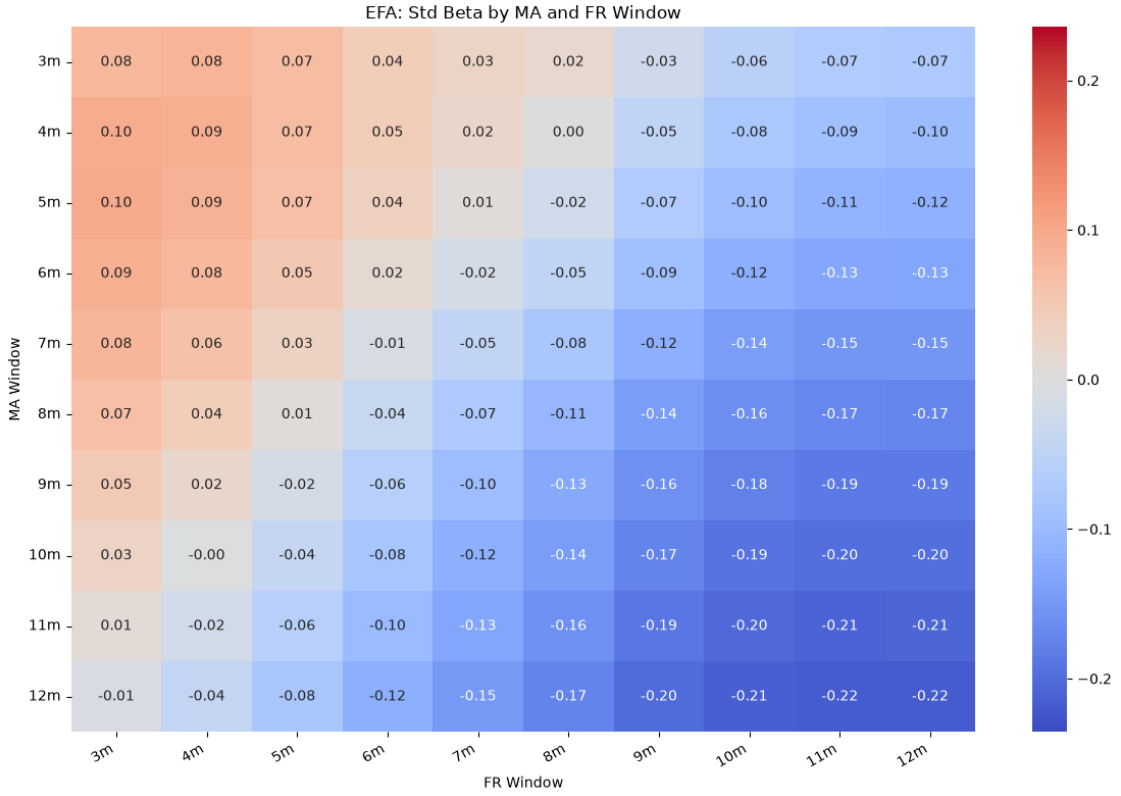
```
        grid = (
            ma_prediction_results[ma_prediction_results["Fund"] == fund]
            .pivot(index="MA_Window", columns="FR_Window", values=metric)
            .reindex(index=ma_order, columns=fr_order)
        )
        grid.index.name = "MA Window"
        grid.columns.name = "FR Window"
        plot_heatmap(
            grid,
            title=f"{fund}: {metric.replace('_', ' ')} by MA and FR Window",
            cmap=cmap,
            vmin=vmin,
            vmax=vmax,
            center=center,
            fmt=fmt,
        )
```

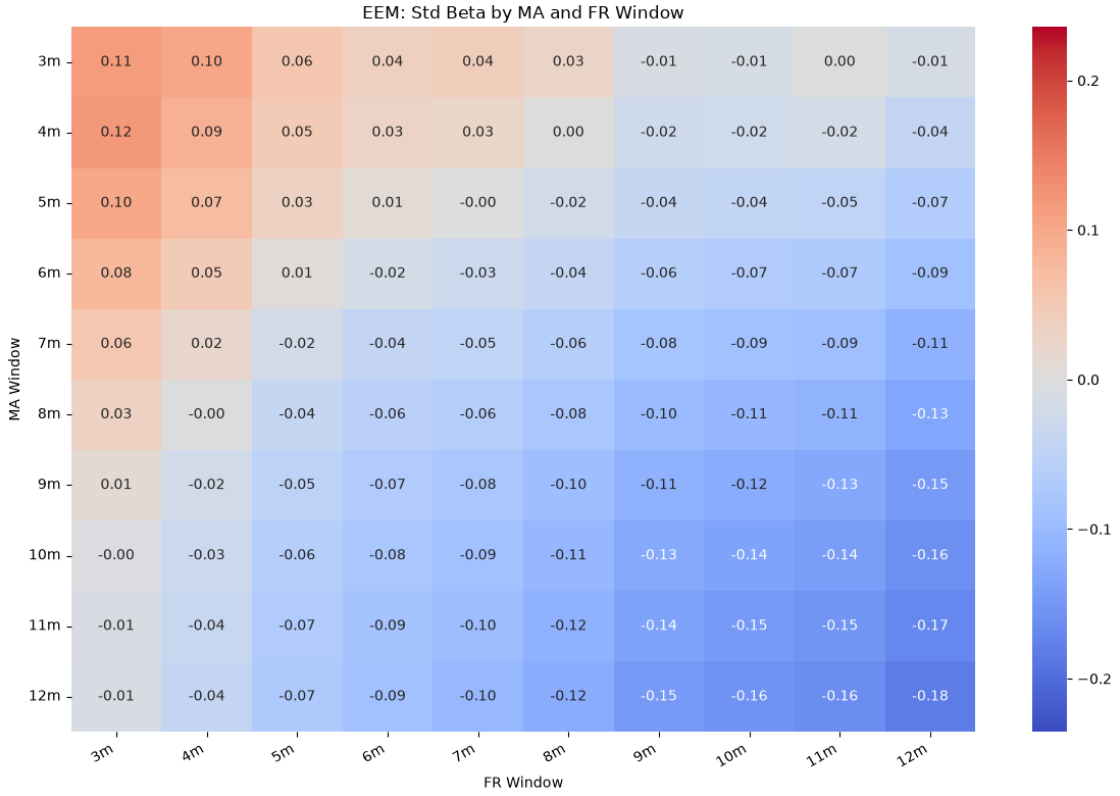
0.9.1 Standardized Beta (Predictive Strength)

The color scale is symmetric around zero (red = positive relationship, blue = negative) and shared across funds so the panels are directly comparable.

[24]:
```
# Symmetric scale so the color range is comparable across funds and nothing␣
↪clips.
beta_limit = ma_prediction_results["Std_Beta"].abs().max()
prediction_heatmap("Std_Beta", vmin=-beta_limit, vmax=beta_limit, center=0.0)
```

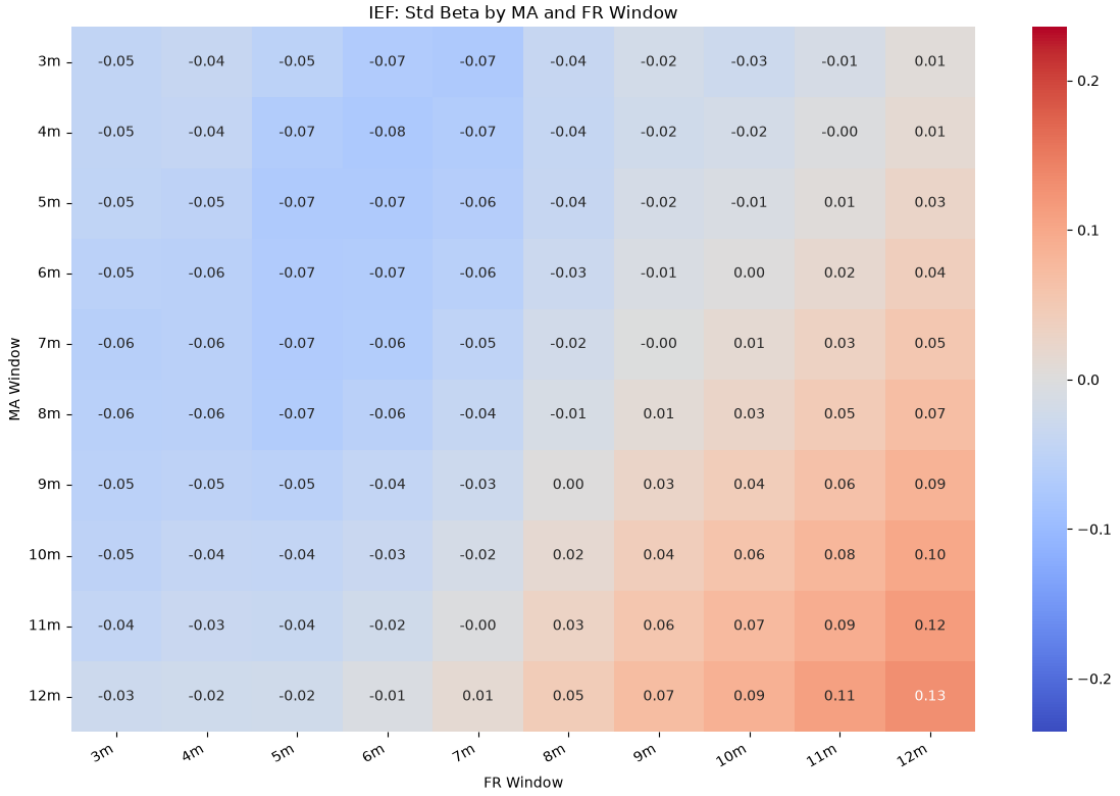

IVV: Std Beta by MA and FR Window
MA Window
FR Window
3m 0.03 0.05 0.04 0.02 0.06 0.07 0.03 0.03 0.05 0.04
4m 0.06 0.06 0.05 0.05 0.08 0.08 0.04 0.04 0.05 0.04
5m 0.07 0.08 0.07 0.07 0.09 0.09 0.06 0.06 0.06 0.04
6m 0.07 0.08 0.07 0.07 0.08 0.08 0.06 0.05 0.05 0.04
7m 0.07 0.08 0.07 0.07 0.08 0.08 0.05 0.04 0.04 0.03
8m 0.07 0.07 0.07 0.06 0.08 0.07 0.04 0.03 0.03 0.02
9m 0.07 0.07 0.07 0.06 0.07 0.06 0.03 0.02 0.02 0.01
10m 0.06 0.07 0.06 0.05 0.06 0.05 0.02 0.01 0.01 0.01
11m 0.06 0.06 0.05 0.04 0.04 0.03 0.01 0.00 0.00 -0.00
12m 0.06 0.06 0.04 0.03 0.03 0.02 -0.00 -0.01 -0.00 -0.01
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.2 0.1 0.0 −0.1 −0.2

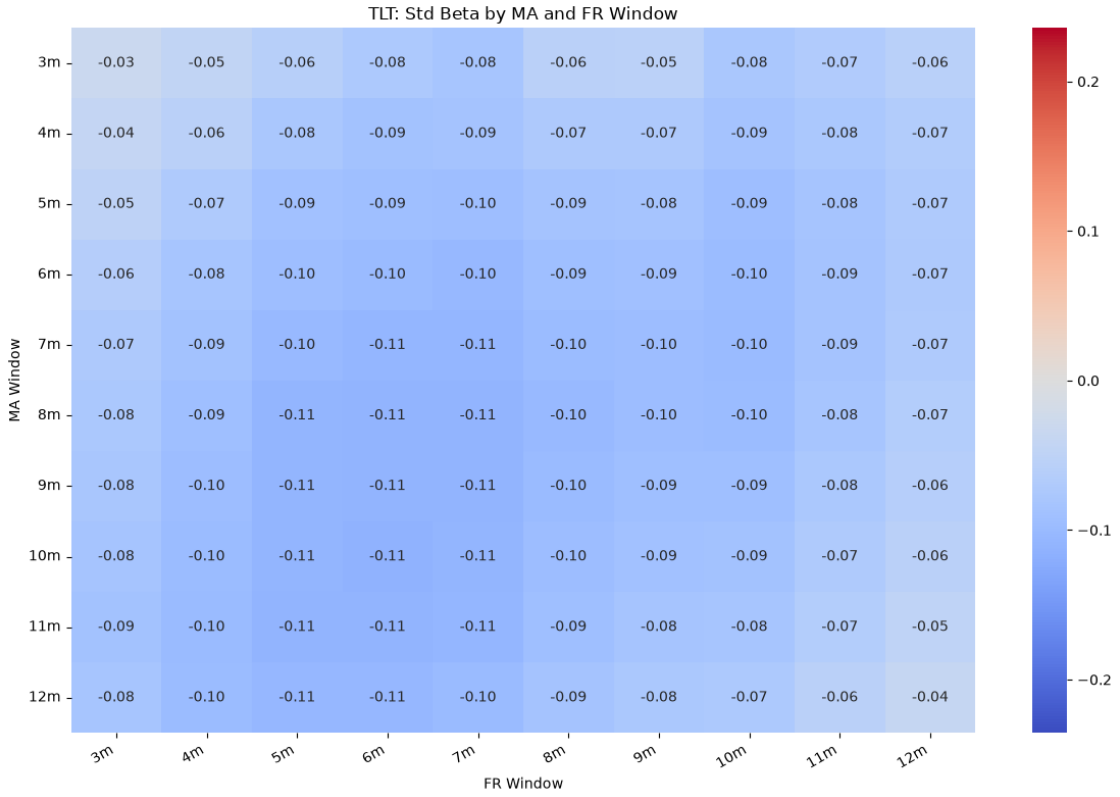

EFA: Std Beta by MA and FR Window
MA Window
FR Window
3m 0.08 0.08 0.07 0.04 0.03 0.02 -0.03 -0.06 -0.07 -0.07
4m 0.10 0.09 0.07 0.05 0.02 0.00 -0.05 -0.08 -0.09 -0.10
5m 0.10 0.09 0.07 0.04 0.01 -0.02 -0.07 -0.10 -0.11 -0.12
6m 0.09 0.08 0.05 0.02 -0.02 -0.05 -0.09 -0.12 -0.13 -0.13
7m 0.08 0.06 0.03 -0.01 -0.05 -0.08 -0.12 -0.14 -0.15 -0.15
8m 0.07 0.04 0.01 -0.04 -0.07 -0.11 -0.14 -0.16 -0.17 -0.17
9m 0.05 0.02 -0.02 -0.06 -0.10 -0.13 -0.16 -0.18 -0.19 -0.19
10m 0.03 -0.00 -0.04 -0.08 -0.12 -0.14 -0.17 -0.19 -0.20 -0.20
11m 0.01 -0.02 -0.06 -0.10 -0.13 -0.16 -0.19 -0.20 -0.21 -0.21
12m -0.01 -0.04 -0.08 -0.12 -0.15 -0.17 -0.20 -0.21 -0.22 -0.22
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.2 0.1 0.0 −0.1 −0.2


EEM: Std Beta by MA and FR Window
MA Window
FR Window
3m
4m
5m
6m
7m
8m
9m
10m
11m
12m
0.11 0.10 0.06 0.04 0.04 0.03 -0.01 -0.01 0.00 -0.01
0.12 0.09 0.05 0.03 0.03 0.00 -0.02 -0.02 -0.02 -0.04
0.10 0.07 0.03 0.01 -0.00 -0.02 -0.04 -0.04 -0.05 -0.07
0.08 0.05 0.01 -0.02 -0.03 -0.04 -0.06 -0.07 -0.07 -0.09
0.06 0.02 -0.02 -0.04 -0.05 -0.06 -0.08 -0.09 -0.09 -0.11
0.03 -0.00 -0.04 -0.06 -0.06 -0.08 -0.10 -0.11 -0.11 -0.13
0.01 -0.02 -0.05 -0.07 -0.08 -0.10 -0.11 -0.12 -0.13 -0.15
-0.00 -0.03 -0.06 -0.08 -0.09 -0.11 -0.13 -0.14 -0.14 -0.16
-0.01 -0.04 -0.07 -0.09 -0.10 -0.12 -0.14 -0.15 -0.15 -0.17
-0.01 -0.04 -0.07 -0.09 -0.10 -0.12 -0.15 -0.16 -0.16 -0.18
0.2
0.1
0.0
−0.1
−0.2


GSG: Std Beta by MA and FR Window
MA Window
FR Window
3m 0.17 0.11 0.10 0.11 0.09 0.08 0.05 0.03 0.01 -0.02
4m 0.14 0.09 0.09 0.09 0.07 0.05 0.03 0.00 -0.02 -0.04
5m 0.11 0.08 0.07 0.07 0.05 0.03 0.00 -0.02 -0.04 -0.06
6m 0.09 0.06 0.05 0.05 0.03 0.01 -0.02 -0.05 -0.06 -0.07
7m 0.08 0.05 0.04 0.03 0.01 -0.01 -0.04 -0.06 -0.08 -0.08
8m 0.07 0.04 0.03 0.02 -0.00 -0.03 -0.05 -0.07 -0.08 -0.09
9m 0.06 0.03 0.01 0.00 -0.02 -0.04 -0.06 -0.08 -0.09 -0.10
10m 0.05 0.02 0.00 -0.01 -0.03 -0.05 -0.07 -0.09 -0.10 -0.11
11m 0.04 0.01 -0.01 -0.02 -0.04 -0.06 -0.08 -0.10 -0.11 -0.11
12m 0.03 -0.00 -0.02 -0.03 -0.05 -0.07 -0.09 -0.10 -0.11 -0.11
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.2 0.1 0.0 −0.1 −0.2


IAU: Std Beta by MA and FR Window
MA Window
FR Window
3m -0.02 0.00 0.03 0.05 0.01 -0.03 -0.03 0.04 0.09 0.08
4m -0.01 0.03 0.05 0.05 -0.00 -0.03 -0.00 0.07 0.11 0.10
5m 0.01 0.04 0.06 0.05 0.00 -0.01 0.03 0.09 0.13 0.12
6m 0.03 0.05 0.06 0.05 0.02 0.02 0.06 0.12 0.16 0.15
7m 0.04 0.05 0.06 0.05 0.03 0.03 0.08 0.14 0.17 0.16
8m 0.03 0.05 0.06 0.06 0.04 0.05 0.09 0.15 0.19 0.18
9m 0.03 0.05 0.06 0.07 0.05 0.06 0.11 0.17 0.20 0.20
10m 0.03 0.05 0.07 0.07 0.06 0.07 0.12 0.18 0.21 0.21
11m 0.04 0.06 0.07 0.08 0.07 0.08 0.13 0.19 0.22 0.22
12m 0.04 0.06 0.08 0.08 0.08 0.09 0.14 0.20 0.24 0.23
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.2 0.1 0.0 −0.1 −0.2


IEF: Std Beta by MA and FR Window
MA Window
FR Window
3m -0.05 -0.04 -0.05 -0.07 -0.07 -0.04 -0.02 -0.03 -0.01 0.01
4m -0.05 -0.04 -0.07 -0.08 -0.07 -0.04 -0.02 -0.02 -0.00 0.01
5m -0.05 -0.05 -0.07 -0.07 -0.06 -0.04 -0.02 -0.01 0.01 0.03
6m -0.05 -0.06 -0.07 -0.07 -0.06 -0.03 -0.01 0.00 0.02 0.04
7m -0.06 -0.06 -0.07 -0.06 -0.05 -0.02 -0.00 0.01 0.03 0.05
8m -0.06 -0.06 -0.07 -0.06 -0.04 -0.01 0.01 0.03 0.05 0.07
9m -0.05 -0.05 -0.05 -0.04 -0.03 0.00 0.03 0.04 0.06 0.09
10m -0.05 -0.04 -0.04 -0.03 -0.02 0.02 0.04 0.06 0.08 0.10
11m -0.04 -0.03 -0.04 -0.02 -0.00 0.03 0.06 0.07 0.09 0.12
12m -0.03 -0.02 -0.02 -0.01 0.01 0.05 0.07 0.09 0.11 0.13
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.2 0.1 0.0 −0.1 −0.2



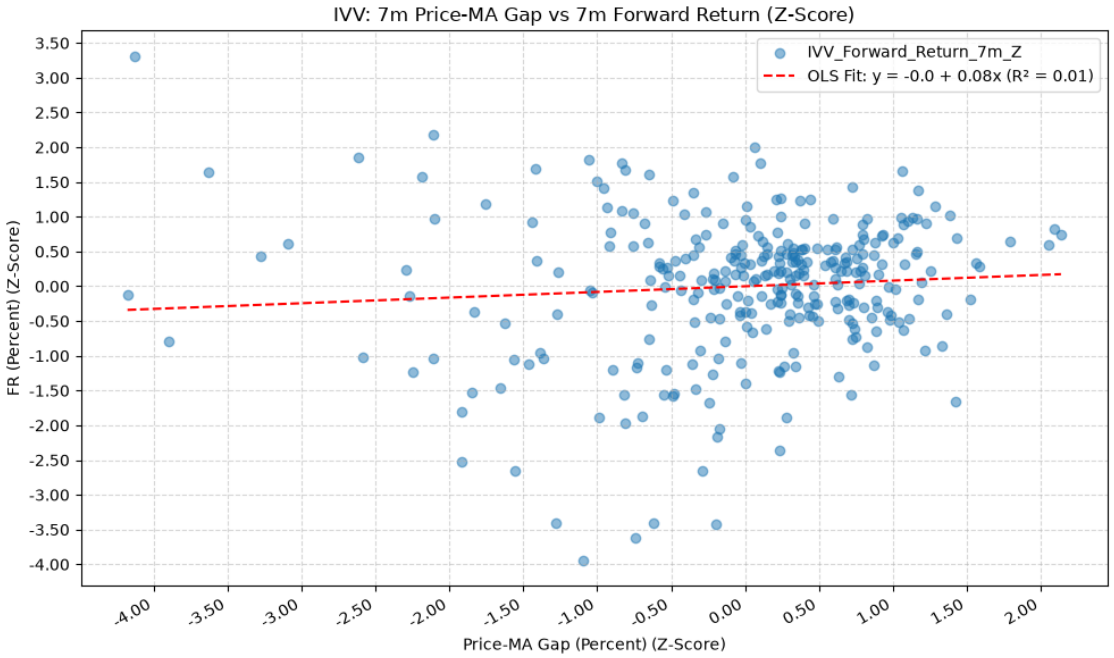
Standardized Beta Example #1 To see what underlies one cell of the heatmaps, here is the granular view for IVV using the 7-month MA and the 7-month forward return, with the scatter of the price-MA gap against the forward return (with a fitted OLS line).

[25]:
```
example_fund = "IVV"
example_ma = "7m"
example_fr = "7m"
fr_window = 7

pair_data = pair_data_dict[(example_fund, example_ma, example_fr)]

plot_scatter(
    df=pair_data,
    x_plot_column=f"{example_fund}_Price_MA_Diff_Percent_{example_ma}_Z",
    y_plot_columns=[f"{example_fund}_Forward_Return_{example_fr}_Z"],
    title=f"{example_fund}: {example_ma} Price-MA Gap vs {example_fr} Forward␣
 ↪Return (Z-Score)",
    x_label="Price-MA Gap (Percent) (Z-Score)",
    x_format="Decimal",
    x_format_decimal_places=2,
    x_tick_spacing="Auto",
```

```
    x_tick_start=None,
    x_tick_rotation=30,
    y_label="FR (Percent) (Z-Score)",
    y_format="Decimal",
    y_format_decimal_places=2,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    plot_OLS_regression_line=True,
    OLS_column=f"{example_fund}_Forward_Return_{example_fr}_Z",
    plot_Ridge_regression_line=False,
    Ridge_column=None,
    plot_RidgeCV_regression_line=False,
    RidgeCV_column=None,
    regression_constant=True,
    grid=True,
    legend=True,
    export_plot=False,
    plot_file_name=None,
)
```

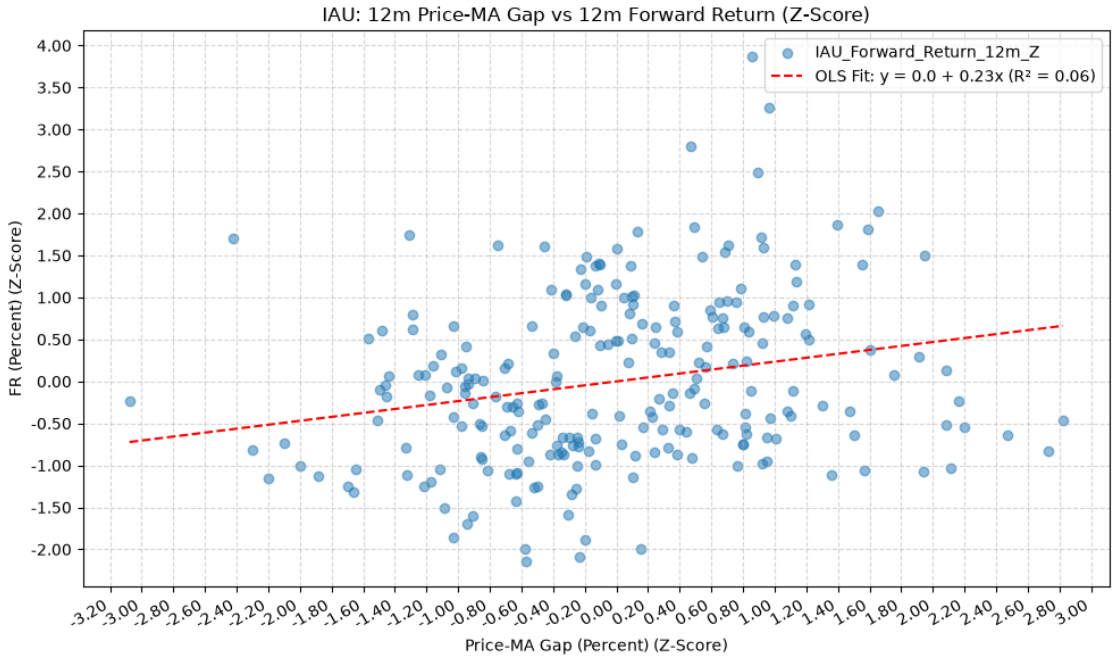


Standardized Beta Example #2 For a second example, we'll use IAU with the 12-month MA and the 12-month forward return, again with the scatter of the price-MA gap against the forward return (with a fitted OLS line).

[26]:
```
example_fund = "IAU"
example_ma = "12m"
example_fr = "12m"
fr_window = 12

pair_data = pair_data_dict[(example_fund, example_ma, example_fr)]

plot_scatter(
    df=pair_data,
    x_plot_column=f"{example_fund}_Price_MA_Diff_Percent_{example_ma}_Z",
    y_plot_columns=[f"{example_fund}_Forward_Return_{example_fr}_Z"],
    title=f"{example_fund}: {example_ma} Price-MA Gap vs {example_fr} Forward␣
 ↪Return (Z-Score)",
    x_label="Price-MA Gap (Percent) (Z-Score)",
    x_format="Decimal",
    x_format_decimal_places=2,
    x_tick_spacing="Auto",
    x_tick_start=None,
    x_tick_rotation=30,
    y_label="FR (Percent) (Z-Score)",
    y_format="Decimal",
    y_format_decimal_places=2,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    plot_OLS_regression_line=True,
    OLS_column=f"{example_fund}_Forward_Return_{example_fr}_Z",
    plot_Ridge_regression_line=False,
    Ridge_column=None,
    plot_RidgeCV_regression_line=False,
    RidgeCV_column=None,
    regression_constant=True,
    grid=True,
    legend=True,
    export_plot=False,
    plot_file_name=None,
)
```



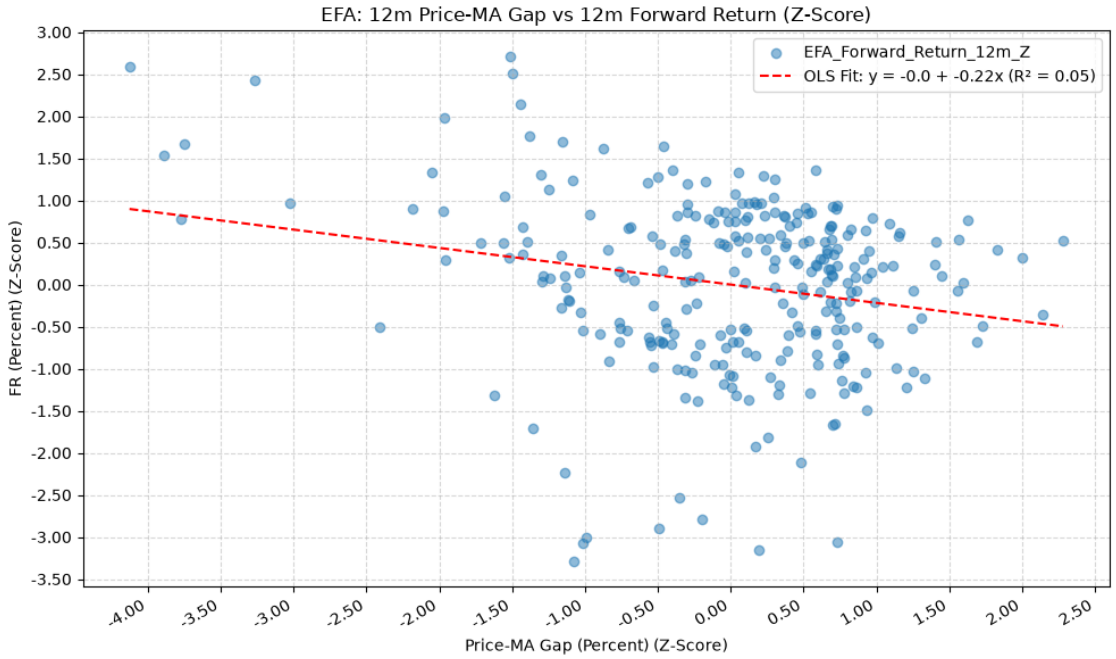
Standardized Beta Example #3 Finally, as a counter example to #2 (which presented positive results), we'll look at EFA with the 12-month MA and the 12-month forward return.

[27]:
```
example_fund = "EFA"
example_ma = "12m"
example_fr = "12m"
fr_window = 12

pair_data = pair_data_dict[(example_fund, example_ma, example_fr)]

plot_scatter(
    df=pair_data,
    x_plot_column=f"{example_fund}_Price_MA_Diff_Percent_{example_ma}_Z",
    y_plot_columns=[f"{example_fund}_Forward_Return_{example_fr}_Z"],
    title=f"{example_fund}: {example_ma} Price-MA Gap vs {example_fr} Forward␣
 ↪Return (Z-Score)",
    x_label="Price-MA Gap (Percent) (Z-Score)",
    x_format="Decimal",
    x_format_decimal_places=2,
    x_tick_spacing="Auto",
    x_tick_start=None,
    x_tick_rotation=30,
    y_label="FR (Percent) (Z-Score)",
    y_format="Decimal",
    y_format_decimal_places=2,
```

```
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    plot_OLS_regression_line=True,
    OLS_column=f"{example_fund}_Forward_Return_{example_fr}_Z",
    plot_Ridge_regression_line=False,
    Ridge_column=None,
    plot_RidgeCV_regression_line=False,
    RidgeCV_column=None,
    regression_constant=True,
    grid=True,
    legend=True,
    export_plot=False,
    plot_file_name=None,
)
```



The difference between examples #2 and #3 is quite interesting. For example #2 using gold, there appears to be a positive relationship between the price-MA gap and the forward return, whereas for example #3 using EFA, this relationship is also evident but in a negative way.

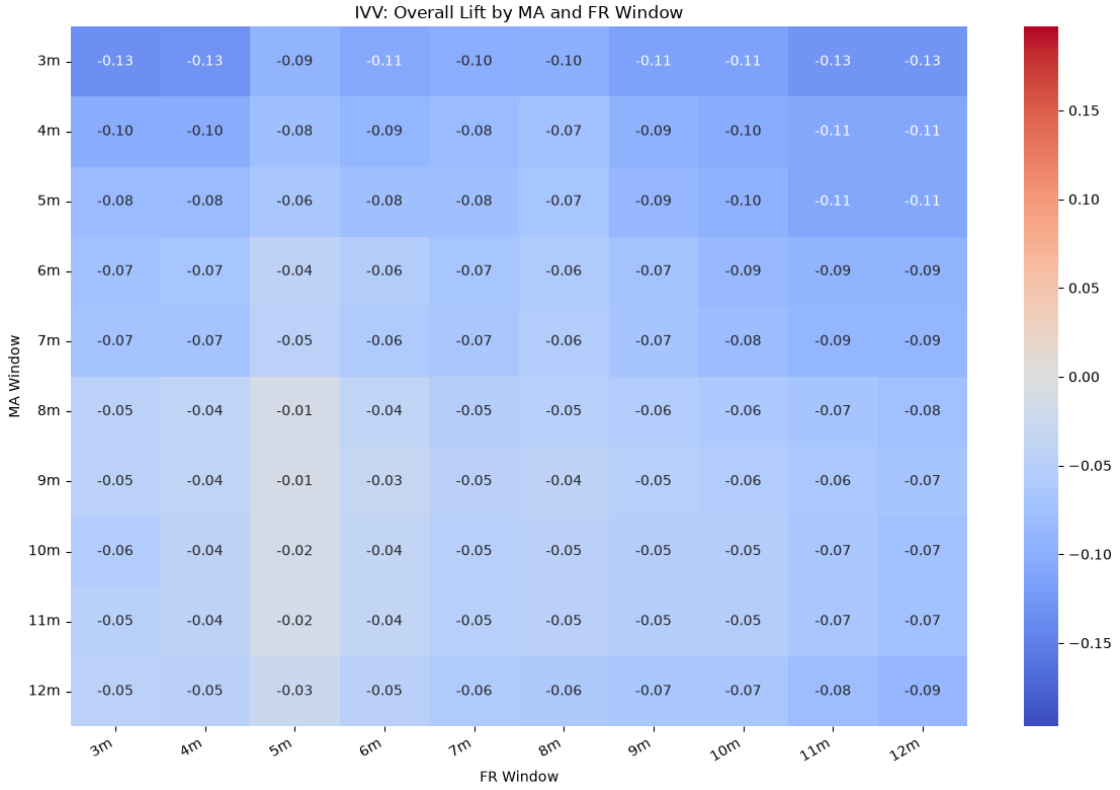
It's difficult to draw any firm conclusions from the heatmaps. Regardless, it is an interesting study into the relationship between price-MA and return.

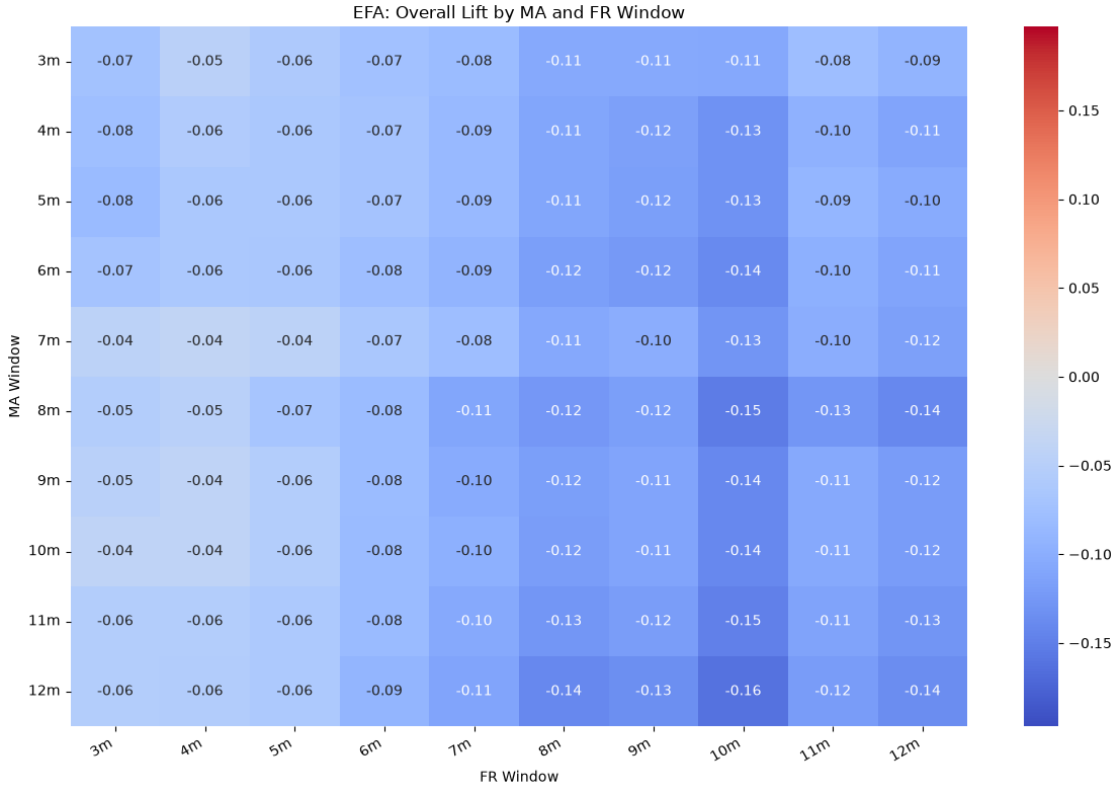
0.9.2 Overall Accuracy Lift

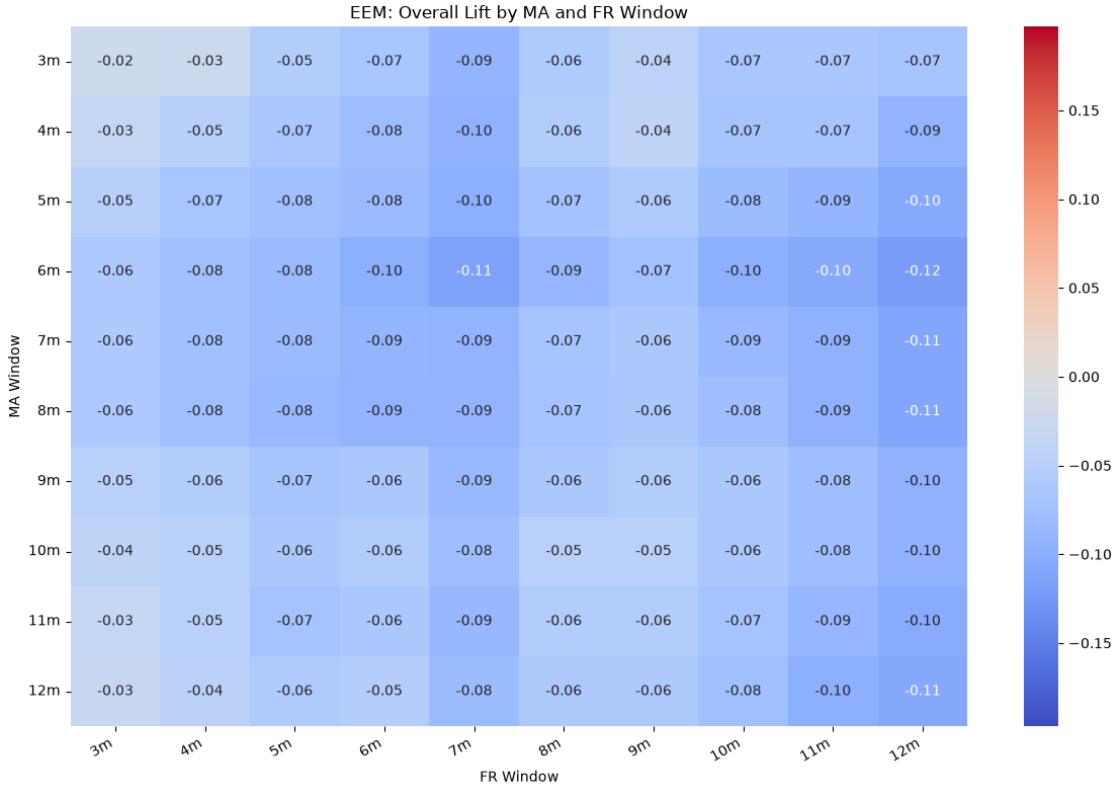
Accuracy lift is the overall accuracy minus its naive benchmark, the majority-class rate `max(base rate, 1 - base rate)` (i.e. how much the price-MA sign beats simply always predicting whichever

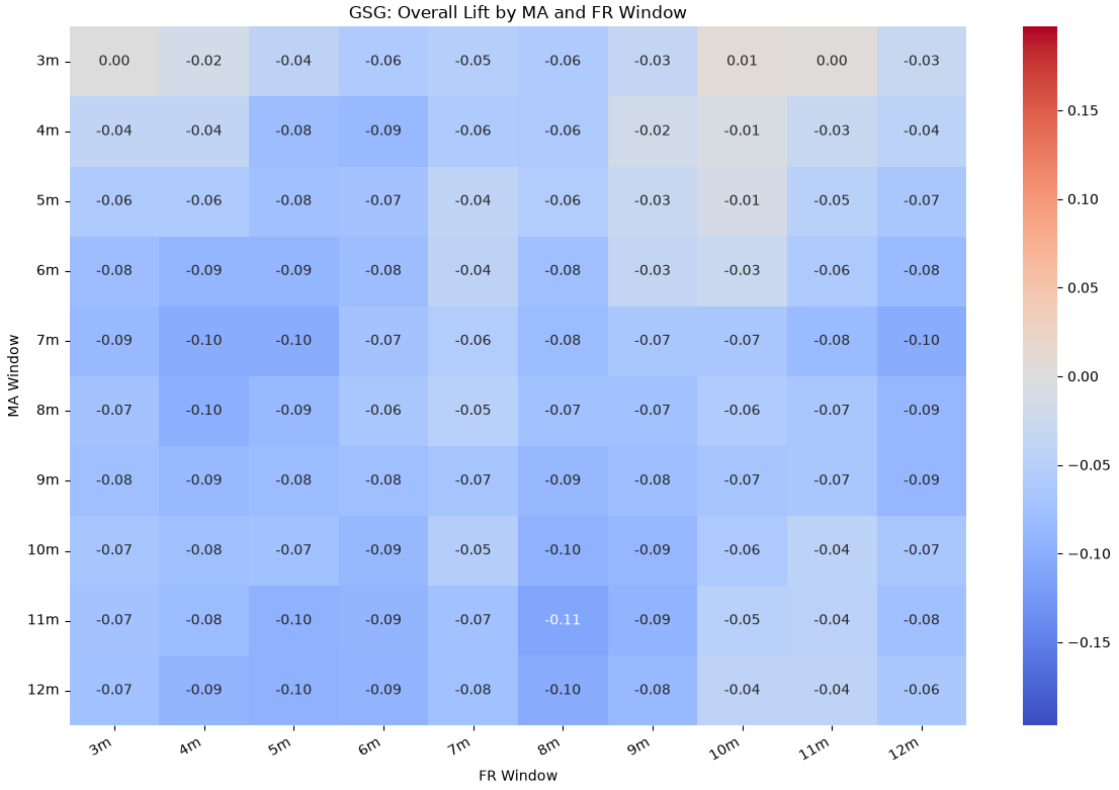
direction is more common). The heatmap is centered on zero, so red cells add value over the naive benchmark, and blue cells do not.

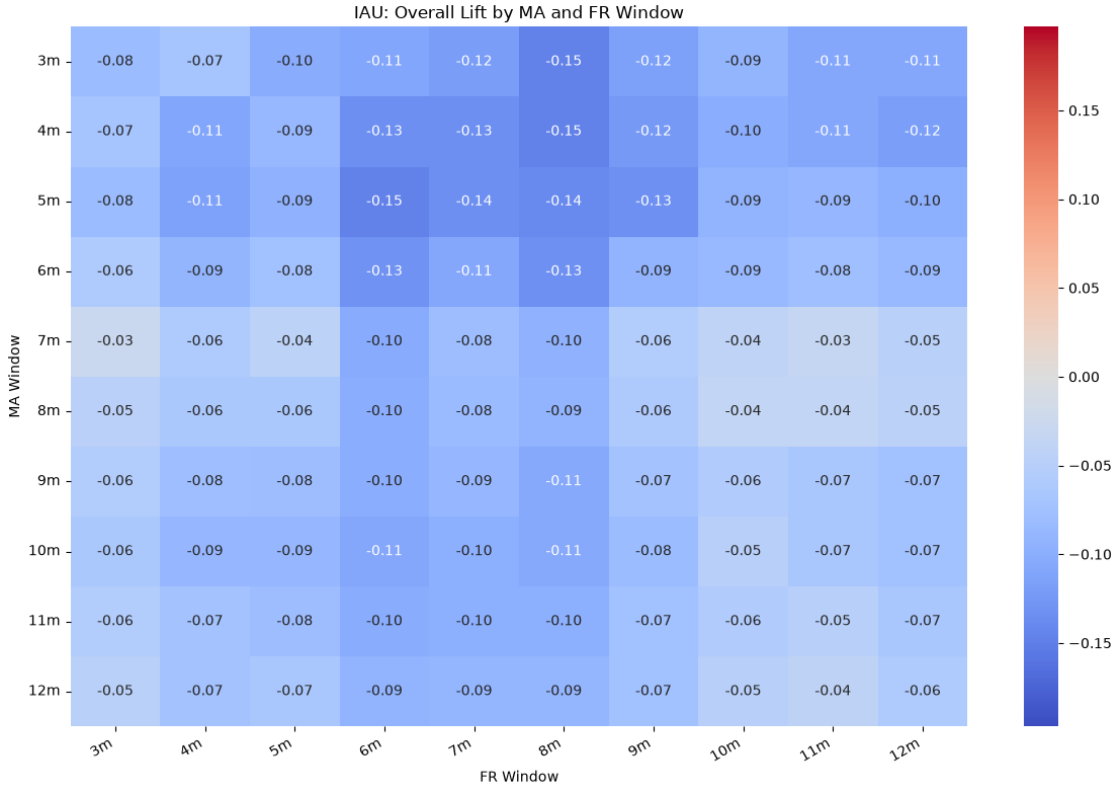
[28]:
```
# Symmetric scale around zero so the color encodes sign and magnitude of the lift.
overall_lift_limit = ma_prediction_results["Overall_Lift"].abs().max()
prediction_heatmap("Overall_Lift", vmin=-overall_lift_limit, vmax=overall_lift_limit, center=0.0)
```

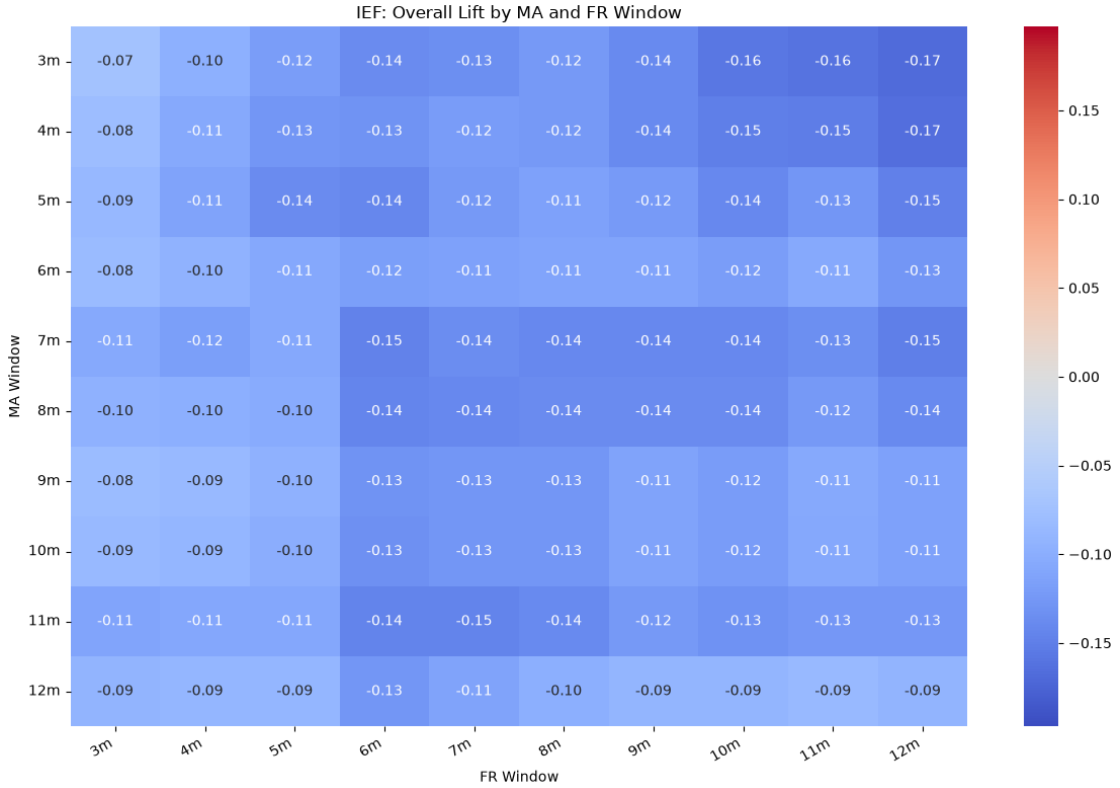


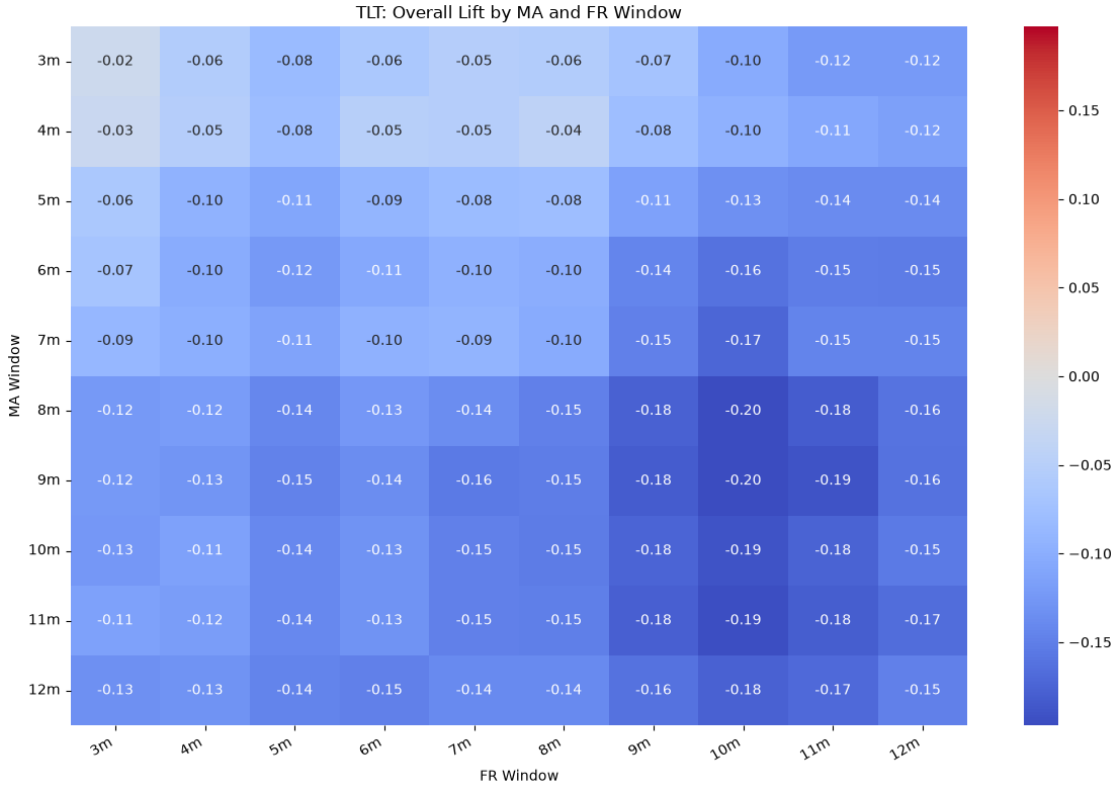

EFA: Overall Lift by MA and FR Window
MA Window
FR Window
3m -0.07 -0.05 -0.06 -0.07 -0.08 -0.11 -0.11 -0.11 -0.08 -0.09
4m -0.08 -0.06 -0.06 -0.07 -0.09 -0.11 -0.12 -0.13 -0.10 -0.11
5m -0.08 -0.06 -0.06 -0.07 -0.09 -0.11 -0.12 -0.13 -0.09 -0.10
6m -0.07 -0.06 -0.06 -0.08 -0.09 -0.12 -0.12 -0.14 -0.10 -0.11
7m -0.04 -0.04 -0.04 -0.07 -0.08 -0.11 -0.10 -0.13 -0.10 -0.12
8m -0.05 -0.05 -0.07 -0.08 -0.11 -0.12 -0.12 -0.15 -0.13 -0.14
9m -0.05 -0.04 -0.06 -0.08 -0.10 -0.12 -0.11 -0.14 -0.11 -0.12
10m -0.04 -0.04 -0.06 -0.08 -0.10 -0.12 -0.11 -0.14 -0.11 -0.12
11m -0.06 -0.06 -0.06 -0.08 -0.10 -0.13 -0.12 -0.15 -0.11 -0.13
12m -0.06 -0.06 -0.06 -0.09 -0.11 -0.14 -0.13 -0.16 -0.12 -0.14
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.15 0.10 0.05 0.00 −0.05 −0.10 −0.15


EEM: Overall Lift by MA and FR Window
MA Window
FR Window
3m -0.02 -0.03 -0.05 -0.07 -0.09 -0.06 -0.04 -0.07 -0.07 -0.07
4m -0.03 -0.05 -0.07 -0.08 -0.10 -0.06 -0.04 -0.07 -0.07 -0.09
5m -0.05 -0.07 -0.08 -0.08 -0.10 -0.07 -0.06 -0.08 -0.09 -0.10
6m -0.06 -0.08 -0.08 -0.10 -0.11 -0.09 -0.07 -0.10 -0.10 -0.12
7m -0.06 -0.08 -0.08 -0.09 -0.09 -0.07 -0.06 -0.09 -0.09 -0.11
8m -0.06 -0.08 -0.08 -0.09 -0.09 -0.07 -0.06 -0.08 -0.09 -0.11
9m -0.05 -0.06 -0.07 -0.06 -0.09 -0.06 -0.06 -0.06 -0.08 -0.10
10m -0.04 -0.05 -0.06 -0.06 -0.08 -0.05 -0.05 -0.06 -0.08 -0.10
11m -0.03 -0.05 -0.07 -0.06 -0.09 -0.06 -0.06 -0.07 -0.09 -0.10
12m -0.03 -0.04 -0.06 -0.05 -0.08 -0.06 -0.06 -0.08 -0.10 -0.11
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.15 0.10 0.05 0.00 −0.05 −0.10 −0.15


GSG: Overall Lift by MA and FR Window
MA Window
FR Window
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.00 -0.02 -0.04 -0.06 -0.05 -0.06 -0.03 0.01 0.00 -0.03
-0.04 -0.04 -0.08 -0.09 -0.06 -0.06 -0.02 -0.01 -0.03 -0.04
-0.06 -0.06 -0.08 -0.07 -0.04 -0.06 -0.03 -0.01 -0.05 -0.07
-0.08 -0.09 -0.09 -0.08 -0.04 -0.08 -0.03 -0.03 -0.06 -0.08
-0.09 -0.10 -0.10 -0.07 -0.06 -0.08 -0.07 -0.07 -0.08 -0.10
-0.07 -0.10 -0.09 -0.06 -0.05 -0.07 -0.07 -0.06 -0.07 -0.09
-0.08 -0.09 -0.08 -0.08 -0.07 -0.09 -0.08 -0.07 -0.07 -0.09
-0.07 -0.08 -0.07 -0.09 -0.05 -0.10 -0.09 -0.06 -0.04 -0.07
-0.07 -0.08 -0.10 -0.09 -0.07 -0.11 -0.09 -0.05 -0.04 -0.08
-0.07 -0.09 -0.10 -0.09 -0.08 -0.10 -0.08 -0.04 -0.04 -0.06
0.15 0.10 0.05 0.00 −0.05 −0.10 −0.15


IAU: Overall Lift by MA and FR Window
MA Window
FR Window
3m -0.08 -0.07 -0.10 -0.11 -0.12 -0.15 -0.12 -0.09 -0.11 -0.11
4m -0.07 -0.11 -0.09 -0.13 -0.13 -0.15 -0.12 -0.10 -0.11 -0.12
5m -0.08 -0.11 -0.09 -0.15 -0.14 -0.14 -0.13 -0.09 -0.09 -0.10
6m -0.06 -0.09 -0.08 -0.13 -0.11 -0.13 -0.09 -0.09 -0.08 -0.09
7m -0.03 -0.06 -0.04 -0.10 -0.08 -0.10 -0.06 -0.04 -0.03 -0.05
8m -0.05 -0.06 -0.06 -0.10 -0.08 -0.09 -0.06 -0.04 -0.04 -0.05
9m -0.06 -0.08 -0.08 -0.10 -0.09 -0.11 -0.07 -0.06 -0.07 -0.07
10m -0.06 -0.09 -0.09 -0.11 -0.10 -0.11 -0.08 -0.05 -0.07 -0.07
11m -0.06 -0.07 -0.08 -0.10 -0.10 -0.10 -0.07 -0.06 -0.05 -0.07
12m -0.05 -0.07 -0.07 -0.09 -0.09 -0.09 -0.07 -0.05 -0.04 -0.06
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.15 0.10 0.05 0.00 −0.05 −0.10 −0.15


IEF: Overall Lift by MA and FR Window
MA Window
FR Window
3m -0.07 -0.10 -0.12 -0.14 -0.13 -0.12 -0.14 -0.16 -0.16 -0.17
4m -0.08 -0.11 -0.13 -0.13 -0.12 -0.12 -0.14 -0.15 -0.15 -0.17
5m -0.09 -0.11 -0.14 -0.14 -0.12 -0.11 -0.12 -0.14 -0.13 -0.15
6m -0.08 -0.10 -0.11 -0.12 -0.11 -0.11 -0.11 -0.12 -0.11 -0.13
7m -0.11 -0.12 -0.11 -0.15 -0.14 -0.14 -0.14 -0.14 -0.13 -0.15
8m -0.10 -0.10 -0.10 -0.14 -0.14 -0.14 -0.14 -0.14 -0.12 -0.14
9m -0.08 -0.09 -0.10 -0.13 -0.13 -0.13 -0.11 -0.12 -0.11 -0.11
10m -0.09 -0.09 -0.10 -0.13 -0.13 -0.13 -0.11 -0.12 -0.11 -0.11
11m -0.11 -0.11 -0.11 -0.14 -0.15 -0.14 -0.12 -0.13 -0.13 -0.13
12m -0.09 -0.09 -0.09 -0.13 -0.11 -0.10 -0.09 -0.09 -0.09 -0.09
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.15 0.10 0.05 0.00 −0.05 −0.10 −0.15

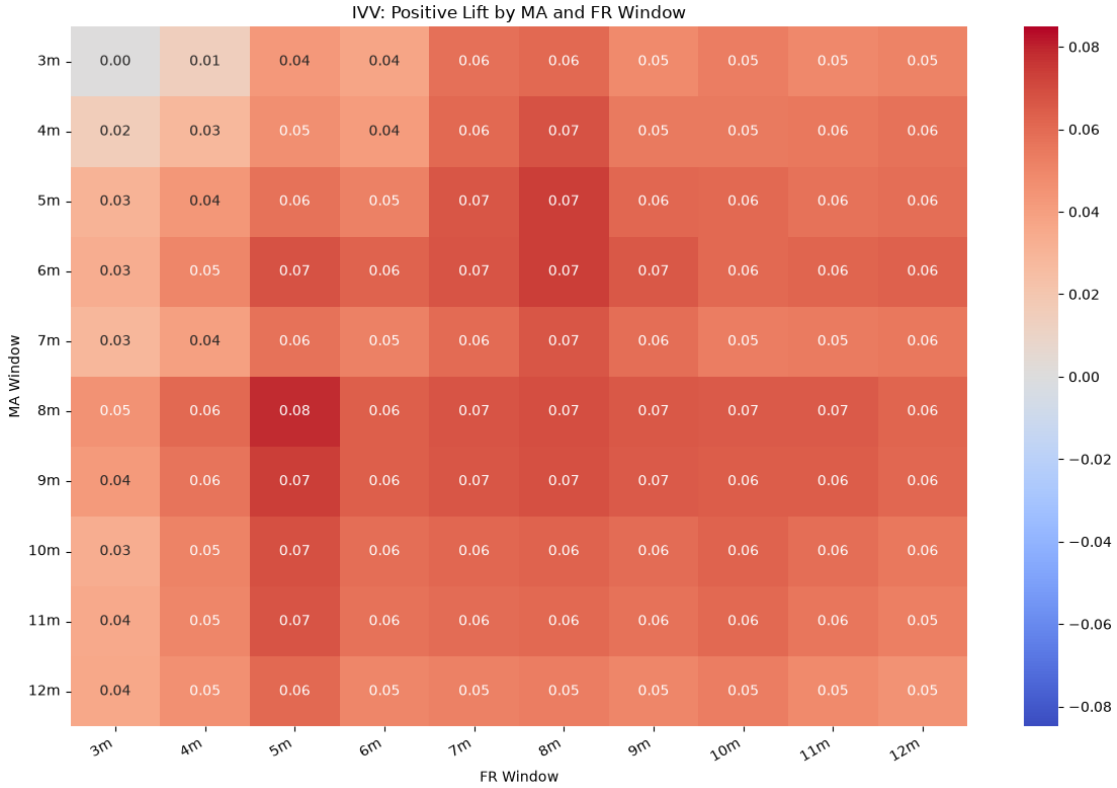


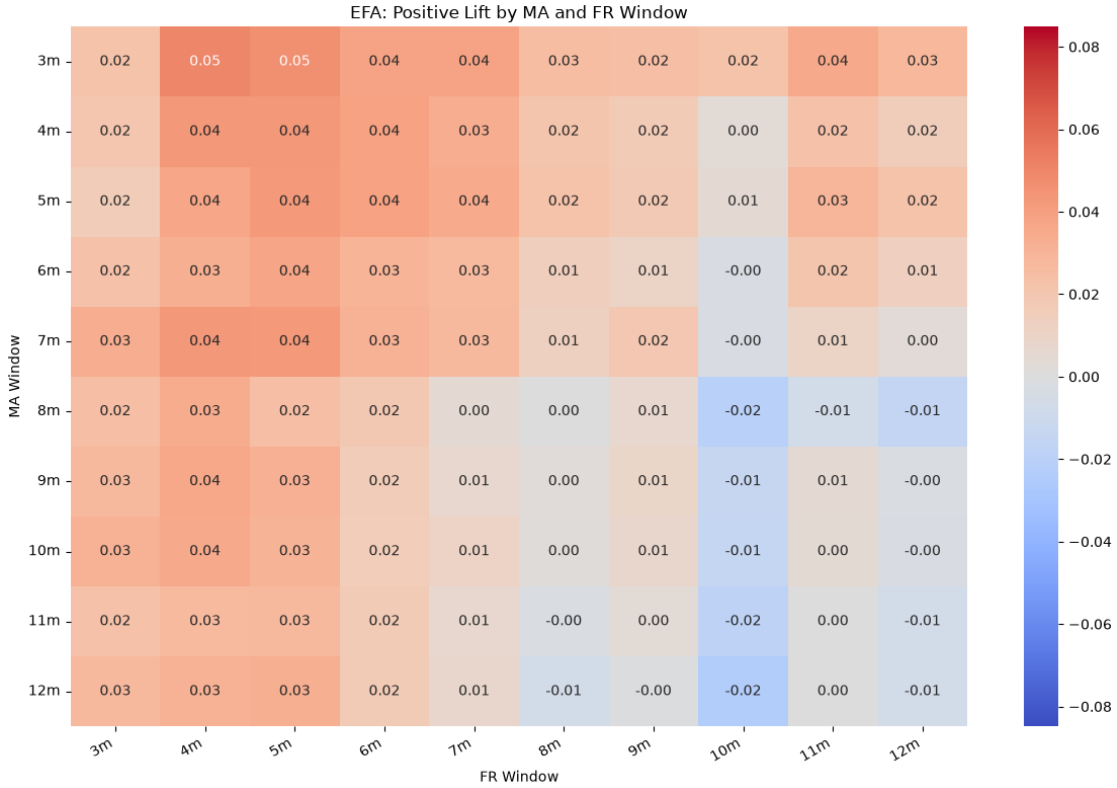
Based on the above headmaps, we can say with confidence that the overall accuracy categorically does not beat the base rate. But… what if we separate the overall accuracy into positive and negative accuracy?

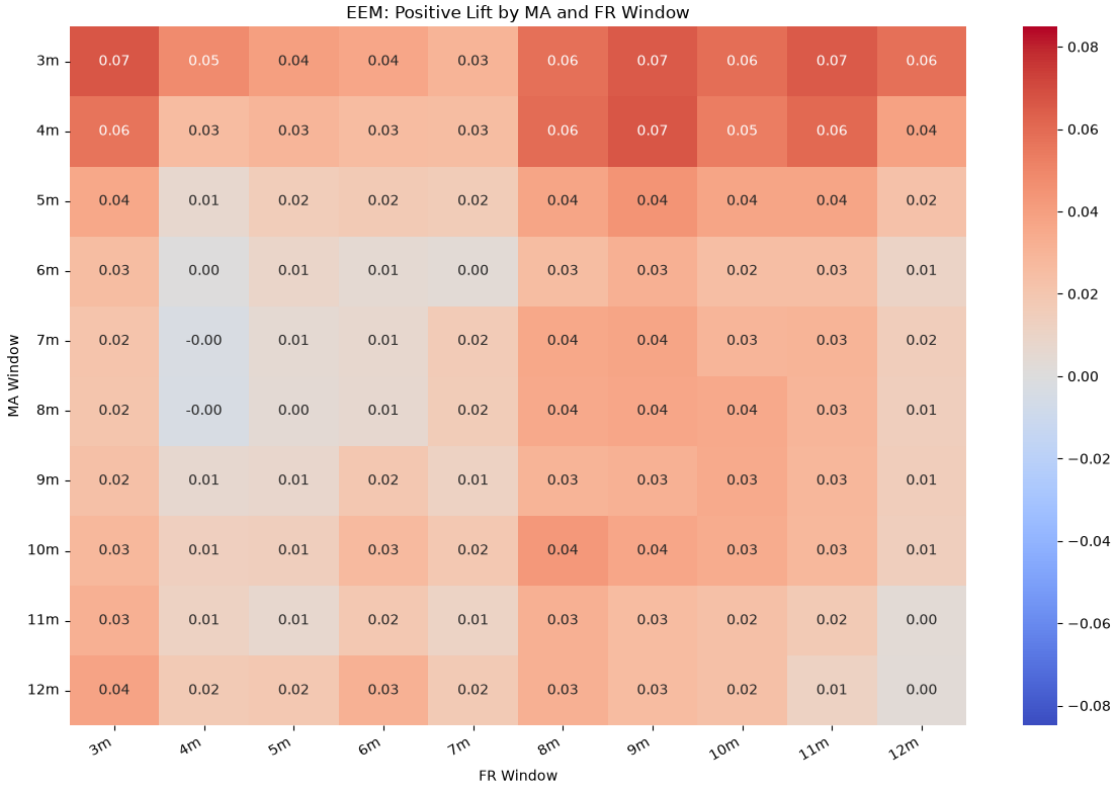
0.9.3 Positive-Prediction Lift

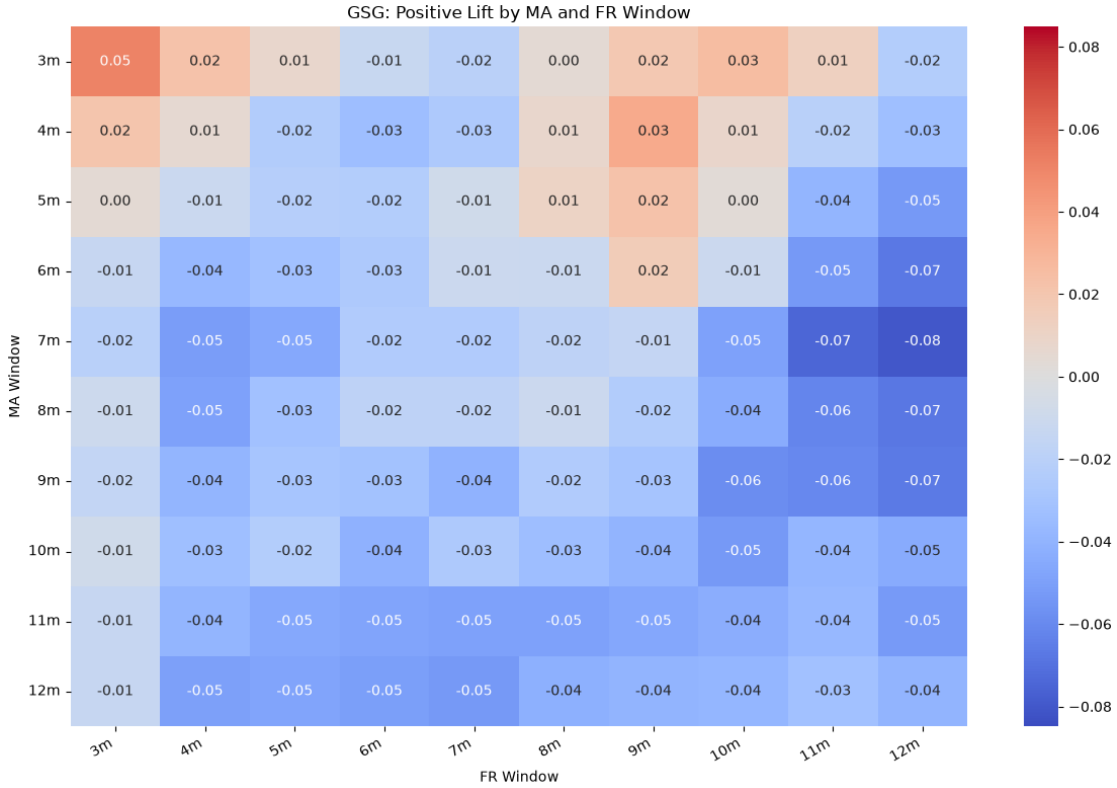
From a long-only standpoint, we care only about the "up" prediction. When the price is above its MA, how often is the forward return positive, *relative to the base rate*? This heatmap is `Positive_Accuracy - base_rate`. Any value above zero indicates that the MA adds predictive power over the base rate.

[29]:
```
# Symmetric scale around zero.
positive_lift_limit = ma_prediction_results["Positive_Lift"].abs().max()
prediction_heatmap("Positive_Lift", vmin=-positive_lift_limit,␣
  ↪vmax=positive_lift_limit, center=0.0)
```


IVV: Positive Lift by MA and FR Window
MA Window
FR Window
3m 0.00 0.01 0.04 0.04 0.06 0.06 0.05 0.05 0.05 0.05
4m 0.02 0.03 0.05 0.04 0.06 0.07 0.05 0.05 0.06 0.06
5m 0.03 0.04 0.06 0.05 0.07 0.07 0.06 0.06 0.06 0.06
6m 0.03 0.05 0.07 0.06 0.07 0.07 0.07 0.06 0.06 0.06
7m 0.03 0.04 0.06 0.05 0.06 0.07 0.06 0.05 0.05 0.06
8m 0.05 0.06 0.08 0.06 0.07 0.07 0.07 0.07 0.07 0.06
9m 0.04 0.06 0.07 0.06 0.07 0.07 0.07 0.06 0.06 0.06
10m 0.03 0.05 0.07 0.06 0.06 0.06 0.06 0.06 0.06 0.06
11m 0.04 0.05 0.07 0.06 0.06 0.06 0.06 0.06 0.06 0.05
12m 0.04 0.05 0.06 0.05 0.05 0.05 0.05 0.05 0.05 0.05
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.08 0.06 0.04 0.02 0.00 −0.02 −0.04 −0.06 −0.08

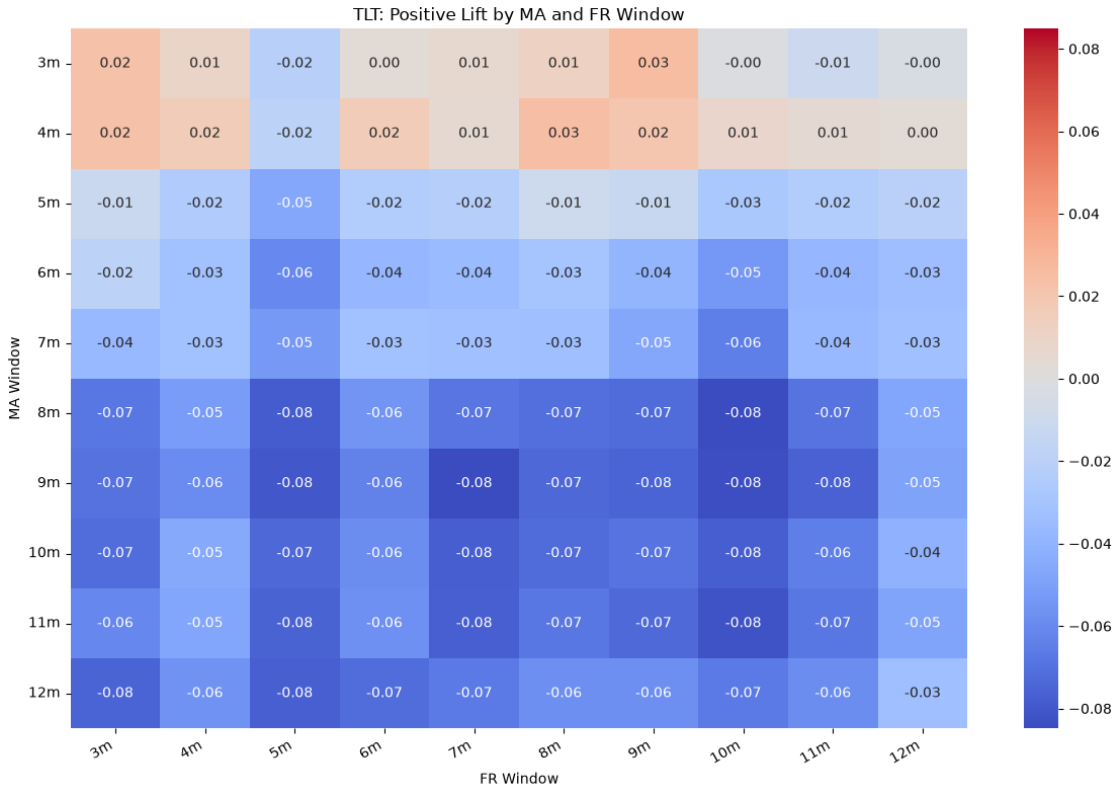

EFA: Positive Lift by MA and FR Window
MA Window
FR Window
3m 0.02 0.05 0.05 0.04 0.04 0.03 0.02 0.02 0.04 0.03
4m 0.02 0.04 0.04 0.04 0.03 0.02 0.02 0.00 0.02 0.02
5m 0.02 0.04 0.04 0.04 0.04 0.02 0.02 0.01 0.03 0.02
6m 0.02 0.03 0.04 0.03 0.03 0.01 0.01 -0.00 0.02 0.01
7m 0.03 0.04 0.04 0.03 0.03 0.01 0.02 -0.00 0.01 0.00
8m 0.02 0.03 0.02 0.02 0.00 0.00 0.01 -0.02 -0.01 -0.01
9m 0.03 0.04 0.03 0.02 0.01 0.00 0.01 -0.01 0.01 -0.00
10m 0.03 0.04 0.03 0.02 0.01 0.00 0.01 -0.01 0.00 -0.00
11m 0.02 0.03 0.03 0.02 0.01 -0.00 0.00 -0.02 0.00 -0.01
12m 0.03 0.03 0.03 0.02 0.01 -0.01 -0.00 -0.02 0.00 -0.01
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.08 0.06 0.04 0.02 0.00 −0.02 −0.04 −0.06 −0.08


EEM: Positive Lift by MA and FR Window
MA Window
FR Window
3m 0.07 0.05 0.04 0.04 0.03 0.06 0.07 0.06 0.07 0.06
4m 0.06 0.03 0.03 0.03 0.03 0.06 0.07 0.05 0.06 0.04
5m 0.04 0.01 0.02 0.02 0.02 0.04 0.04 0.04 0.04 0.02
6m 0.03 0.00 0.01 0.01 0.00 0.03 0.03 0.02 0.03 0.01
7m 0.02 -0.00 0.01 0.01 0.02 0.04 0.04 0.03 0.03 0.02
8m 0.02 -0.00 0.00 0.01 0.02 0.04 0.04 0.04 0.03 0.01
9m 0.02 0.01 0.01 0.02 0.01 0.03 0.03 0.03 0.03 0.01
10m 0.03 0.01 0.01 0.03 0.02 0.04 0.04 0.03 0.03 0.01
11m 0.03 0.01 0.01 0.02 0.01 0.03 0.03 0.02 0.02 0.00
12m 0.04 0.02 0.02 0.03 0.02 0.03 0.03 0.02 0.01 0.00
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.08 0.06 0.04 0.02 0.00 −0.02 −0.04 −0.06 −0.08


GSG: Positive Lift by MA and FR Window
MA Window
FR Window
3m 0.05 0.02 0.01 -0.01 -0.02 0.00 0.02 0.03 0.01 -0.02
4m 0.02 0.01 -0.02 -0.03 -0.03 0.01 0.03 0.01 -0.02 -0.03
5m 0.00 -0.01 -0.02 -0.02 -0.01 0.01 0.02 0.00 -0.04 -0.05
6m -0.01 -0.04 -0.03 -0.03 -0.01 -0.01 0.02 -0.01 -0.05 -0.07
7m -0.02 -0.05 -0.05 -0.02 -0.02 -0.02 -0.01 -0.05 -0.07 -0.08
8m -0.01 -0.05 -0.03 -0.02 -0.02 -0.01 -0.02 -0.04 -0.06 -0.07
9m -0.02 -0.04 -0.03 -0.03 -0.04 -0.02 -0.03 -0.06 -0.06 -0.07
10m -0.01 -0.03 -0.02 -0.04 -0.03 -0.03 -0.04 -0.05 -0.04 -0.05
11m -0.01 -0.04 -0.05 -0.05 -0.05 -0.05 -0.05 -0.04 -0.04 -0.05
12m -0.01 -0.05 -0.05 -0.05 -0.05 -0.04 -0.04 -0.04 -0.03 -0.04
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.08 0.06 0.04 0.02 0.00 −0.02 −0.04 −0.06 −0.08


IAU: Positive Lift by MA and FR Window
MA Window
FR Window
3m 0.02 0.06 0.03 0.05 0.05 0.02 0.04 0.05 0.05 0.06
4m 0.02 0.02 0.03 0.02 0.02 0.01 0.02 0.03 0.03 0.03
5m 0.01 0.01 0.02 0.00 0.01 0.01 0.01 0.03 0.04 0.04
6m 0.02 0.02 0.03 0.00 0.02 0.00 0.03 0.03 0.04 0.04
7m 0.03 0.04 0.04 0.02 0.03 0.02 0.05 0.05 0.06 0.06
8m 0.02 0.03 0.02 0.01 0.02 0.02 0.04 0.05 0.06 0.05
9m 0.01 0.02 0.01 0.01 0.02 0.01 0.03 0.03 0.04 0.03
10m 0.00 0.01 0.00 0.00 0.01 0.01 0.02 0.04 0.03 0.03
11m 0.01 0.02 0.00 0.00 0.01 0.01 0.02 0.03 0.04 0.03
12m 0.01 0.02 0.02 0.01 0.02 0.02 0.02 0.03 0.05 0.04
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.08 0.06 0.04 0.02 0.00 −0.02 −0.04 −0.06 −0.08


IEF: Positive Lift by MA and FR Window
MA Window
FR Window
3m 0.01 0.01 0.00 -0.01 0.02 0.03 0.02 0.01 -0.00 0.00
4m -0.00 -0.00 -0.02 -0.01 0.01 0.01 0.01 0.01 -0.00 -0.01
5m -0.01 -0.01 -0.03 -0.02 0.01 0.02 0.02 0.01 0.02 0.00
6m -0.01 0.00 -0.00 -0.01 0.01 0.02 0.02 0.02 0.03 0.01
7m -0.03 -0.02 -0.01 -0.03 -0.01 -0.01 -0.00 0.00 0.00 -0.01
8m -0.02 -0.01 -0.01 -0.03 -0.01 -0.01 -0.00 0.00 0.01 -0.00
9m -0.01 0.00 -0.00 -0.02 -0.01 -0.01 0.01 0.01 0.02 0.01
10m -0.02 -0.01 -0.01 -0.03 -0.01 -0.01 0.00 0.00 0.01 0.01
11m -0.04 -0.02 -0.02 -0.04 -0.03 -0.02 -0.01 -0.01 -0.01 -0.01
12m -0.03 -0.01 -0.01 -0.03 -0.01 -0.00 0.01 0.01 0.01 0.01
3m 4m 5m 6m 7m 8m 9m 10m 11m 12m
0.08 0.06 0.04 0.02 0.00 −0.02 −0.04 −0.06 −0.08



Here we have a more subtle set of results. For some ETFs (IVV, EFA, EEM, and IAU), there generally is a postive lift, albeit small in many cases.

Prediction Lift Example #1 Recall our definitions from above for accuracy:

```
overall_accuracy = (pair_data[pred_col] == pair_data[actual_col]).mean()
```

```
pos_accuracy = ((pair_data[pred_col] == 1) & (pair_data[actual_col] == 1)).sum()
/ (pair_data[pred_col] == 1).sum()
```

```
neg_accuracy = ((pair_data[pred_col] == -1) & (pair_data[actual_col] ==
-1)).sum() / (pair_data[pred_col] == -1).sum()
```

And for lift:

```
base_rate = (pair_data[fr_col] > 0).mean()
```

```
overall_lift = overall_accuracy - max(base_rate, 1 - base_rate)
```

```
positive_lift = pos_accuracy - base_rate
```

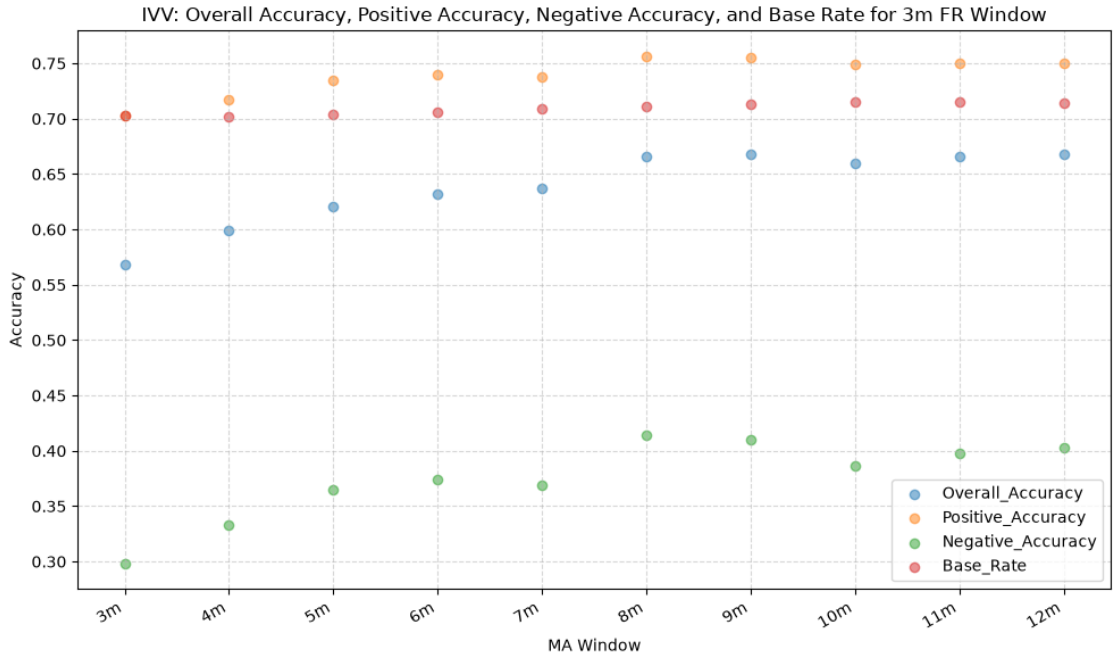
```
negative_lift = neg_accuracy - (1 - base_rate)
```

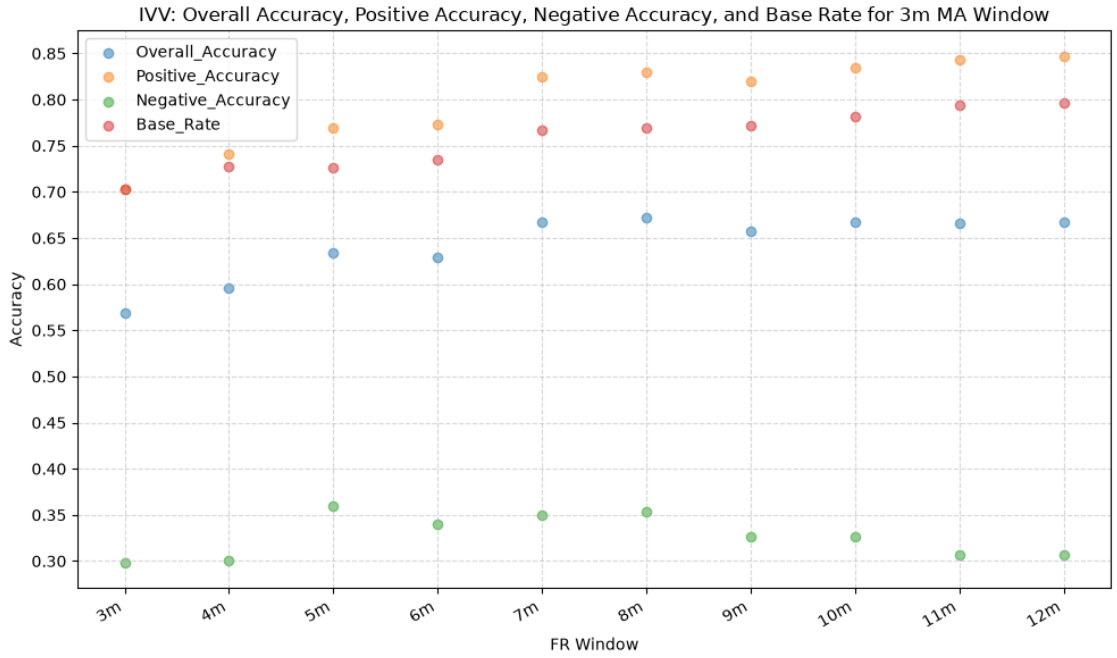
So here is an example comparing the overall accuracy, positive accuracy, negative accuracy, and base rate, followed by the corresponding values for the overall lift, positive lift, and negative lift.

[30]:
```
example_fund = "IVV"
example_fr = "3m"
example_ma = "3m"

plot_scatter(
    df=ma_prediction_results[(ma_prediction_results["Fund"] == example_fund) &␣
 ↪(ma_prediction_results["FR_Window"] == example_fr)],
    x_plot_column="MA_Window",
    y_plot_columns=["Overall_Accuracy", "Positive_Accuracy",␣
 ↪"Negative_Accuracy", "Base_Rate"],
    title=f"{example_fund}: Overall Accuracy, Positive Accuracy, Negative␣
 ↪Accuracy, and Base Rate for {example_fr} FR Window",
    x_label="MA Window",
    x_format="String",
    x_format_decimal_places=2,
    x_tick_spacing="Auto",
    x_tick_start=None,
    x_tick_rotation=30,
    y_label="Accuracy",
    y_format="Decimal",
    y_format_decimal_places=2,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
)

plot_scatter(
    df=ma_prediction_results[(ma_prediction_results["Fund"] == example_fund) &␣
 ↪(ma_prediction_results["MA_Window"] == example_ma)],
    x_plot_column="FR_Window",
    y_plot_columns=["Overall_Accuracy", "Positive_Accuracy",␣
 ↪"Negative_Accuracy", "Base_Rate"],
    title=f"{example_fund}: Overall Accuracy, Positive Accuracy, Negative␣
 ↪Accuracy, and Base Rate for {example_ma} MA Window",
    x_label="FR Window",
    x_format="String",
    x_format_decimal_places=2,
    x_tick_spacing="Auto",
    x_tick_start=None,
    x_tick_rotation=30,
    y_label="Accuracy",
    y_format="Decimal",
    y_format_decimal_places=2,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
)
```

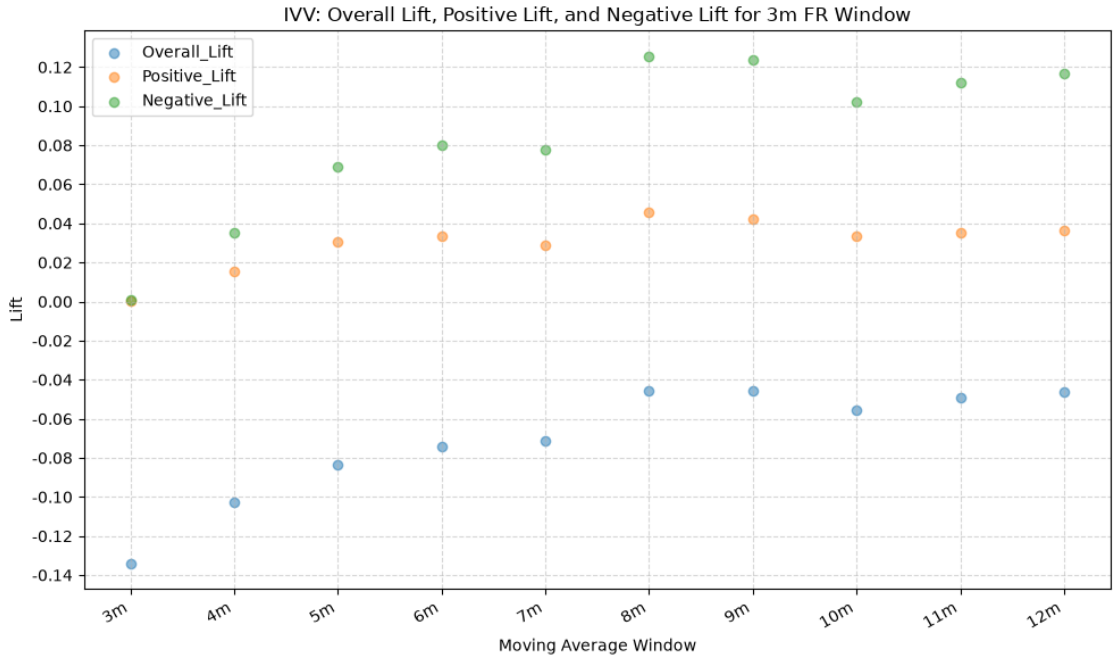


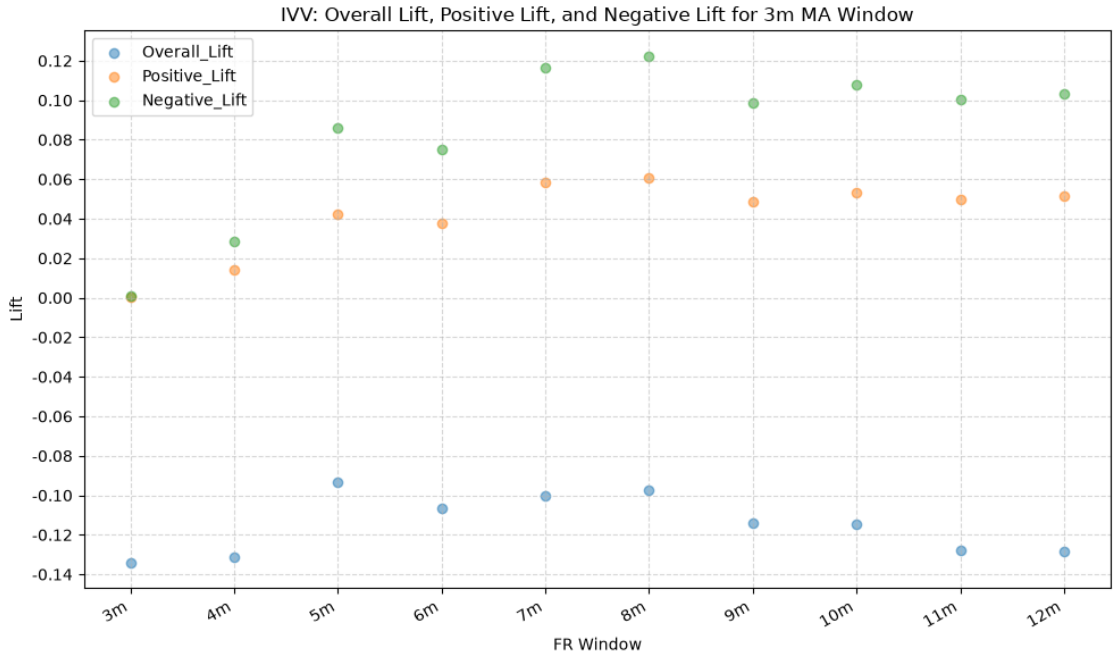


The above plot provides confirmation of our intuition; the negative accuracy actually reduces the overall accuracy, because the model more often predicts false negatives, meaning that the returns are positive when the model predicts negative. However, we also see how the positive accuracy exceeds the base rate, which is truly our measure of success here.

[31]:
```
plot_scatter(
    df=ma_prediction_results[(ma_prediction_results["Fund"] == example_fund) &␣
 ↪(ma_prediction_results["FR_Window"] == example_fr)],
    x_plot_column="MA_Window",
    y_plot_columns=["Overall_Lift", "Positive_Lift", "Negative_Lift"],
    title=f"{example_fund}: Overall Lift, Positive Lift, and Negative Lift for␣
 ↪{example_fr} FR Window",
    x_label="Moving Average Window",
    x_format="String",
    x_format_decimal_places=2,
    x_tick_spacing="Auto",
    x_tick_start=None,
    x_tick_rotation=30,
    y_label="Lift",
    y_format="Decimal",
    y_format_decimal_places=2,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
)

plot_scatter(
    df=ma_prediction_results[(ma_prediction_results["Fund"] == example_fund) &␣
 ↪(ma_prediction_results["MA_Window"] == example_ma)],
    x_plot_column="FR_Window",
    y_plot_columns=["Overall_Lift", "Positive_Lift", "Negative_Lift"],
    title=f"{example_fund}: Overall Lift, Positive Lift, and Negative Lift for␣
 ↪{example_ma} MA Window",
    x_label="FR Window",
    x_format="String",
    x_format_decimal_places=2,
    x_tick_spacing="Auto",
    x_tick_start=None,
    x_tick_rotation=30,
    y_label="Lift",
    y_format="Decimal",
    y_format_decimal_places=2,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
)
```





The above plots might be the cleanest explanation of the predictive effect. The model actually provides positive lift for both the positive and negative predictions, across different moving average and FR windows, indicating that it is able to improve upon the base rate in both cases. Still, the overall lift is constrained by the negative accuracy, which reduces the overall accuracy and thus the

overall lift. This is slightly counterintuitive, because one might expect that improving the positive accuracy would automatically lead to a higher overall accuracy.

0.10 Analysis Of Results

0.10.1 1. Positive, Negative, and Overall Accuracy

From the above plots, we can see that some ETFs (which we are using as a proxy for asset classes), appear to have a stronger relationship between the price-MA difference and the forward returns than others. This "relationship" takes the form of the overall accuracy of the predictions, which tells us how well the price-MA difference predicts the direction of the future returns.

1.1. Mean Overall Accuracy by Fund and MA Window Here we plot the overall mean prediction accuracy for each ETF and MA window combination.

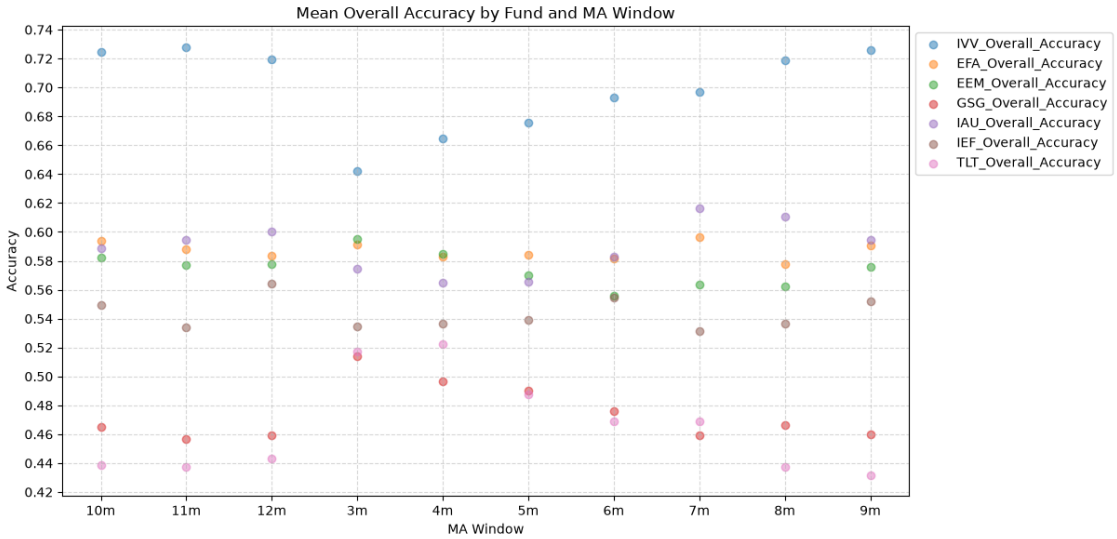
[32]:
```
# Calc overall mean accuracy for each fund across all MA windows
temp = ma_prediction_results.groupby(["Fund", "MA_Window"])["Overall_Accuracy"].
 ↪mean().reset_index()

# Create a new DataFrame with unique MA_Window values and merge the mean␣
 ↪accuracy for each fund
accuracy = pd.DataFrame({"MA_Window": temp["MA_Window"].unique()})

for fund in fund_list:
    fund_temp = temp[temp["Fund"] == fund][["MA_Window", "Overall_Accuracy"]].
 ↪rename(
        columns={"Overall_Accuracy": f"{fund}_Overall_Accuracy"}
    )
    accuracy = pd.merge(accuracy, fund_temp, on="MA_Window", how="outer")

plot_scatter(
    df=accuracy,
    x_plot_column="MA_Window",
    y_plot_columns=["IVV_Overall_Accuracy",
                    "EFA_Overall_Accuracy",
                    "EEM_Overall_Accuracy",
                    "GSG_Overall_Accuracy",
                    "IAU_Overall_Accuracy",
                    "IEF_Overall_Accuracy",
                    "TLT_Overall_Accuracy"],
    title="Mean Overall Accuracy by Fund and MA Window",
    x_label="MA Window",
    x_format="String",
    x_format_decimal_places=0,
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=0,
    y_label="Accuracy",
```

```
    y_format="Decimal",
    y_format_decimal_places=2,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    plot_OLS_regression_line=False,
    OLS_column=None,
    plot_Ridge_regression_line=False,
    Ridge_column=None,
    plot_RidgeCV_regression_line=False,
    RidgeCV_column=None,
    regression_constant=True,
    grid=True,
    legend=True,
    legend_location="upper left",
    legend_anchor=(1, 1),
    export_plot=False,
    plot_file_name=None,
)
```



The stock funds (IVV, EFA, and EEM) post the highest mean overall accuracy, and the commodity fund (GSG) and long-term bonds (TLT) the lowest. It is tempting to read 0.50 as the neutral line, but as we've discussed above that is the wrong measure here. Because most of these assets drift upward over the sample, forward returns are positive more than half the time, so even a model without any predictive power performs well by being long. The true benchmark is each asset's *base rate*, which we look at next.

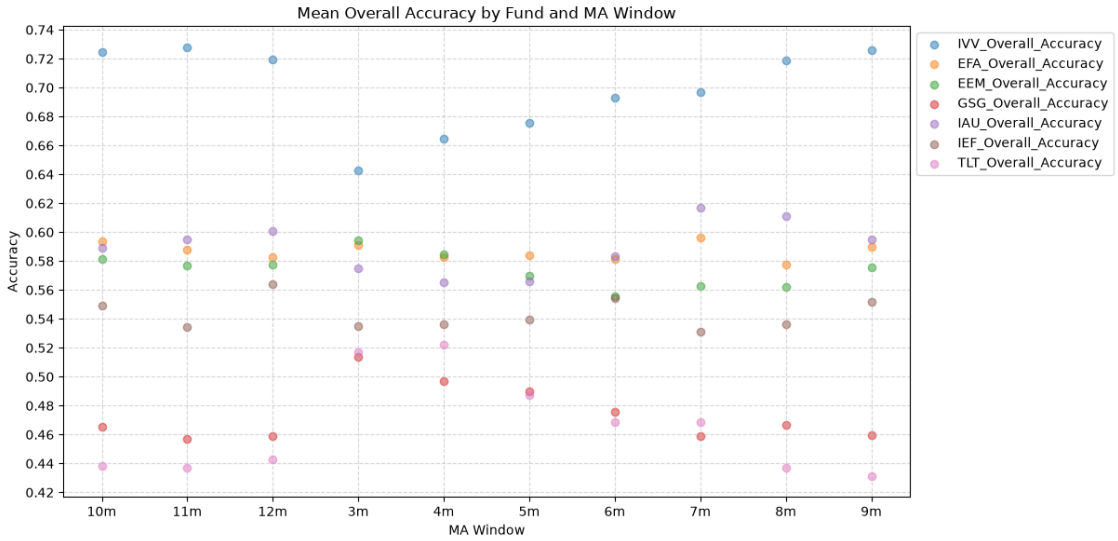
1.2. The Base Rate by Fund and MA Window The base rate is the unconditional proportion of positive forward returns — what you would score by always predicting "up," and ignoring the moving average entirely. It varies a little by MA window only because each window is measured

over a slightly different sample.

[33]:
```
temp = ma_prediction_results.groupby(["Fund", "MA_Window"])["Base_Rate"].mean().
 ↪reset_index()

for fund in fund_list:
    fund_temp = temp[temp["Fund"] == fund][["MA_Window", "Base_Rate"]].rename(
        columns={"Base_Rate": f"{fund}_Base_Rate"}
    )
    accuracy = pd.merge(accuracy, fund_temp, on="MA_Window", how="outer")

plot_scatter(
    df=accuracy,
    x_plot_column="MA_Window",
    y_plot_columns=["IVV_Base_Rate",
                    "EFA_Base_Rate",
                    "EEM_Base_Rate",
                    "GSG_Base_Rate",
                    "IAU_Base_Rate",
                    "IEF_Base_Rate",
                    "TLT_Base_Rate"],
    title="Base Rate (Share of Positive Forward Returns) by Fund and MA Window",
    x_label="MA Window",
    x_format="String",
    x_format_decimal_places=0,
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=0,
    y_label="Base Rate",
    y_format="Decimal",
    y_format_decimal_places=2,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    plot_OLS_regression_line=False,
    OLS_column=None,
    plot_Ridge_regression_line=False,
    Ridge_column=None,
    plot_RidgeCV_regression_line=False,
    RidgeCV_column=None,
    regression_constant=True,
    grid=True,
    legend=True,
    legend_location="upper left",
    legend_anchor=(1, 1),
    export_plot=False,
    plot_file_name=None,
)
```



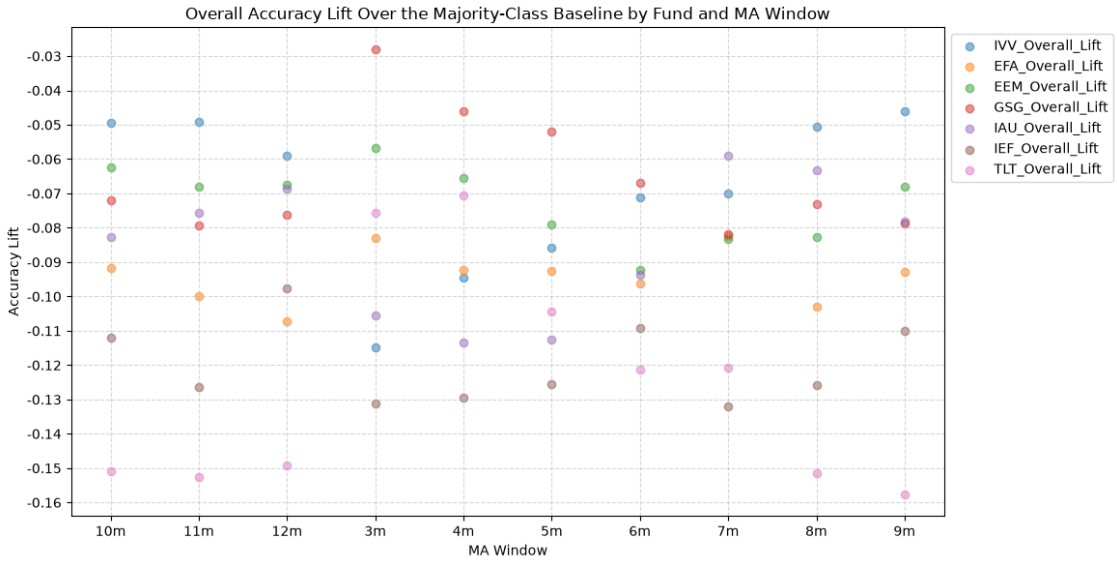
1.3. Overall Accuracy Lift by Fund and MA Window Subtracting the naive benchmark, the majority-class rate `max(base rate, 1 - base rate)`, from overall accuracy gives the *lift*, which is how much the price-MA sign actually adds over always betting the more common direction.

[34]:
```
temp = ma_prediction_results.groupby(["Fund", "MA_Window"])["Overall_Lift"].
 ↪mean().reset_index()

for fund in fund_list:
    fund_temp = temp[temp["Fund"] == fund][["MA_Window", "Overall_Lift"]].
 ↪rename(
        columns={"Overall_Lift": f"{fund}_Overall_Lift"}
    )
    accuracy = pd.merge(accuracy, fund_temp, on="MA_Window", how="outer")

plot_scatter(
    df=accuracy,
    x_plot_column="MA_Window",
    y_plot_columns=["IVV_Overall_Lift",
                    "EFA_Overall_Lift",
                    "EEM_Overall_Lift",
                    "GSG_Overall_Lift",
                    "IAU_Overall_Lift",
                    "IEF_Overall_Lift",
                    "TLT_Overall_Lift"],
    title="Overall Accuracy Lift Over the Majority-Class Baseline by Fund and␣
 ↪MA Window",
    x_label="MA Window",
    x_format="String",
```

```
    x_format_decimal_places=0,
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=0,
    y_label="Accuracy Lift",
    y_format="Decimal",
    y_format_decimal_places=2,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    plot_OLS_regression_line=False,
    OLS_column=None,
    plot_Ridge_regression_line=False,
    Ridge_column=None,
    plot_RidgeCV_regression_line=False,
    RidgeCV_column=None,
    regression_constant=True,
    grid=True,
    legend=True,
    legend_location="upper left",
    legend_anchor=(1, 1),
    export_plot=False,
    plot_file_name=None,
)
```



This plot tells us that, overall, the MA predictions actually hurt the overall accuracy compared to the naive benchmark. This is the same idea that we discussed in the prediction lift example.

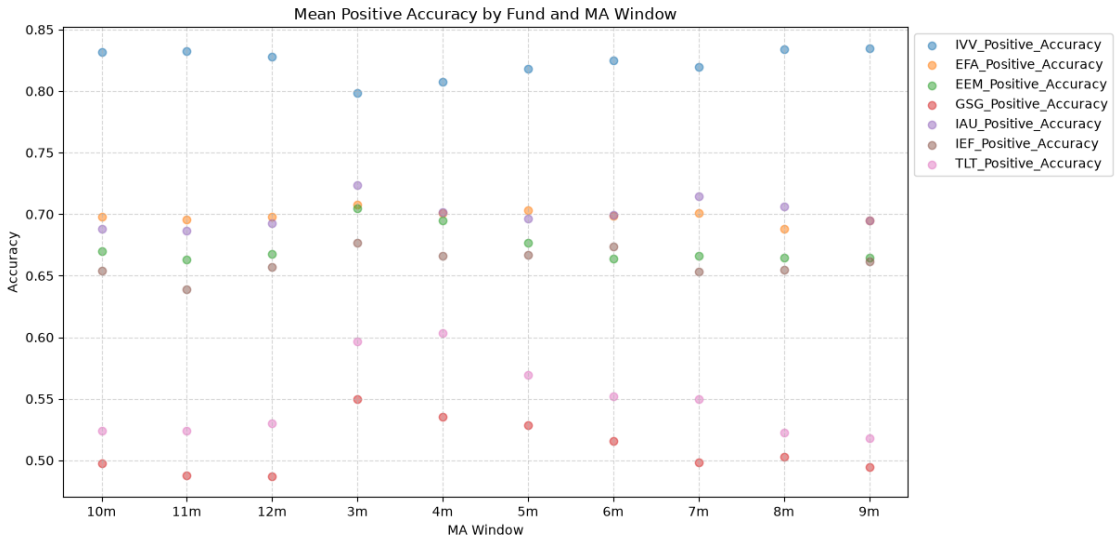
1.4. Mean Positive Accuracy by Fund and MA Window The overall accuracy statistic includes both the positive (are future returns positive when the price is above its MA?) and the negative (are future returns negative when the price is below its MA?) directions. As we mentioned above, from a long-only perspective we care only about the positive direction, which we plot next. Raw positive accuracy looks high, but keep the base rate from section 1.2 in mind, because predicting the common outcome (up) is inherently easier than predicting the rare one (down).

[35]:
```
# Calc mean positive accuracy for each fund across all MA windows
temp = ma_prediction_results.groupby(["Fund",␣
 ↪"MA_Window"])["Positive_Accuracy"].mean().reset_index()

for fund in fund_list:
    fund_temp = temp[temp["Fund"] == fund][["MA_Window", "Positive_Accuracy"]].
 ↪rename(
        columns={"Positive_Accuracy": f"{fund}_Positive_Accuracy"}
    )
    accuracy = pd.merge(accuracy, fund_temp, on="MA_Window", how="outer")

plot_scatter(
    df=accuracy,
    x_plot_column="MA_Window",
    y_plot_columns=["IVV_Positive_Accuracy",
                    "EFA_Positive_Accuracy",
                    "EEM_Positive_Accuracy",
                    "GSG_Positive_Accuracy",
                    "IAU_Positive_Accuracy",
                    "IEF_Positive_Accuracy",
                    "TLT_Positive_Accuracy"],
    title="Mean Positive Accuracy by Fund and MA Window",
    x_label="MA Window",
    x_format="String",
    x_format_decimal_places=0,
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=0,
    y_label="Accuracy",
    y_format="Decimal",
    y_format_decimal_places=2,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    plot_OLS_regression_line=False,
    OLS_column=None,
    plot_Ridge_regression_line=False,
    Ridge_column=None,
    plot_RidgeCV_regression_line=False,
    RidgeCV_column=None,
    regression_constant=True,
```

```
    grid=True,
    legend=True,
    legend_location="upper left",
    legend_anchor=(1, 1),
    export_plot=False,
    plot_file_name=None,
)
```



The mean positive accuracy sits above the overall accuracy for every fund. It is tempting to read that difference as the moving average being especially good at calling up-moves, but it is mostly the base-rate effect. Positive predictions land on the common outcome, so they are right more often almost by construction. To see whether the moving average actually helps, we have to subtract the base rate.

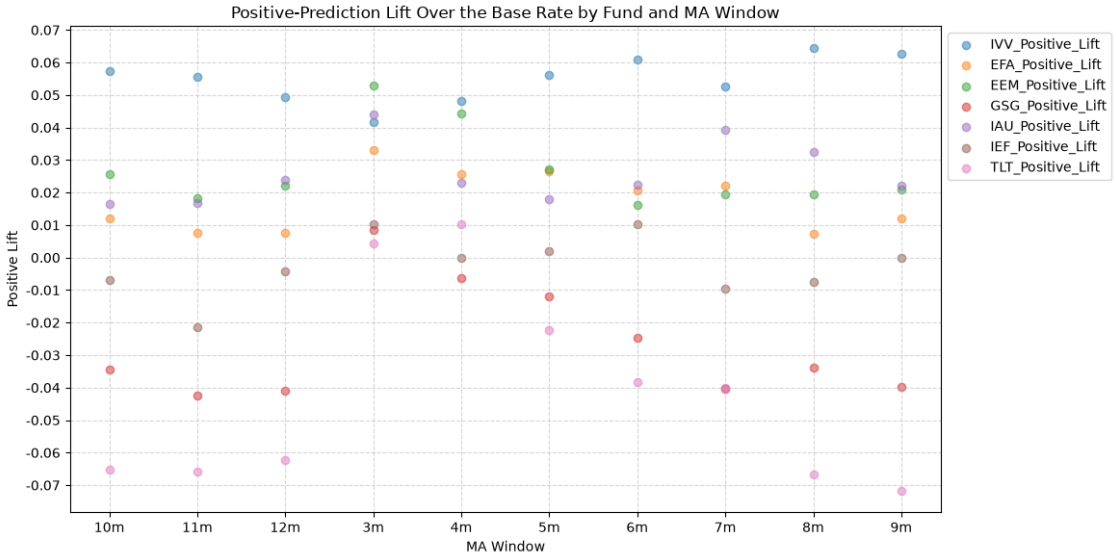
1.5. Positive-Prediction Lift by Fund and MA Window Positive lift is positive accuracy minus the base rate. Does being above the moving average beat simply assuming an up move? Zero means the moving average adds nothing over the underlying probabilities, and positive means it genuinely improves the prediction. This is the counterpart to the raw positive-accuracy plot above, much of what looked like predictive skill was just the upward drift, and what is found here is the part actually attributable to the moving average. We are distilling the heatmap down to each MA window, essentially moving from 2 dimensions to 1 dimension.

[36]:
```
temp = ma_prediction_results.groupby(["Fund", "MA_Window"])["Positive_Lift"].
 ↪mean().reset_index()

for fund in fund_list:
    fund_temp = temp[temp["Fund"] == fund][["MA_Window", "Positive_Lift"]].
 ↪rename(
        columns={"Positive_Lift": f"{fund}_Positive_Lift"}
```

```
    )
    accuracy = pd.merge(accuracy, fund_temp, on="MA_Window", how="outer")

plot_scatter(
    df=accuracy,
    x_plot_column="MA_Window",
    y_plot_columns=["IVV_Positive_Lift",
                    "EFA_Positive_Lift",
                    "EEM_Positive_Lift",
                    "GSG_Positive_Lift",
                    "IAU_Positive_Lift",
                    "IEF_Positive_Lift",
                    "TLT_Positive_Lift"],
    title="Positive-Prediction Lift Over the Base Rate by Fund and MA Window",
    x_label="MA Window",
    x_format="String",
    x_format_decimal_places=0,
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=0,
    y_label="Positive Lift",
    y_format="Decimal",
    y_format_decimal_places=2,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    plot_OLS_regression_line=False,
    OLS_column=None,
    plot_Ridge_regression_line=False,
    Ridge_column=None,
    plot_RidgeCV_regression_line=False,
    RidgeCV_column=None,
    regression_constant=True,
    grid=True,
    legend=True,
    legend_location="upper left",
    legend_anchor=(1, 1),
    export_plot=False,
    plot_file_name=None,
)
```



Here we have some interesting results that could be useful. For the stock funds (IVV, EFA, EEM) and gold (GLD), the MA predictions tend to provide a slight improvenment over the naive benchmark, somewhere between 1-6%.

For the commodity fund (GSG) and long-term bonds (TLT), however, the MA predictions generally underperform the naive benchmark, indicating that the moving average signal may not be as effective for these asset classes when predicting the direction of future returns.

0.10.2 2. Distribution of Future Returns Based on Price-MA Difference

Next, we can consider the shape of the distribution of the future returns, split on whether the the price-MA difference is positive or negative.

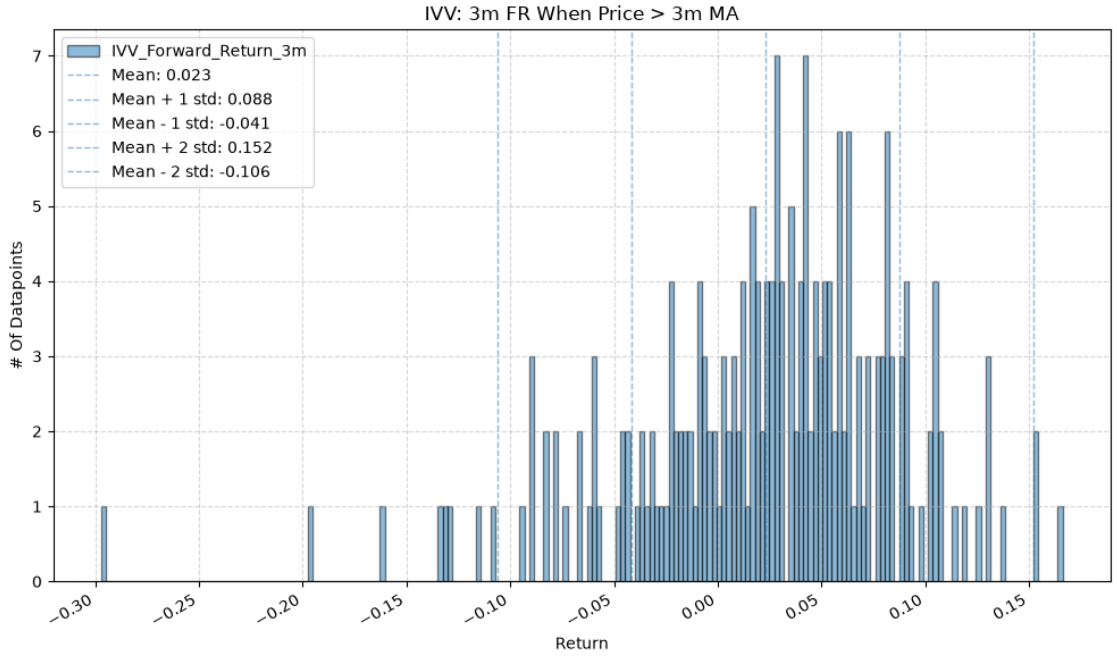
2.1. Mean Future Return When Price-MA Difference is Positive by Fund and MA Window For a per-window and per-moving average view of a single fund, the plot below shows IVV's forward-return mean, with +/- 2 standard deviation bands, when the price-MA relationship is positive, for the the 3-month moving average and 3-month forward return window.

[37]:
```
example_fund = "IVV"
example_ma = "3m"
example_fr = "3m"

example_data = fund_data[example_fund].dropna(subset=[
    f"{example_fund}_Price_MA_Diff_Percent_{example_ma}",
    f"{example_fund}_Forward_Return_{example_fr}",
])

plot_histogram(
```

```
    df=example_data[example_data[f"{example_fund}_MA_Prediction_{example_ma}"]␣
 ↪== 1],
    plot_columns=[f"{example_fund}_Forward_Return_{example_fr}"],
    title=f"{example_fund}: {example_fr} FR When Price > {example_ma} MA",
    x_label="Return",
    x_tick_spacing="Auto",
    x_tick_rotation=30,
    y_label="# Of Datapoints",
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    grid=True,
    legend=True,
    export_plot=False,
    plot_file_name=None,
)
```



Expanding this to the broader context of all funds and moving averages, we can plot the aggregate distribution of mean future returns when the price-MA difference is positive.

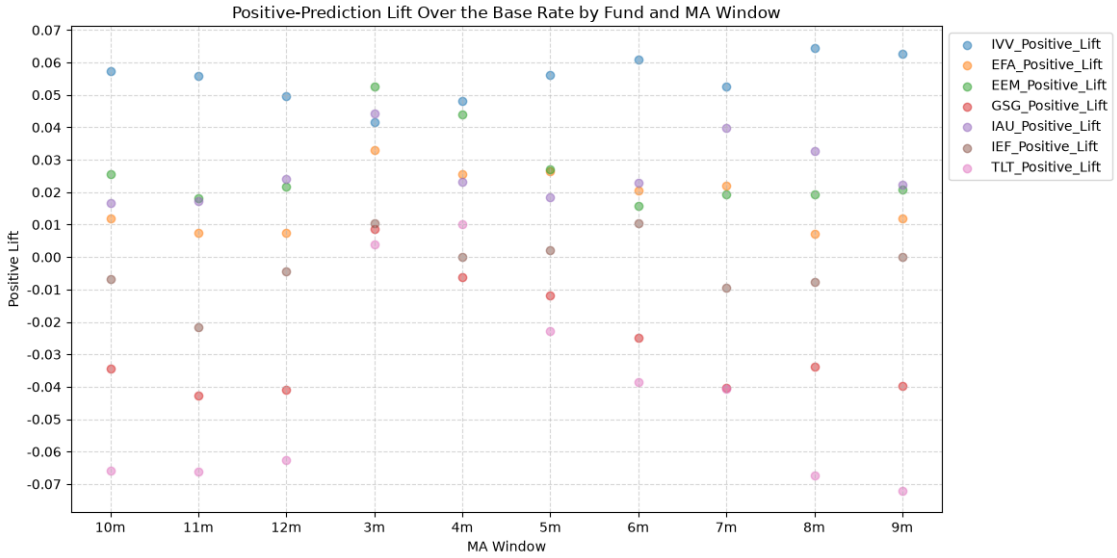
[38]:
```
temp = ma_prediction_results.groupby(["Fund",␣
 ↪"MA_Window"])["Positive_Mean_Return"].mean().reset_index()

distribution = pd.DataFrame({"MA_Window": temp["MA_Window"].unique()})

for fund in fund_list:
```

```
    fund_temp = temp[temp["Fund"] == fund][["MA_Window",␣
 ↪"Positive_Mean_Return"]].rename(
        columns={"Positive_Mean_Return": f"{fund}_Positive_Mean_Return"}
    )
    distribution = pd.merge(distribution, fund_temp, on="MA_Window",␣
 ↪how="outer")

plot_scatter(
    df=distribution,
    x_plot_column="MA_Window",
    y_plot_columns=["IVV_Positive_Mean_Return",
                    "EFA_Positive_Mean_Return",
                    "EEM_Positive_Mean_Return",
                    "GSG_Positive_Mean_Return",
                    "IAU_Positive_Mean_Return",
                    "IEF_Positive_Mean_Return",
                    "TLT_Positive_Mean_Return"],
    title="Mean Future Return When Price-MA Difference is Positive by Fund and␣
 ↪MA Window",
    x_label="MA Window",
    x_format="String",
    x_format_decimal_places=0,
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=0,
    y_label="Return",
    y_format="Decimal",
    y_format_decimal_places=3,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    plot_OLS_regression_line=False,
    OLS_column=None,
    plot_Ridge_regression_line=False,
    Ridge_column=None,
    plot_RidgeCV_regression_line=False,
    RidgeCV_column=None,
    regression_constant=True,
    grid=True,
    legend=True,
    legend_location="upper left",
    legend_anchor=(1, 1),
    export_plot=False,
    plot_file_name=None,
)
```



In a perfect world, we want to see similar numbers across all MA windows, and all of those numbers would be relatively high. If every window gives us roughly the same distribution of future returns, that would indicate a consistent predictive relationship across all windows. But, what we often see is that the distributions vary quite a bit depending on the window, which suggests that it's useful to consider a combination of windows to form a more robust signal.

Note that gold (IAU) has a smaller variation and much higher values (mean return is ~1.5% greater than IVV) across MA windows compared to the other funds.

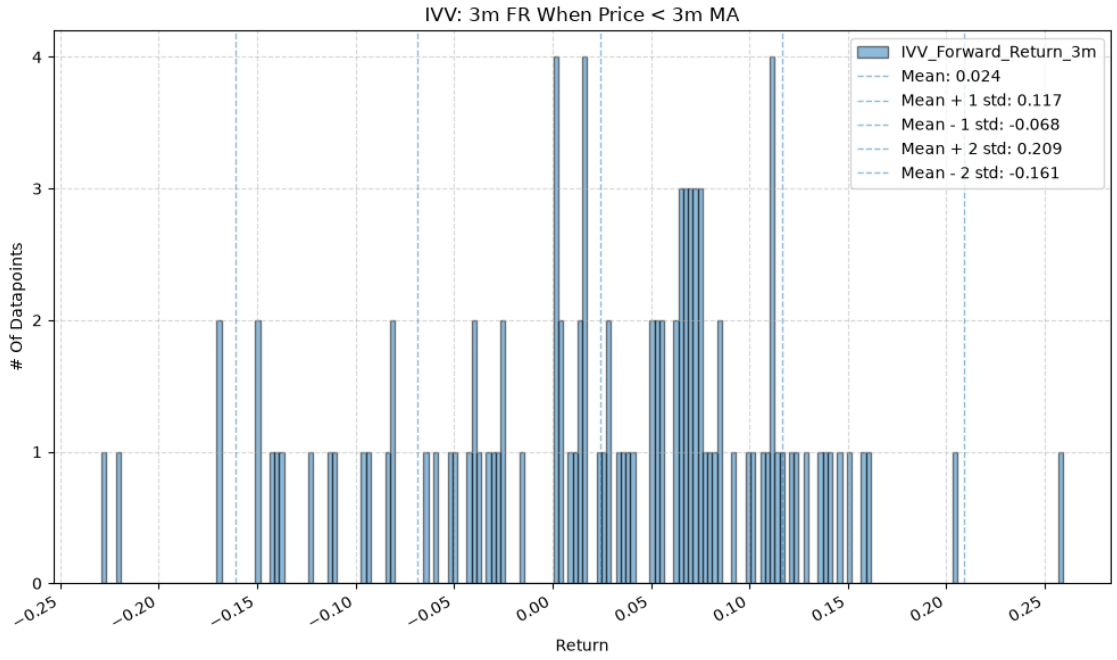
2.2. Mean Future Return When Price-MA Difference is Negative by Fund and MA Window Similar as to what we did above, the plot below shows IVV's forward-return mean, with +/- 2 standard deviation bands, when the price-MA relationship is negative, for the the 3-month moving average and 3-month forward return window.

[39]:
```
example_fund = "IVV"
example_ma = "3m"
example_fr = "3m"

example_data = fund_data[example_fund].dropna(subset=[
    f"{example_fund}_Price_MA_Diff_Percent_{example_ma}",
    f"{example_fund}_Forward_Return_{example_fr}",
])

plot_histogram(
    df=example_data[example_data[f"{example_fund}_MA_Prediction_{example_ma}"]␣
 ↪== -1],
    plot_columns=[f"{example_fund}_Forward_Return_{example_fr}"],
    title=f"{example_fund}: {example_fr} FR When Price < {example_ma} MA",
    x_label="Return",
```

```
    x_tick_spacing="Auto",
    x_tick_rotation=30,
    y_label="# Of Datapoints",
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    grid=True,
    legend=True,
    export_plot=False,
    plot_file_name=None,
)
```

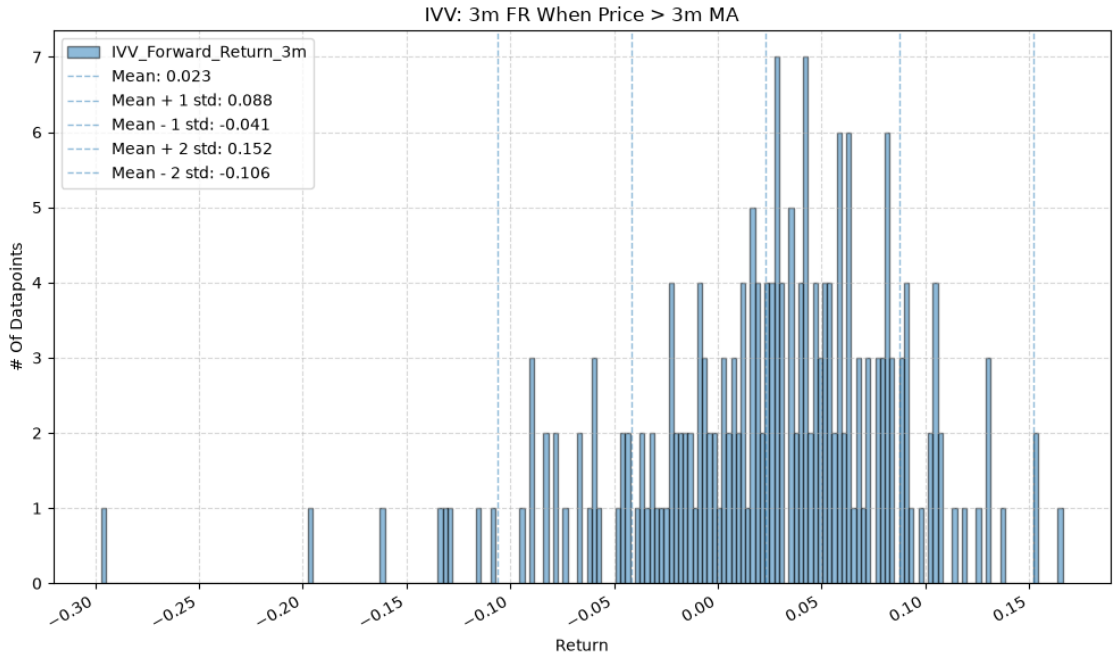


Again, expanding this to the broader context of all funds and moving averages, we can plot the aggregate distribution of future returns when the price-MA difference is negative.

[40]:
```
temp = ma_prediction_results.groupby(["Fund",␣
↪"MA_Window"])["Negative_Mean_Return"].mean().reset_index()

for fund in fund_list:
    fund_temp = temp[temp["Fund"] == fund][["MA_Window",␣
↪"Negative_Mean_Return"]].rename(
        columns={"Negative_Mean_Return": f"{fund}_Negative_Mean_Return"}
    )
    distribution = pd.merge(distribution, fund_temp, on="MA_Window",␣
↪how="outer")
```

```
plot_scatter(
    df=distribution,
    x_plot_column="MA_Window",
    y_plot_columns=["IVV_Negative_Mean_Return",
                    "EFA_Negative_Mean_Return",
                    "EEM_Negative_Mean_Return",
                    "GSG_Negative_Mean_Return",
                    "IAU_Negative_Mean_Return",
                    "IEF_Negative_Mean_Return",
                    "TLT_Negative_Mean_Return"],
    title="Mean Future Return When Price-MA Difference is Negative by Fund and␣
 ↪MA Window",
    x_label="MA Window",
    x_format="String",
    x_format_decimal_places=0,
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=0,
    y_label="Return",
    y_format="Decimal",
    y_format_decimal_places=3,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    plot_OLS_regression_line=False,
    OLS_column=None,
    plot_Ridge_regression_line=False,
    Ridge_column=None,
    plot_RidgeCV_regression_line=False,
    RidgeCV_column=None,
    regression_constant=True,
    grid=True,
    legend=True,
    legend_location="upper left",
    legend_anchor=(1, 1),
    export_plot=False,
    plot_file_name=None,
)
```



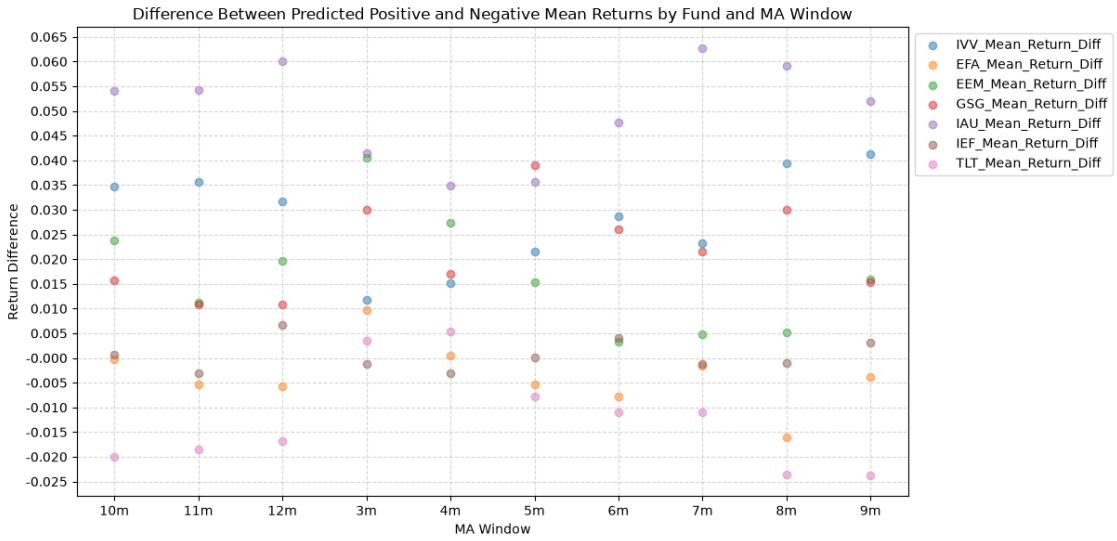
Interesting, the mean returns are mostly still positive. That suggests that even when the price-MA difference is negative, future returns tend to be positive on average, though perhaps smaller than when the difference is positive. This checks with our theory (and evidence) that returns generally drift upward over time.

2.3. Difference Between Predicted Positive and Negative Mean Returns by Fund and MA Window Next, we can look at the *difference* in mean future returns between when the price-MA difference is positive and when it is negative.

[41]:
```
for fund in fund_list:
    distribution[f"{fund}_Mean_Return_Diff"] =␣
 ↪distribution[f"{fund}_Positive_Mean_Return"] -␣
 ↪distribution[f"{fund}_Negative_Mean_Return"]

plot_scatter(
    df=distribution,
    x_plot_column="MA_Window",
    y_plot_columns=["IVV_Mean_Return_Diff",
                    "EFA_Mean_Return_Diff",
                    "EEM_Mean_Return_Diff",
                    "GSG_Mean_Return_Diff",
                    "IAU_Mean_Return_Diff",
                    "IEF_Mean_Return_Diff",
                    "TLT_Mean_Return_Diff"],
    title="Difference Between Predicted Positive and Negative Mean Returns by␣
 ↪Fund and MA Window",
```

```
    x_label="MA Window",
    x_format="String",
    x_format_decimal_places=0,
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=0,
    y_label="Return Difference",
    y_format="Decimal",
    y_format_decimal_places=3,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    plot_OLS_regression_line=False,
    OLS_column=None,
    plot_Ridge_regression_line=False,
    Ridge_column=None,
    plot_RidgeCV_regression_line=False,
    RidgeCV_column=None,
    regression_constant=True,
    grid=True,
    legend=True,
    legend_location="upper left",
    legend_anchor=(1, 1),
    export_plot=False,
    plot_file_name=None,
)
```



In general, the mean of the future positive predicted returns is higher than the mean of the future negative predicted returns, which is what we would expect. However, the distributions are not normal, and there are some outliers that skew the distributions. This seems to be the case with

TLT.

Note again, gold has the highest difference in mean return between when the price-MA difference is positive and when it is negative.

2.4. Difference Between Positive Mean +2 Std and Positive Mean -2 Std Returns by Fund and MA Window As mentioned above, we think that there are outliers that skew the distributions of the forward returns. But by how much? And how can we quantify their impact? One way is to look at the standard deviations. Here's the plots for the difference between 2 standard deviations above and below the mean for the forward returns.

[42]:
```
temp = ma_prediction_results.groupby(["Fund",␣
↪"MA_Window"])["Positive_Mean_Plus_Two_Std"].mean().reset_index()

for fund in fund_list:
    fund_temp = temp[temp["Fund"] == fund][["MA_Window",␣
↪"Positive_Mean_Plus_Two_Std"]].rename(
        columns={"Positive_Mean_Plus_Two_Std":␣
↪f"{fund}_Positive_Mean_Plus_Two_Std"}
    )
    distribution = pd.merge(distribution, fund_temp, on="MA_Window",␣
↪how="outer")

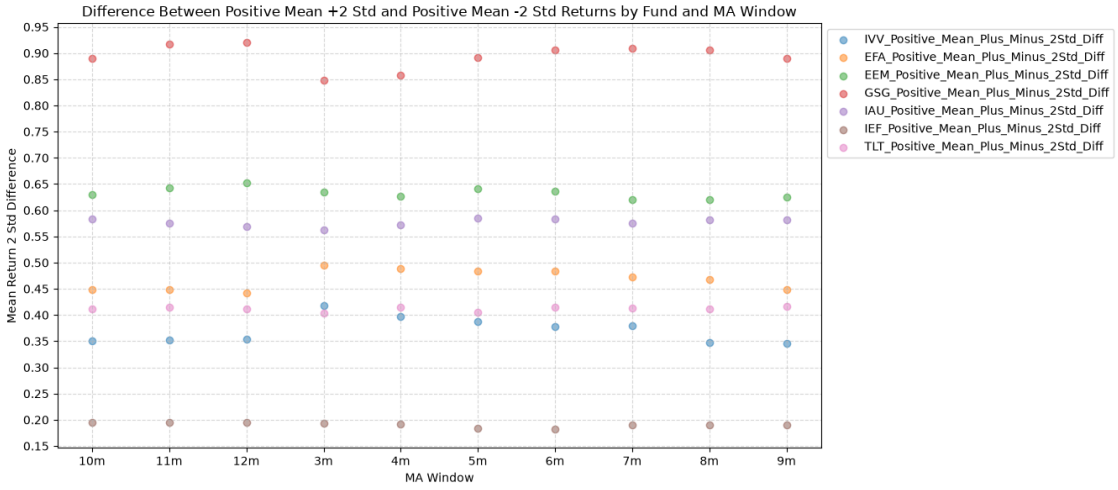
temp = ma_prediction_results.groupby(["Fund",␣
↪"MA_Window"])["Positive_Mean_Minus_Two_Std"].mean().reset_index()

for fund in fund_list:
    fund_temp = temp[temp["Fund"] == fund][["MA_Window",␣
↪"Positive_Mean_Minus_Two_Std"]].rename(
        columns={"Positive_Mean_Minus_Two_Std":␣
↪f"{fund}_Positive_Mean_Minus_Two_Std"}
    )
    distribution = pd.merge(distribution, fund_temp, on="MA_Window",␣
↪how="outer")

for fund in fund_list:
    distribution[f"{fund}_Positive_Mean_Plus_Minus_2Std_Diff"] =␣
↪distribution[f"{fund}_Positive_Mean_Plus_Two_Std"] -␣
↪distribution[f"{fund}_Positive_Mean_Minus_Two_Std"]

plot_scatter(
    df=distribution,
    x_plot_column="MA_Window",
    y_plot_columns=["IVV_Positive_Mean_Plus_Minus_2Std_Diff",
                    "EFA_Positive_Mean_Plus_Minus_2Std_Diff",
                    "EEM_Positive_Mean_Plus_Minus_2Std_Diff",
                    "GSG_Positive_Mean_Plus_Minus_2Std_Diff",
```

```
                    "IAU_Positive_Mean_Plus_Minus_2Std_Diff",
                    "IEF_Positive_Mean_Plus_Minus_2Std_Diff",
                    "TLT_Positive_Mean_Plus_Minus_2Std_Diff"],
    title="Difference Between Positive Mean +2 Std and Positive Mean -2 Std␣
↪Returns by Fund and MA Window",
    x_label="MA Window",
    x_format="String",
    x_format_decimal_places=0,
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=0,
    y_label="Mean Return 2 Std Difference",
    y_format="Decimal",
    y_format_decimal_places=2,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    plot_OLS_regression_line=False,
    OLS_column=None,
    plot_Ridge_regression_line=False,
    Ridge_column=None,
    plot_RidgeCV_regression_line=False,
    RidgeCV_column=None,
    regression_constant=True,
    grid=True,
    legend=True,
    legend_location="upper left",
    legend_anchor=(1, 1),
    export_plot=False,
    plot_file_name=None,
)
```



GSG is a mess… the commodity fund has a lot of noise and outliers. We'd like to see a tight distribution, which might give us confidence that the forward returns are within a reasonable range and not being dominated by extreme values.

Here, the 7-10 year bond ETF is the leader, with a much tighter distribution and fewer extreme values skewing the results.

2.5. Difference Between Negative Mean +2 Std and Negative Mean -2 Std Returns by Fund and MA Window Now for the difference between 2 standard deviations above and below the mean for the forward returns when price-MA is negative.

[43]:
```
temp = ma_prediction_results.groupby(["Fund",␣
↪"MA_Window"])["Negative_Mean_Plus_Two_Std"].mean().reset_index()

for fund in fund_list:
    fund_temp = temp[temp["Fund"] == fund][["MA_Window",␣
↪"Negative_Mean_Plus_Two_Std"]].rename(
        columns={"Negative_Mean_Plus_Two_Std":␣
↪f"{fund}_Negative_Mean_Plus_Two_Std"}
    )
    distribution = pd.merge(distribution, fund_temp, on="MA_Window",␣
↪how="outer")

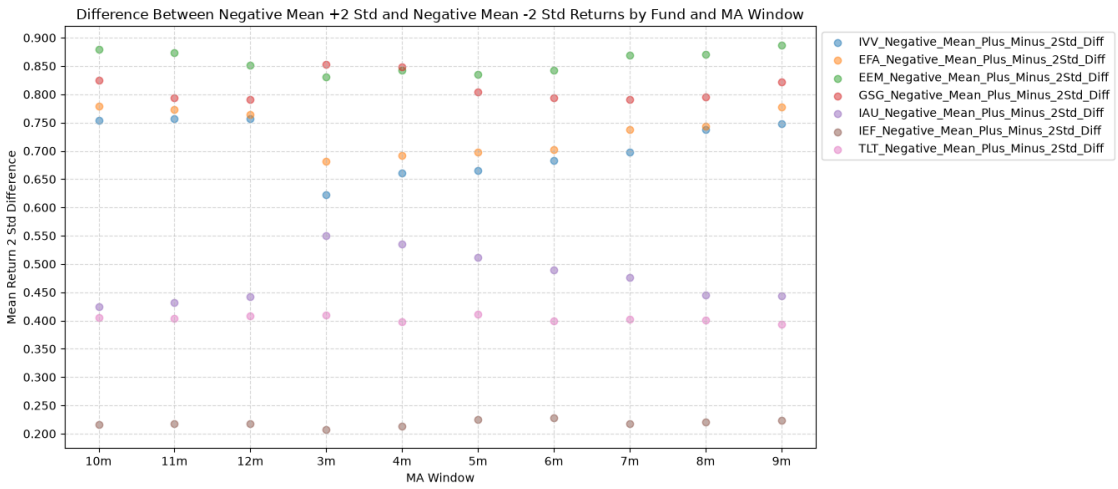
temp = ma_prediction_results.groupby(["Fund",␣
↪"MA_Window"])["Negative_Mean_Minus_Two_Std"].mean().reset_index()

for fund in fund_list:
    fund_temp = temp[temp["Fund"] == fund][["MA_Window",␣
↪"Negative_Mean_Minus_Two_Std"]].rename(
        columns={"Negative_Mean_Minus_Two_Std":␣
↪f"{fund}_Negative_Mean_Minus_Two_Std"}
    )
    distribution = pd.merge(distribution, fund_temp, on="MA_Window",␣
↪how="outer")

for fund in fund_list:
    distribution[f"{fund}_Negative_Mean_Plus_Minus_2Std_Diff"] =␣
↪distribution[f"{fund}_Negative_Mean_Plus_Two_Std"] -␣
↪distribution[f"{fund}_Negative_Mean_Minus_Two_Std"]

plot_scatter(
    df=distribution,
    x_plot_column="MA_Window",
    y_plot_columns=["IVV_Negative_Mean_Plus_Minus_2Std_Diff",
                    "EFA_Negative_Mean_Plus_Minus_2Std_Diff",
                    "EEM_Negative_Mean_Plus_Minus_2Std_Diff",
                    "GSG_Negative_Mean_Plus_Minus_2Std_Diff",
```

```
                    "IAU_Negative_Mean_Plus_Minus_2Std_Diff",
                    "IEF_Negative_Mean_Plus_Minus_2Std_Diff",
                    "TLT_Negative_Mean_Plus_Minus_2Std_Diff"],
    title="Difference Between Negative Mean +2 Std and Negative Mean -2 Std␣
↪Returns by Fund and MA Window",
    x_label="MA Window",
    x_format="String",
    x_format_decimal_places=0,
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=0,
    y_label="Mean Return 2 Std Difference",
    y_format="Decimal",
    y_format_decimal_places=3,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    plot_OLS_regression_line=False,
    OLS_column=None,
    plot_Ridge_regression_line=False,
    Ridge_column=None,
    plot_RidgeCV_regression_line=False,
    RidgeCV_column=None,
    regression_constant=True,
    grid=True,
    legend=True,
    legend_location="upper left",
    legend_anchor=(1, 1),
    export_plot=False,
    plot_file_name=None,
)
```



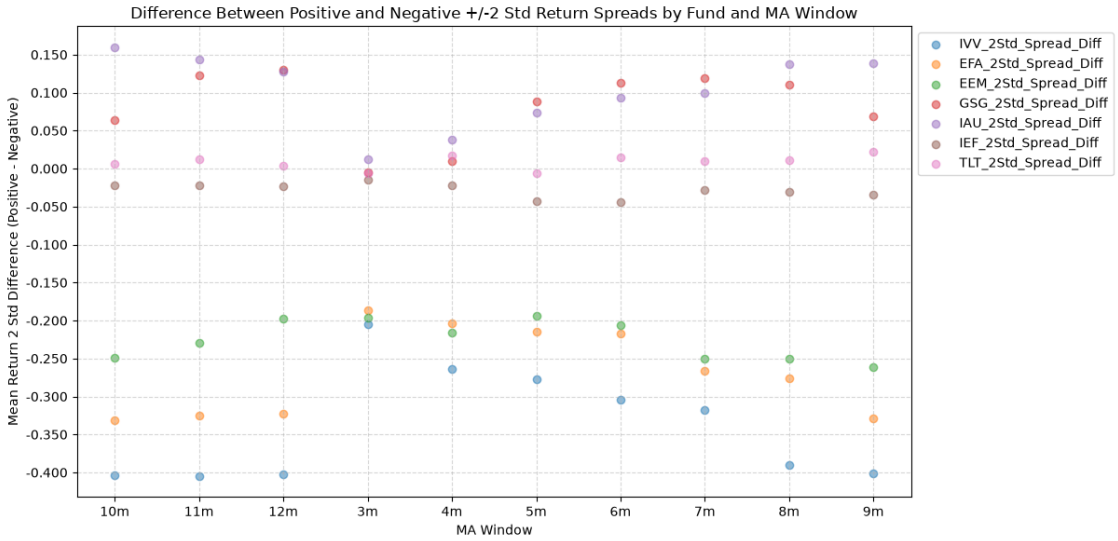
This might not be as relevant to us (from a long-only perspective), but it's still worth looking at to understand how the asset classes rank. Emerging markets tend to have more volatility and outliers, so this plot makes sense intuitively. IEF again ranks high here, but we also need to remember that IEF is the least volatile of the asset classes (by quite a large margin), which likely contributes to its high ranking.

2.6. Difference Between Positive and Negative +/- 2 Std Return Spreads by Fund and MA Window Finally, we have the difference between the 2 standard deviation spreads.

[44]:
```
for fund in fund_list:
    distribution[f"{fund}_2Std_Spread_Diff"] =␣
 ↪distribution[f"{fund}_Positive_Mean_Plus_Minus_2Std_Diff"] -␣
 ↪distribution[f"{fund}_Negative_Mean_Plus_Minus_2Std_Diff"]

plot_scatter(
    df=distribution,
    x_plot_column="MA_Window",
    y_plot_columns=["IVV_2Std_Spread_Diff",
                    "EFA_2Std_Spread_Diff",
                    "EEM_2Std_Spread_Diff",
                    "GSG_2Std_Spread_Diff",
                    "IAU_2Std_Spread_Diff",
                    "IEF_2Std_Spread_Diff",
                    "TLT_2Std_Spread_Diff"],
    title="Difference Between Positive and Negative +/-2 Std Return Spreads by␣
 ↪Fund and MA Window",
    x_label="MA Window",
    x_format="String",
    x_format_decimal_places=0,
    x_tick_spacing=1,
    x_tick_start=None,
    x_tick_rotation=0,
    y_label="Mean Return 2 Std Difference (Positive - Negative)",
    y_format="Decimal",
    y_format_decimal_places=3,
    y_tick_spacing="Auto",
    y_tick_rotation=0,
    plot_OLS_regression_line=False,
    OLS_column=None,
    plot_Ridge_regression_line=False,
    Ridge_column=None,
    plot_RidgeCV_regression_line=False,
    RidgeCV_column=None,
    regression_constant=True,
    grid=True,
    legend=True,
    legend_location="upper left",
```

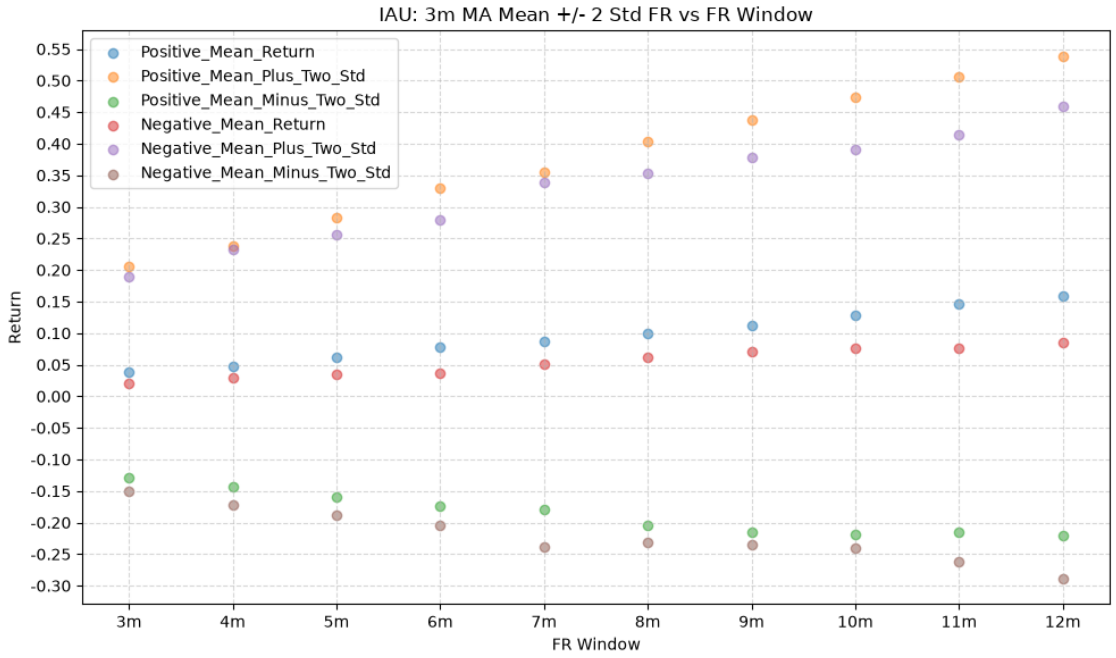
```
    legend_anchor=(1, 1),
    export_plot=False,
    plot_file_name=None,
)
```

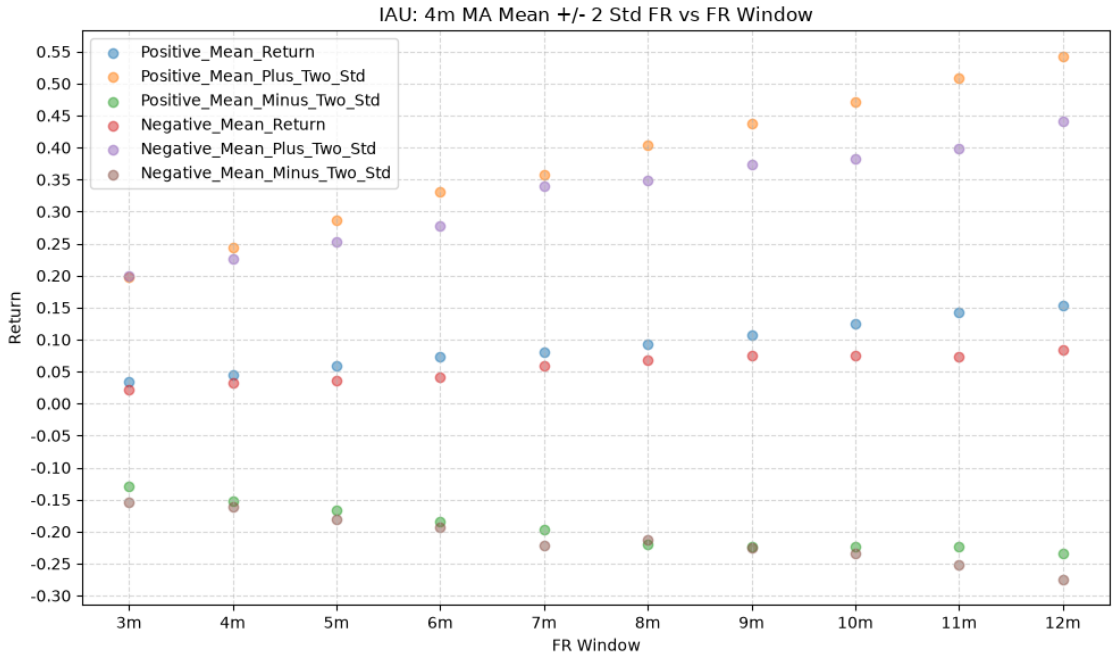


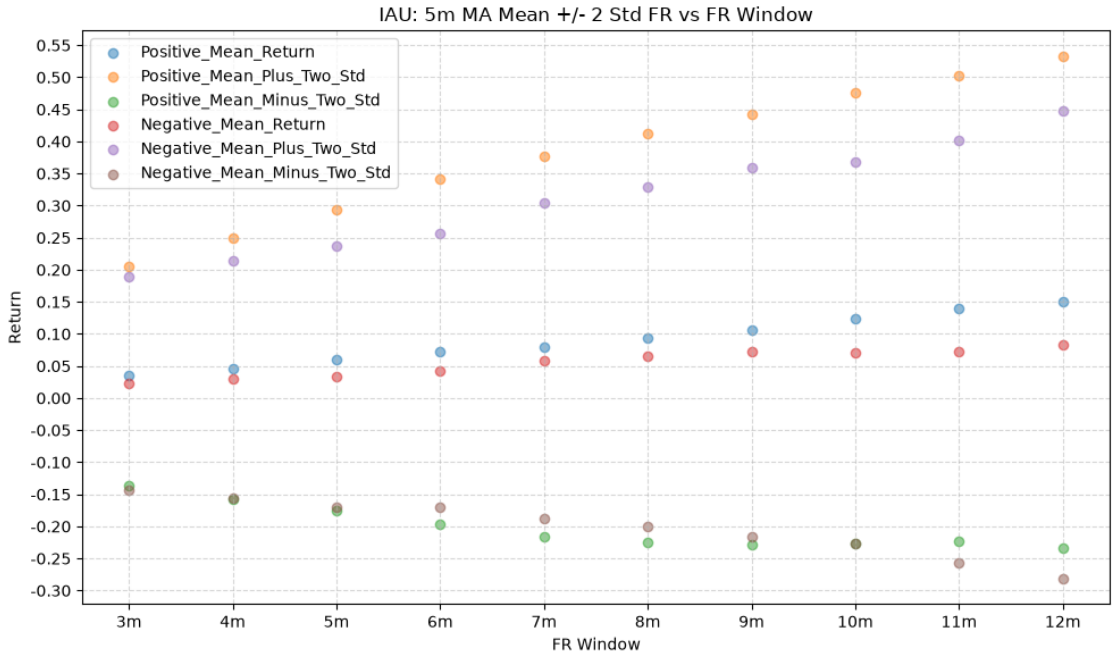
This plot really compares the kurtosis of the forward return distributions by looking at the difference between the 2 standard deviation spreads for positive and negative price-MA conditions. If these values are positive, it means that the forward return distributions have fatter tails for positive price-MA conditions relative to negative price-MA conditions, and the opposite is true if the values are negative. If the distributions are similar, then we would expect to see values around 0. So anything <=0 is good here (with a long-only perspective), which brings us back to gold, which is positive. Running the plots for gold:

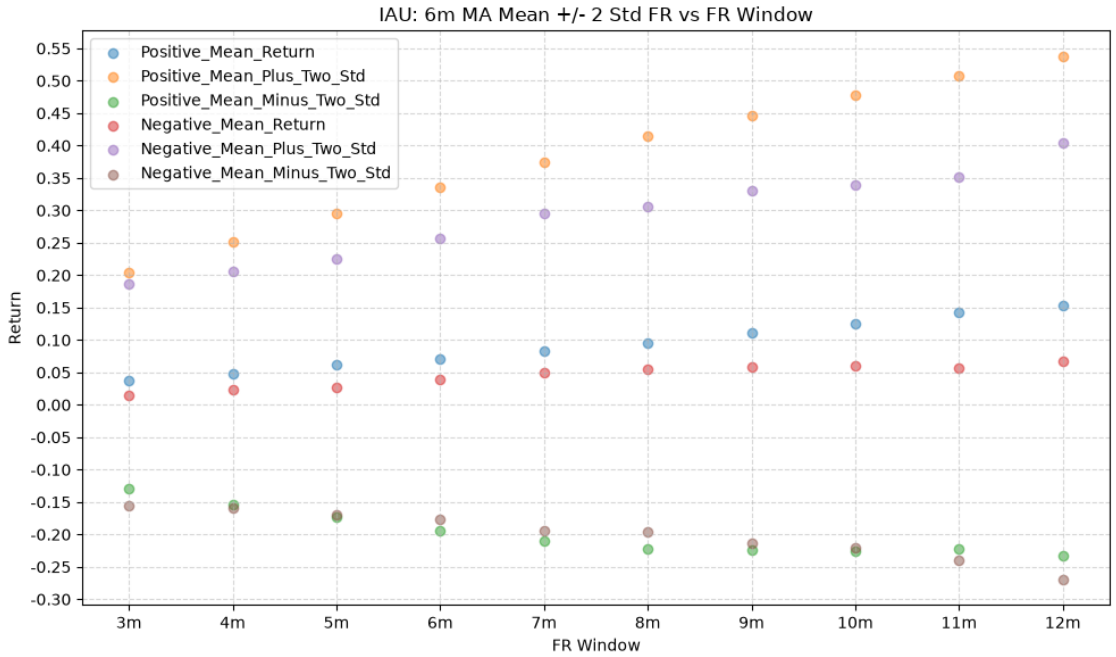
[45]:
```
for fund, data in fund_data.items():
    if fund not in ["IAU"]:
        continue
    for ma_label, ma_window in ma_windows.items():
        plot_scatter(
            df=ma_prediction_results[(ma_prediction_results["Fund"] == fund) &␣
 ↪(ma_prediction_results["MA_Window"] == ma_label)],
            x_plot_column="FR_Window",
            y_plot_columns=["Positive_Mean_Return",␣
 ↪"Positive_Mean_Plus_Two_Std", "Positive_Mean_Minus_Two_Std",␣
 ↪"Negative_Mean_Return", "Negative_Mean_Plus_Two_Std",␣
 ↪"Negative_Mean_Minus_Two_Std"],
            title=f"{fund}: {ma_label} MA Mean +/- 2 Std FR vs FR Window",
            x_label="FR Window",
            x_format="String",
            x_format_decimal_places=0,
```

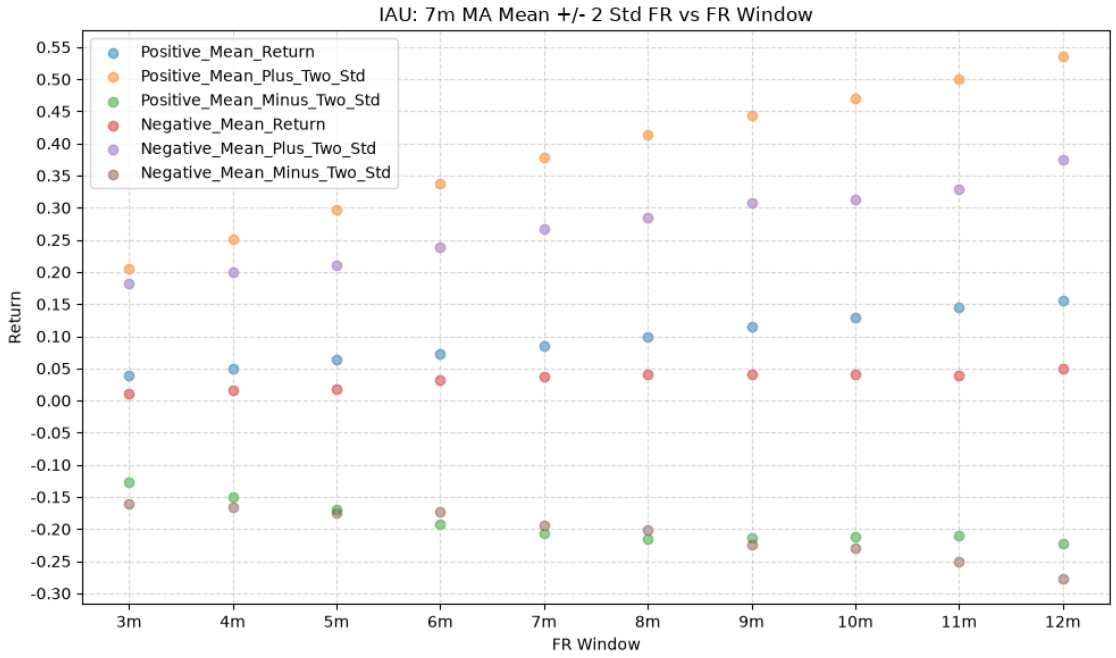
```
        x_tick_spacing=1,
        x_tick_start=None,
        x_tick_rotation=0,
        y_label="Return",
        y_format="Decimal",
        y_format_decimal_places=2,
        y_tick_spacing="Auto",
        y_tick_rotation=0,
        plot_OLS_regression_line=False,
        OLS_column=None,
        plot_Ridge_regression_line=False,
        Ridge_column=None,
        plot_RidgeCV_regression_line=False,
        RidgeCV_column=None,
        regression_constant=True,
        grid=True,
        legend=True,
        export_plot=False,
        plot_file_name=None,
    )
```

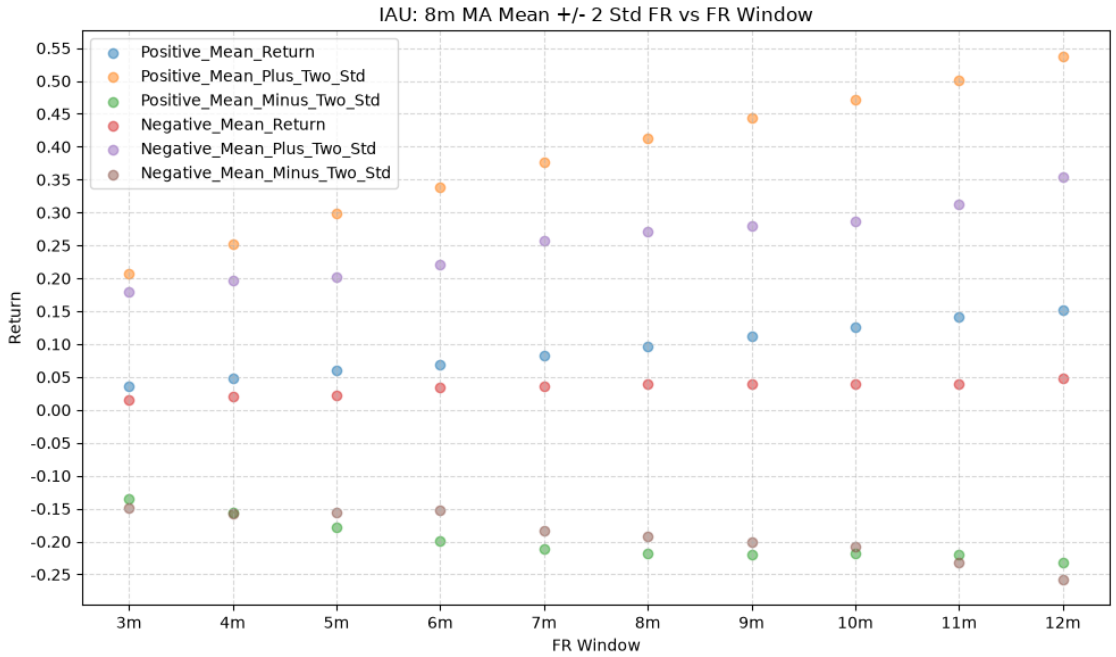


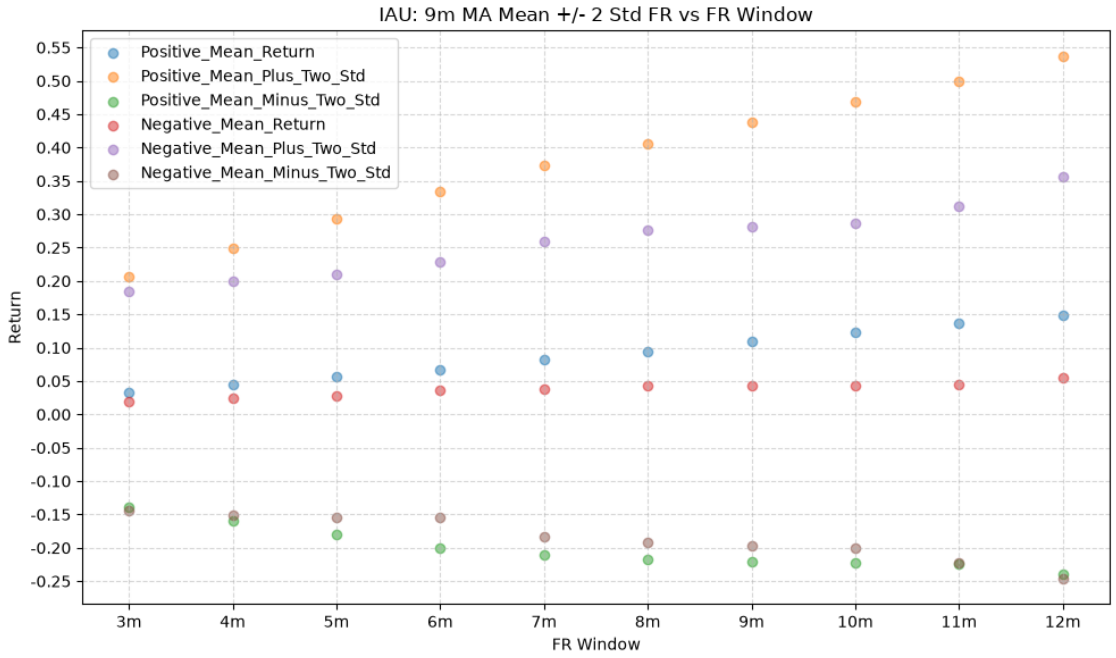

IAU: 4m MA Mean +/- 2 Std FR vs FR Window
Positive_Mean_Return
Positive_Mean_Plus_Two_Std
Positive_Mean_Minus_Two_Std
Negative_Mean_Return
Negative_Mean_Plus_Two_Std
Negative_Mean_Minus_Two_Std
Return
FR Window
3m
4m
5m
6m
7m
8m
9m
10m
11m
12m

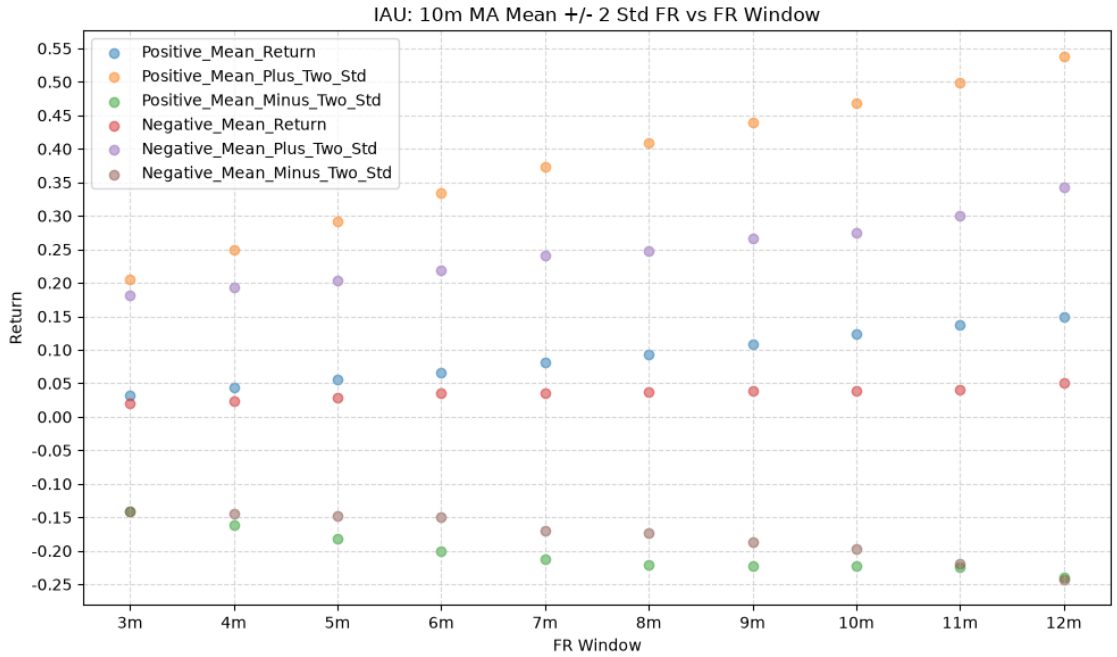

IAU: 5m MA Mean +/- 2 Std FR vs FR Window
Positive_Mean_Return
Positive_Mean_Plus_Two_Std
Positive_Mean_Minus_Two_Std
Negative_Mean_Return
Negative_Mean_Plus_Two_Std
Negative_Mean_Minus_Two_Std
Return
FR Window
3m
4m
5m
6m
7m
8m
9m
10m
11m
12m

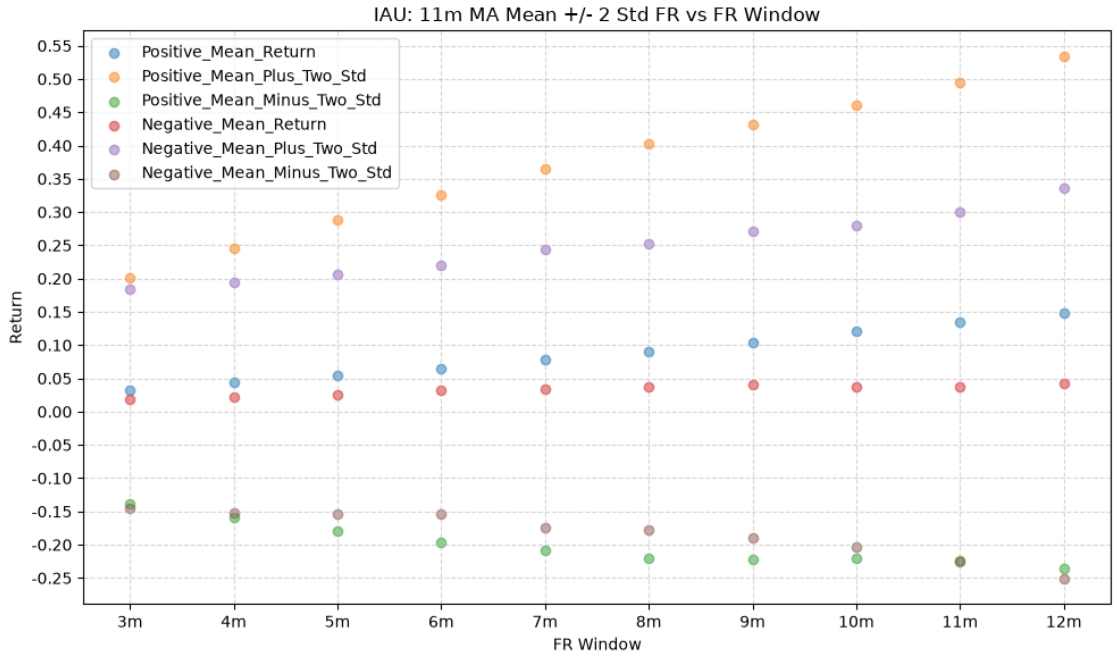

IAU: 6m MA Mean +/- 2 Std FR vs FR Window
Positive_Mean_Return
Positive_Mean_Plus_Two_Std
Positive_Mean_Minus_Two_Std
Negative_Mean_Return
Negative_Mean_Plus_Two_Std
Negative_Mean_Minus_Two_Std
Return
FR Window
3m
4m
5m
6m
7m
8m
9m
10m
11m
12m

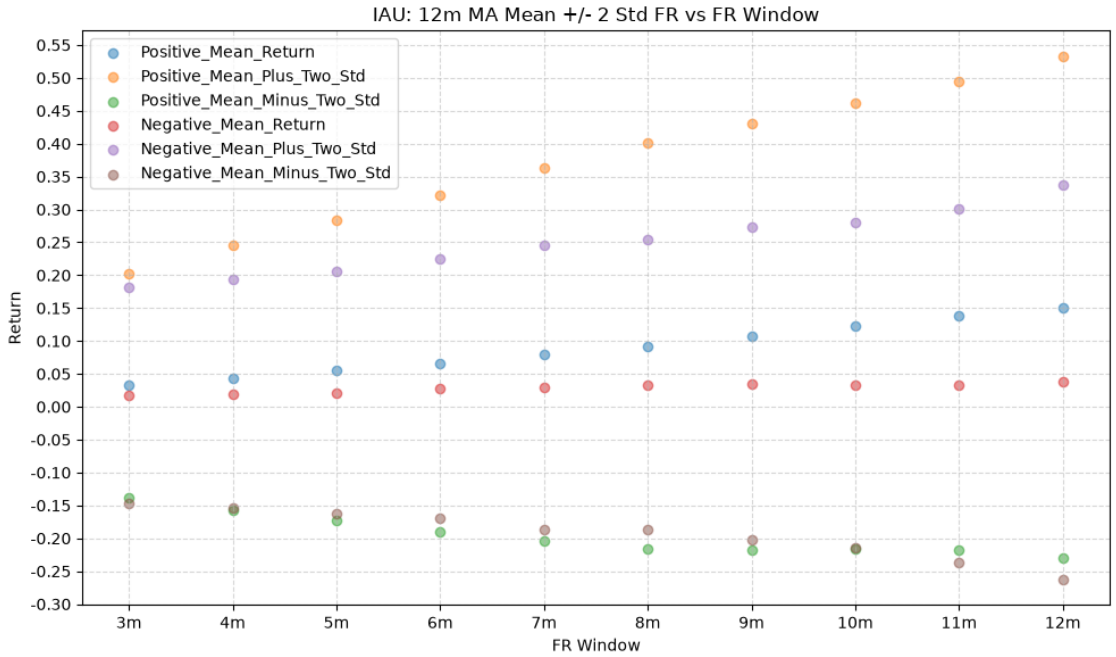

IAU: 7m MA Mean +/- 2 Std FR vs FR Window
Positive_Mean_Return
Positive_Mean_Plus_Two_Std
Positive_Mean_Minus_Two_Std
Negative_Mean_Return
Negative_Mean_Plus_Two_Std
Negative_Mean_Minus_Two_Std
Return
FR Window
3m
4m
5m
6m
7m
8m
9m
10m
11m
12m


IAU: 8m MA Mean +/- 2 Std FR vs FR Window
Positive_Mean_Return
Positive_Mean_Plus_Two_Std
Positive_Mean_Minus_Two_Std
Negative_Mean_Return
Negative_Mean_Plus_Two_Std
Negative_Mean_Minus_Two_Std
Return
FR Window
3m
4m
5m
6m
7m
8m
9m
10m
11m
12m


IAU: 9m MA Mean +/- 2 Std FR vs FR Window
Positive_Mean_Return
Positive_Mean_Plus_Two_Std
Positive_Mean_Minus_Two_Std
Negative_Mean_Return
Negative_Mean_Plus_Two_Std
Negative_Mean_Minus_Two_Std
Return
FR Window
3m
4m
5m
6m
7m
8m
9m
10m
11m
12m


IAU: 10m MA Mean +/- 2 Std FR vs FR Window
Positive_Mean_Return
Positive_Mean_Plus_Two_Std
Positive_Mean_Minus_Two_Std
Negative_Mean_Return
Negative_Mean_Plus_Two_Std
Negative_Mean_Minus_Two_Std
Return
FR Window
3m
4m
5m
6m
7m
8m
9m
10m
11m
12m


IAU: 11m MA Mean +/- 2 Std FR vs FR Window
Positive_Mean_Return
Positive_Mean_Plus_Two_Std
Positive_Mean_Minus_Two_Std
Negative_Mean_Return
Negative_Mean_Plus_Two_Std
Negative_Mean_Minus_Two_Std
Return
FR Window
3m
4m
5m
6m
7m
8m
9m
10m
11m
12m



We can see that the mean +2 std return line is for the positive price-MA condition is actually above the mean +2 std return line for the negative price-MA condition and by quite some margin, which is different than any of the other funds. The explanation? The forward positive return tail is fatter, which is actually a more desirable condition. While we'd like a tight distribution, we also will accept a fatter positive tail, as that means that the forward returns are more likely to be positive and larger than they would be otherwise.

0.11 Conclusion

This analysis shows that the predictive power of moving averages varies across different asset classes. We found that no asset class or individual fund showed any consistent predictive power across all conditions. However, quoting from section 1.5:

> Here we have some interesting results that could be useful. For the stock funds (IVV, EFA, EEM) and gold (GLD), the MA predictions tend to provide a slight improvenment over the naive benchmark, somewhere between 1-6%.
>
> For the commodity fund (GSG) and long-term bonds (TLT), however, the MA predictions generally underperform the naive benchmark, indicating that the moving average signal may not be as effective for these asset classes when predicting the direction of future returns.

The equity funds and gold did show some level of predictive power when considering only the positive price-MA conditions, and these findings suggest that investors may benefit from using the price-MA relationship when developing trend following strategies.

0.12 Future Investigation

In a future post, we will explore developing trend following strategies based on the price-MA relationship and assess their effectiveness across different asset classes, economic conditions, and market regimes.

0.13 References

Nothing here yet.